TECHNOLOGICAL UNIVERSITY DUBLIN



Project Report of ThreadUD

by

Julia Udvari, Nikita Ciuciu, David Thornton

Project Report

in the

School of Computing
Department of Informatics and Cyber Security

May 2025

Declaration of Authorship

We, Julia Udvari, Nikita Ciuciu and David Thornton, declare that this thesis titled, 'Project Report of ThreadUD' and the work presented in it are our own. We confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: Julia, Nikita, David

Date: 15/05/2025

TECHNOLOGICAL UNIVERSITY DUBLIN

Abstract

School of Computing
Department of Informatics and Cyber Security

Bachelor of Science in Informatics and Cyber Security

by Julia Udvari, Nikita Ciuciu, David Thornton

This report outlines the development of an e-learning application "ThreadUD" that will be utilized by students of TU Dublin to enhance and promote academic conversations and peer-learning. The project aims to fill in the gap in collaborative academic resources by creating a platform for questions, ideas, feedback and discussions that help the students. The app is developed using React Native was chosen for its cross-platform capabilities and modern feel. The team follows Agile methodology for the software development cycle, as it has an iterative and adaptive progress. Key features will be real-time thread posts, personalized notifications and strong account management to make it usable and secure. The current research is supported by an extensive review of the literature related to UI/UX design, how other technological frameworks compare to React Native and how technology can develop collaboration, improve learning outcomes and build a connected community for academic conversation. ...

Contents

Declaration of Authorship i

Abstract ii

1 Introduction 1
1.1 Overview of system . . . 1
1.1.1 Background and Motivation . . . 2
1.1.2 Aims and Objectives . . . 3
1.1.2.1 Aims . . . 3
1.1.2.2 Objectives . . . 3
1.2 Methodology . . . 5
1.2.1 Overiew of chosen method . . . 5
1.2.2 Justification . . . 6
1.3 Groupware Tools . . . 9
1.3.1 File and Code Sharing . . . 9
1.3.1.1 GitHub . . . 9
1.3.1.2 Codeberg . . . 11
1.3.1.3 Chosen Tool: GitHub . . . 12
1.3.1.4 Comparison with Other Groupware Tools . . . 12
1.3.2 Communication . . . 13
1.3.2.1 Microsoft Teams . . . 13
1.3.2.2 Discord . . . 15
1.3.2.3 Chosen Tool: Discord . . . 16
1.3.2.4 Comparison with Other Communication Tools . . . 16
1.3.3 Document Collaboration and Sharing . . . 17
1.3.3.1 OneDrive . . . 17
1.3.3.2 Google Workspace . . . 18
1.3.3.3 Tool Chosen: OneDrive . . . 19
1.3.3.4 Comparison with Other Document Collaboration Platforms . . . 19
1.3.4 Writing Tools . . . 20
1.3.4.1 Microsoft Word Overview . . . 20
1.3.4.2 Google Docs . . . 21
1.3.4.3 Chosen Tool: Microsoft Word . . . 22

1.3.4.4 Comparison with Other Writing Platforms 22
1.3.5 Project Management 23
1.3.5.1 Microsoft Project 23
1.3.5.2 Trello 24
1.3.5.3 Chosen Tool: Microsoft Project 25
1.3.5.4 Comparison with Other Project Management Platforms . 25
1.4 Version Control Software 26
1.4.1 How the team will use GitHub 26

2 Literature Reviews **34**
2.1 Introduction 34
2.2 The importance of academic conversation and peer learning and how this can be enhanced using technology 35
2.2.1 What is academic conversation and peer learning? Why are they important? 35
2.2.2 Why are they important? 35
2.2.3 How can technology be used as a tool for learning? 37
2.2.4 How can technology be used as a tool for academic conversation and peer learning? 37
2.2.5 How is using technology in this way beneficial? How does it enhance peer learning and academic conversation? 37
2.2.6 Summary of Findings 39
2.3 UX and UI Considerations in Mobile Learning Apps 40
2.3.1 Introduction 40
2.3.2 User Interface (UI) 40
2.3.3 User Experience (UX) 43
2.3.4 Summary of Findings 46
2.4 Why is React Native better than others for this type of project? Technologies and Frameworks 47
2.4.1 Introduction 47
2.4.2 Overview of React Native 47
2.4.3 Comparative Evaluation with Other Technologies 49
2.4.4 Justification for Choosing React Native 52
2.4.5 Summary of Findings 54
2.5 Conclusion 55

3 System Analysis and Design **56**
3.1 High-level Use Cases (Specifications) 56
3.1.1 User Registration 56
3.1.2 User Login 57
3.1.3 View Home Page 58
3.1.4 Join a Thread 59
3.1.5 Create a New Thread 61
3.1.6 Make a Post 62
3.1.7 Like a Comment 63
3.1.8 Comment on a Post 64
3.1.9 Search for Threads 66
3.1.10 Update Profile 67

3.1.11 Receive Push Notifications . . . 68
3.1.12 Log Out . . . 69
3.1.13 Moderator Manages Posts . . . 70
3.1.14 View my Posts . . . 72
3.1.15 View my Comments . . . 72
3.1.16 Reply to a Comment . . . 73
3.1.17 Like a Post . . . 74
3.1.18 Share a Post . . . 75
3.1.19 Forgot Password . . . 76
3.2 Use Case Diagrams . . . 78
3.3 System Design . . . 81
3.3.1 System Diagrams . . . 81
3.3.1.1 Entity Relationship Diagram . . . 81
3.3.1.2 UML Diagram . . . 81
3.3.2 High-end Detailed Wireframes . . . 81

4 Implementation of System 83
4.1 Development Methodology . . . 83
4.2 Tools and Technologies . . . 83
4.3 Front-end Implementation . . . 84
4.4 Back-end Implementation . . . 84
4.5 Data Models . . . 84
4.6 Security and Moderation . . . 85

5 Testing and Evaluation 86
5.1 Peer Reviewed Test-Driven Development . . . 86
5.2 User Testing . . . 87
5.3 Stress Testing . . . 87
5.4 User Interface and Experience Testing . . . 91
5.5 Requirements/Use Case Testing . . . 96
5.5.1 Feature: Login . . . 97
5.5.1.1 Scenario: Successfully login normal user . . . 97
5.5.1.2 Scenario: Mismatch username and password . . . 97
5.5.1.3 Scenario: Unregistered email . . . 97
5.5.1.4 Scenario: Successfully login admin user . . . 98
5.5.2 Feature: Authenticate User . . . 98
5.5.2.1 Scenario: Successfully authenticated . . . 98
5.5.2.2 Scenario: Invalid code entered . . . 98
5.5.3 Feature: Register . . . 98
5.5.3.1 Scenario: Successfully Register user . . . 99
5.5.3.2 Scenario: Register with invalid information . . . 99
5.5.3.3 Scenario: Register with email already in use . . . 99
5.5.4 Feature: Confirm register . . . 99
5.5.4.1 Scenario: Successfully confirm register . . . 99
5.5.4.2 Scenario: Invalid code given . . . 100
5.5.5 Feature: Log out . . . 100
5.5.5.1 Scenario: Successfully log out . . . 100

5.5.5.2 Scenario: User isn't logged in . . . 100
5.5.6 Feature: Login interaction when not logged in . . . 100
5.5.6.1 Scenario: User isn't logged in . . . 100
5.5.7 Feature: Like/unlike Post . . . 101
5.5.7.1 Scenario: User wants to like post . . . 101
5.5.7.2 Scenario: User wants to unlike post . . . 101
5.5.7.3 Scenario: User attempts to like post when logged out . . . 101
5.5.8 Feature: Leave Comment/Reply . . . 101
5.5.8.1 Scenario: Successfully leave comment . . . 101
5.5.8.2 Scenario: Comment flagged . . . 102
5.5.8.3 Scenario: No comment is entered / whitespace entered . . . 102
5.5.8.4 Scenario: Attempts to comment when logged out . . . 102
5.5.9 Feature: Delete comment/reply . . . 102
5.5.9.1 Scenario: Successfully delete comment/reply . . . 102
5.5.9.2 Scenario: Comment does not belong to user . . . 102
5.5.10 Feature: Make Post . . . 103
5.5.10.1 Scenario: Successfully make post . . . 103
5.5.10.2 Scenario: Post flagged . . . 103
5.5.10.3 Scenario: Invalid data entered . . . 103
5.5.10.4 Scenario: Attempts to make post when logged out . . . 103
5.5.11 Feature: Make Thread . . . 104
5.5.11.1 Scenario: Successfully make thread . . . 104
5.5.11.2 Scenario: Invalid data entered . . . 104
5.5.11.3 Scenario: Thread name in use . . . 104
5.5.11.4 Scenario: Attempts to make thread when logged out . . . 104
5.5.12 Feature: Update Profile . . . 105
5.5.12.1 Scenario: Successfully update profile . . . 105
5.5.12.2 Scenario: Invalid data entered . . . 105
5.5.12.3 Scenario: No new data entered . . . 105
5.5.12.4 Scenario: Attempts to update profile when logged out . . . 105
5.5.13 Feature: Manage flagged Post/Comment . . . 105
5.5.13.1 Scenario: Successfully manage content . . . 106
5.5.13.2 Scenario: User isn't logged in as admin user . . . 106
5.6 Additional Testing of Moderator and Bad Words Filter . . . 106
5.7 Evaluation . . . 106

6 Conclusions and Future Work 108
6.1 Summary of Project Outcomes . . . 108
6.2 Achievement Against Objectives . . . 108
6.3 Limitations and Lessons Learned . . . 109
6.4 Further Work . . . 109

7 Personal Reflections on Project Experiences 110
7.1 David . . . 110
7.2 Julia . . . 111
7.3 Nikita . . . 111

A Detailed Project Plan **113**
A.1 Project Timeline . . . 113
A.2 Task Allocation Breakdown . . . 113

B Project Diary (parts 1 and 2) **115**
B.1 Onedrive Link . . . 115

C Code listings **116**
C.1 GitHub Repository Link . . . 116
C.2 Backend Server Address: . . . 116
C.3 Database modules used with mongoose: . . . 116

Bibliography **118**

List of References **118**

List of Figures **126**

Chapter 1

Introduction

1.1 Overview of system

This project proposal for an e-learning app, specifically aimed at TU Dublin students, will promote academic conversation between students and allow them to share their ideas, experiences, and questions with others. This report will outline why this project is needed, the aims and objectives, and what methods and tools will be used to develop the project. It also includes a detailed literature review on several topics related to this project, including research about:

- What technologies the project will be developed in and why they are advantageous compared to others.
- The importance of good user interface design on an e-learning app such as this and what previous research has found makes a good user experience.
- The importance of academic conversation and peer learning and how this can be enhanced using technology.

The research done in this literature review led the team to come to several conclusions, including that:

- For the app specified, and considering the previous experience of the team, developing the app using the react native framework would be best suited as it is the most modern option that is not platform dependent meaning it can be used to make IOS or Android apps while at the same time ensuring the work is still done most prominently by the team, instead of developing with frameworks such as

Flutter which are much less hands on, ensuring not only that the app is developed as intended but also that the team learns more from the process.

- Just developing a functional app won't be enough, and user interface design and the general user experience will play a huge role in the success of the app. The team will need to fully embrace user-oriented design and technological innovations to create a successful app.
- Peer learning and academic conversation are pivotal parts of any learning experience and provide many benefits more traditional learning styles don't. Furthermore, using technology to enhance this experience has numerous advantages, including providing learners with a more flexible learning environment, connecting learners with those they wouldn't have been able to reach previously, and providing learners with a new platform to share their ideas and teach others.

Once this research was complete and the team had a better understanding of the importance of the project and how it should be developed, they were able to further plan the development of the project, deciding to use an agile methodology during development, to use GitHub as a version control system and to use several other groupware tools throughout development.

1.1.1 Background and Motivation

This research paper is being conducted as the team felt that a student learning and academic conversation platform for students could be a great tool for student learning. In the initial planning phases, the team knew that they wanted to create some kind of project that contributed to e-learning, and after further planning and research decided to create the peer learning forum-based app. The team decided to create this project as there are not any resources like this available at TUD for students. During this planning phase, the team realised that even though there were plenty of learning resources available to students, there was a lack of communication and in particular peer learning opportunities for students.

Once this problem had been identified, the team then had to ask the question, how could they breach this gap in group learning and communication resources? Breaching this gap is the goal of the forum-based app the team will develop.

Before developing an app to meet this goal, the team first had to do research into existing resources like this and how they would develop the app. This proposal document was the result of this research, and will also highlight the benefits of creating the specified app.

1.1.2 Aims and Objectives

1.1.2.1 Aims

The project's scope is to design and develop an online interactive platform especially suited to students' needs for cooperation and peer academic support. The system is aimed at serving as a virtual community where students can ask questions, share their ideas, and provide feedback concerning coursework and topics related to modules taken by them. In most learning environments, there is an evident lack of a specific area, outside of the classroom, where students can discuss their courses, get peer advice, and provide solutions to academic problems. Yet this application will bridge that chasm by acting as that communal solution helping to foster learning through peer assistance (Huijser et al., 2008).

Hence, the main objective is to provide an accessible and smooth environment that would enable the students to be more communicative about academic discussions, establish relationships with other students, and learn from their diverse experiences in the process. In turn, the app will let students feel connected with studies through supportive networks, reduce social isolation, and facilitate a culture of collaborative learning. It also tends to make learning more accessible because, through this forum, students can reach for help at any time, whereby academic discussions are held without time and space constraints (Razak & See, 2010).

Ultimately, this project connects the gulf between a student and their academic community to make them feel even more connected, supported, and empowered through college life. Specific, feature-rich, user-friendly design may leverage the benefits of academic collaboration, timely academic support, and a sense of belonging for students.

1.1.2.2 Objectives

Below are specific and measurable terms for what has been done to realize the goals of this aim. Create an interface that makes it easy for students to create and participate in discussions, sorted by module or academic topic. The objective here was to develop an intuitive web-based user interface that facilitates students in navigating between different modules and threads. This is to ensure that students easily locate discussions relevant to their subject matter, post questions, and respond to these questions (Razak & See, 2010). This objective entails creating modules or, in this case, "threads" for each academic subject matter. Users can define their discussion to topics relevant to their coursework.

In this regard, it is necessary to enable real-time chats to support live discussion and immediate feedback among students. Real-time communication is indispensable in the establishment of a dynamic, interactive community. The inclusion of chat functionality will allow the students to converse immediately with urgent academic nature questions or intricate problems they are working out at the time (Tang et al., 2022). This will be critical to facilitate timely peer-to-peer support, thereby turning the platform interactive and responsive to users.

Provide a notification of activity when events occur within the module/thread they are following. There are push notifications to keep students updated on the latest replies, likes, or posts within their selected modules, hence keeping them naively informed and continually involved in hopes of not missing any precious insight or contribution from another student (Abuhassna et al., 2020).

Implement account administration, so users can customize their experience:

Personalization of accounts will allow students to set the application to their needs, following only modules or topics relevant to their academic journey. This ambition improves not only the user experience but arguably makes the app more useful as it saves the students from having to filter out information irrelevant to their studies (Chandra & Palvia, 2021).

Usability testing and feedback sessions:

Usability testing with students will provide the feedback required for any improvements, and it will keep the app student centered. It is an important objective to be achieved because it would create a product in which students would feel the need and help them with their academic problems (Sinclair, 2017).

If resources allow, the project timeline should provide for a beta launch of the application to capture user feedback in further tuning of the app for its final release. Beta launch can be considered the quintessential test for larger-scale testing of the platform. In this regard, feedback from actual users of the app during this stage gives insight into how it functions in the real-world environment. An objective that will come in handy to fine-tune the app and thus fix problems before release, hence ensuring a smoother, glitch-free experience for all users (Trimble & Fan, 2023).

The project would thereby achieve its aim and objectives by providing an engaging, interactive, and accessible academic platform for effective peer-to-peer learning and student engagement. Each objective is crafted to address a major component of the app, from user interface and engagement to personalization and continuous improvement, to

ensure that the app serves the needs of the student community (Tibingana-Ahimbisibwe et al., 2022).

1.2 Methodology

1.2.1 Overiew of chosen method

The Agile methodology will be used in the development of this project, to be employed by the team. Agile is an iterative and adaptive methodology toward project management and software development that embraces constant changes in requirements, coupled with incremental improvements, toward meeting user needs more effectively (Nigam & Gupta, 2017).



FIGURE 1: (netsolutions, June 2022), Agile development methodology cycle

Why Agile was Chosen by the Team

Agile will most perfectly suit this particular case of a project due to its flexibility and user-oriented nature: It allows for immediate reactions of the team to a new idea, feedback, or metamorphosis of the project's needs. In projects that need collaboration and consistent updates it's the structured yet adaptive nature of Agile that provides an avenue for frequent check-ins, review cycles and continuous improvement (Hansen & Kristoffersen, 2024).

What is Agile?

Agile does the development iteratively in small manageable steps or blocks called "sprints," which generally last from two to four weeks. This allows the team to build upon the project incrementally by focusing on only certain functionalities within one sprint and adapting as necessary. After the results of each sprint are analysed with the stakeholders

at the end, realignment of goals is performed so that future projects better conform to current user needs and the purposes of the project at hand (Nigam & Gupta, 2017).

How does Agile Work?

Agile breaks the project into a number of sprints. Each sprint commences with a planning session in which major deliverables are identified and their activities allocated. During each sprint, daily stand-ups serve to get updates, solve problems, and coordinate. At the end of the sprint there is a review meeting to present the iteration of the latest product, with feedback to guide the focus of the next sprint (Hansen & Kristoffersen, 2024).

1.2.2 Justification

It has been chosen because the Agile methodology has a few distinct advantages over other popular approaches in software development, including Scrum, Waterfall, Spiral, DevOps, and Lean Development. Agile's flexibility and iteration make it able to adapt very fast in case of changes, which makes Agile ideal for projects with either evolving requirements or frequent stakeholder feedback (Rigby et al., 2018).

Agile vs Scrum

While Scrum is a framework within the Agile methodology, Agile provides a more general, adaptive approach. Scrum operates within a more rigid framework of roles, ceremonies, and artifacts. These can sometimes render it inelastic if adaptation needs to occur frequently within a project that falls outside of the fixed sprint structures of Scrum. Agile is easier to adapt to in terms of priority timeline and goals of the project, especially when projects face high uncertainty or requirements in evolution (Rigby et al., 2018).

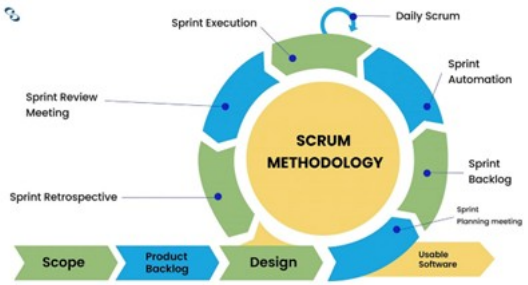


FIGURE 2: (Maria, 2024) Scrum development methodology cycle)

Agile vs Waterfall

Projects that require the Waterfall methodology are those in which the requirements are fairly well-defined and understood, and where change is less likely to occur during the project. In contrast, Waterfall can be less effective when requirements evolve because it allows little room for feedback and iterative progress, unlike Agile which has an iterative process that enables continuous feedback and adjustment to reduce the risk of costly changes at the end of the development cycle. This makes Agile more adaptable to projects where requirements may change over time, unlike the fixed structure of Waterfall (Stojanov et al., 2015; G2, 2024).

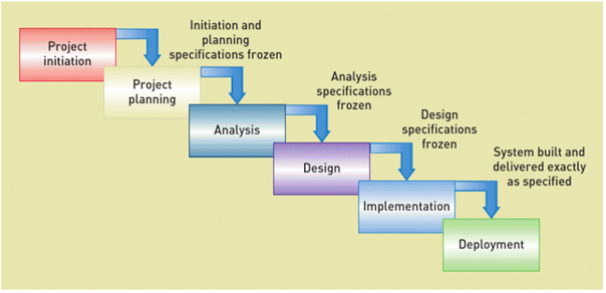


FIGURE 3: (Maria, 2024), Waterfall development methodology cycle

Agile vs Spiral

The Spiral methodology combines the traits of iterative and Waterfall approaches. It is used in high-risk projects with a demand for heavy risk analysis and review within each stage of the project. Much as Spiral offers successive evaluation for risk elimination, it may be more time-consuming and complex as a result of its multi-phase staged structure. Agile, on the other hand, allows for faster delivery of functional product increments. It also enables teams to handle the risks but in a much more adaptive form rather than inflexible phases. Agile is more suitable for projects whose nature demands very frequent updates and that need to make rapid progress (Boehm & Hansen, 2001).

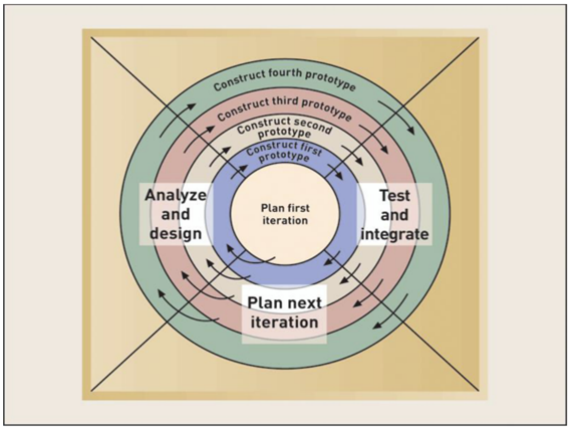


FIGURE 4: (Maria, 2024), Spiral development methodology cycle

Agile vs DevOps Methodology

Whereas DevOps is about knitting development and operations together for continuous delivery and deployment, most often aimed at rapid release cycles or better coordination between the development and operations teams, the more general goal of DevOps is infrastructure-oriented, aimed at the optimization of deployment and delivery rather than the development itself. In this vein, Agile is based on iterative development and the range of adaptation to user needs, fitting better with functional requirements that have been in evolution, as opposed to, for example, infrastructure or deployment-centric projects (Leite et al., 2019).

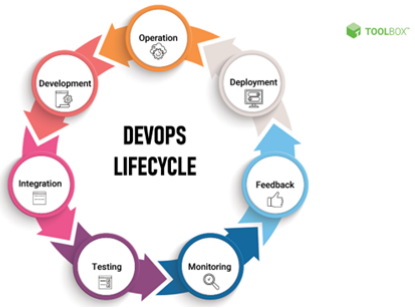


FIGURE 5: (spiceworks, March 2022), DevOps development methodology cycle

Agile vs Lean Development

The goal of Lean Development is the maximization of value by minimization of wastes. It concentrates on efficiency and quickness in each stage of the development process. While Lean can also share the same regard for flexibility and continuous improvement with Agile, Agile's structured iteration cycles contrastingly offer a more focused context of collaboration, feedback, and regular goal setting. This kind of iterative structure makes Agile particularly fitting in complex projects with diverse, changing requirements, while Lean has special value in a context where efficiency and waste reduction are crucial (Poppendieck & Cusumano, 2012).



FIGURE 6: (instinctools, April 2017), Lean development methodology cycle

1.3 Groupware Tools

1.3.1 File and Code Sharing

1.3.1.1 GitHub

GitHub is a web-based platform, probably the most famous for code hosting and collaboration (Lanubile et al., 2010). It is owned by Microsoft. GitHub supports the Git version control system and is extensively used both for open source as well as private projects. It boasts of an intuitive user interface, amazing community support, and strong integration options for CI/CD, project management, and documentation (Lanubile et al., 2010).

Key Features:

- Repository Hosting: Provides unlimited public repositories; private repositories come free and paid for.
- Version Control: Git-powered to track changes atomically, maintain histories of code, and branch.
- Collaboration Tools: GitHub Issues keep track of progress on tasks, GitHub Discussions for engaging with the community, and pull requests review peer code.
- CI/CD: Native GitHub Actions power continuous integration and deployment pipelines.
- Integration and API: Integrates with multiple tools such as Slack and Asana, and IDEs, while having a REST API with GraphQL to customize automation.
- Community and Documentation: GitHub Pages for documentation and large-scale community engagement are ideal for open-source projects.

Why GitHub for This Project?

GitHub stands out because of extensive community support, ease of use, and broad functionality (Lanubile et al., 2010). For a student project where code sharing is involved, pull request and issue tracking systems in GitHub are very functional regarding code contribution management and also bug tracking. That means its popularity and resources make it much easier for team members to find any type of guidance they need, whether working on larger projects or learning Git (Lanubile et al., 2010).

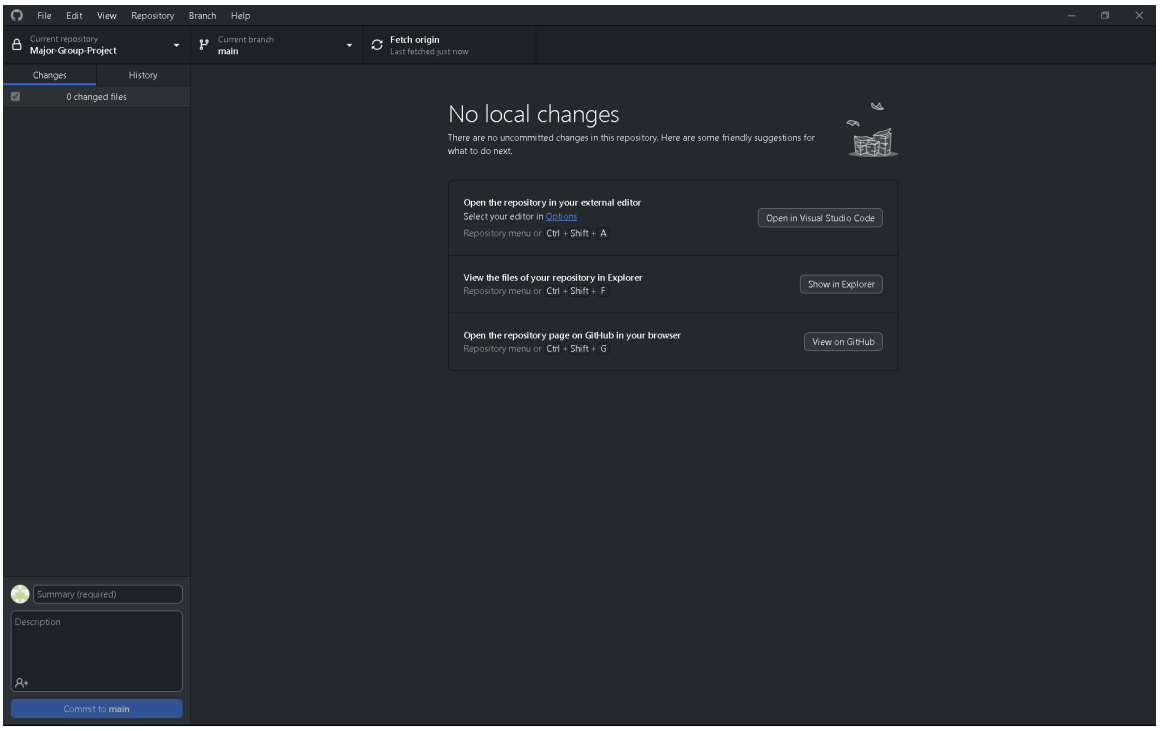


FIGURE 7: (Author's own, 2024) Screenshot of GitHub Desktop repository

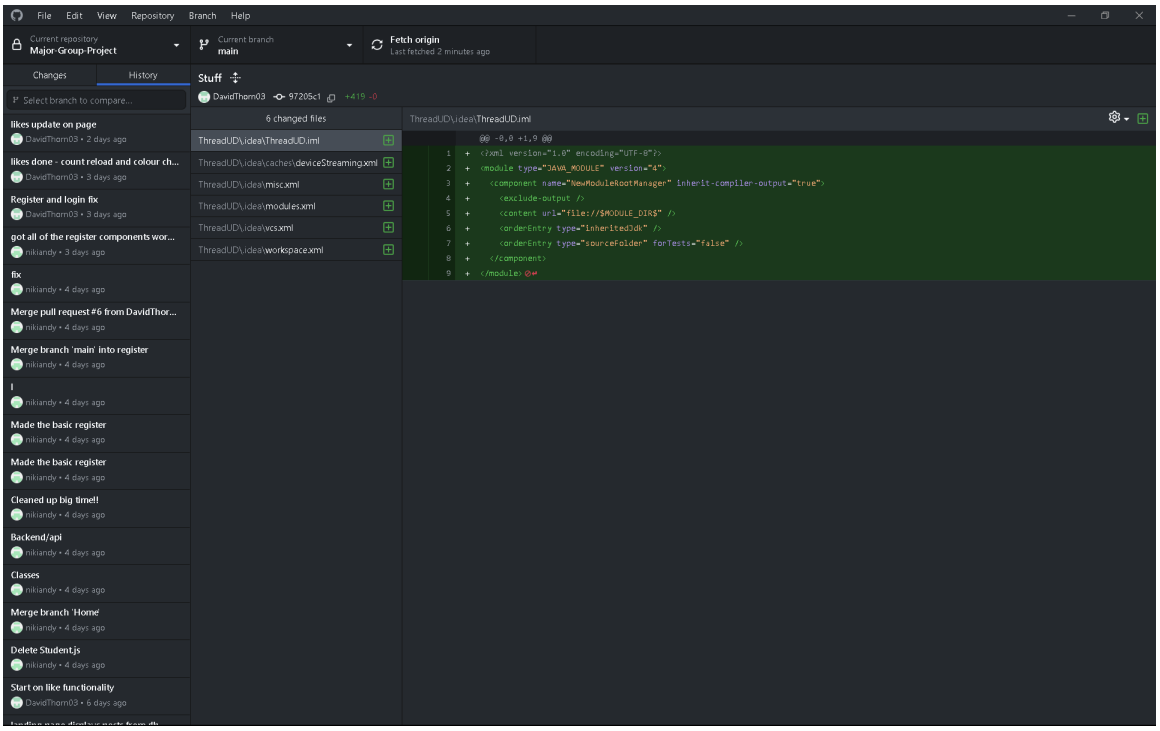


FIGURE 8: (Author's own, 2024) Screenshot of GitHub Commit History

1.3.1.2 Codeberg

Codeberg is a non-profit alternative to GitHub. Free code hosting with policies for maintaining privacy is offered. It is forked from Gitea, a self-hosted service for Git, and the interface somewhat resembles that of GitHub. Among others, it advocates for no tracking, and it does not engage in corporate funding (Batist & Roe, 2024).

Key Features:

- Repository Hosting: Free hosting for both public and private repositories is available, untracked.
- Version Control: Git-based, such as GitHub, among others, supporting all features like branching, forking, and pull requests.
- Community and Privacy: Strong support for privacy and community-driven development with no ads or corporate collection of data (Batist & Roe, 2024).
- Issue Tracking and Code Review: It has GitHub-like pull requests and issues; by default, there is no CI/CD integration.

Compared to GitHub

Codeberg, while offering a similar Git-based ecosystem, is extremely attractive for users with a high regard for privacy. Yet, it doesn't provide the large-scale ecosystem present in GitHub nor the built-in CI/CD capabilities. GitHub works quite well with third-party integrations, too, and has a larger community; moreover, it supports education-oriented features such as GitHub Classroom. This makes GitHub more fit for a university project where collaboration, learning, and community resources are important (Batist & Roe, 2024).

1.3.1.3 Chosen Tool: GitHub

With its great feature set, ease of use, and solid integrations, GitHub will be much more comfortable to use in this project. Support for GitHub Actions will make most of the work with the processes of continuous integration/continuous deployment much easier, and it provides much more resources in troubleshooting and learning. Codeberg would definitely be a good alternative if, for some reason, one is interested in keeping things private or just working on smaller projects, but GitHub's established presence and collaboration tools make it perfect for the scope and needs of this student project (Batist & Roe, 2024).

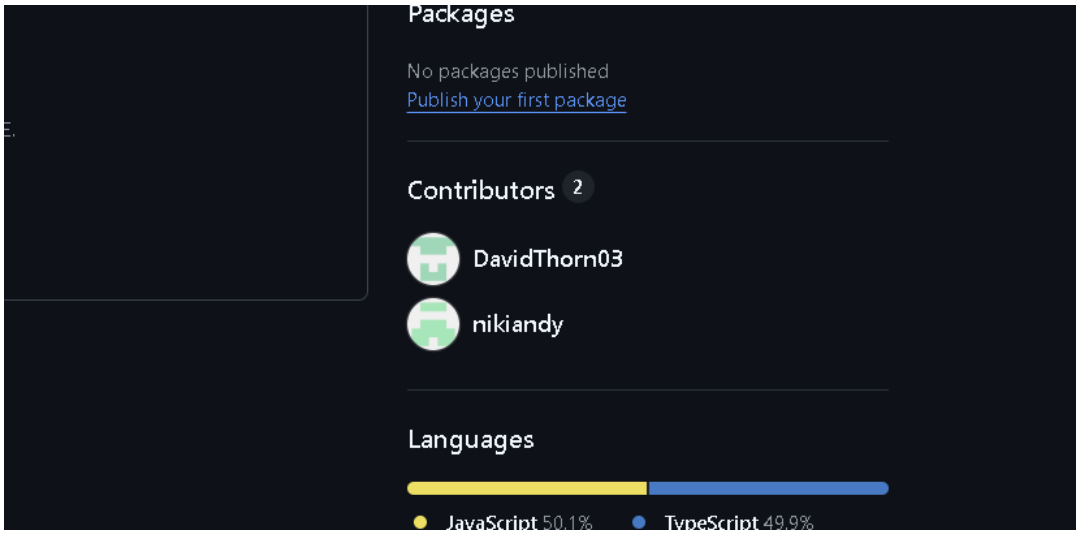


FIGURE 9: (Author's own, 2024) Screenshot of GitHub Contributors Section

1.3.1.4 Comparison with Other Groupware Tools

OneDev

Free, self-hosted, providing issue tracking, project management, and a CI/CD system. It is less intuitive than GitHub. It also doesn't have the large-scale integrations and community resources that GitHub offers features that could be very useful for educational purposes and large projects alike (Amoroso d'Aragona et al., 2023).

Radicle

It is a decentralized peer-to-peer code collaboration platform and a good standard of secure and private hosting of code (Greenberg & Roseman, 1999). However, it needs all the resources and documentation centralized into GitHub. Thus, it is less apt for projects where on its side are extensive collaboration and resources.

Google Cloud

With code in Google-managed Git repositories, services from Google Cloud are deeply integrated, thus ideal for projects deeply embedded in the Google ecosystem. However, they don't have GitHub's focus on community that will be more conducive to student learning and collaboration (Kulkarni et al., 2015).

AWS CodeCommit

AWS CodeCommit is deeply integrated into the AWS ecosystem, a lot more so than Google Cloud. It provides secure, private Git hosting along with AWS resources. It's a bit more complex for the absolute beginner and isn't about communities in open source like GitHub (Kleppmann & Beresford, 2017).

Launchpad

Focused on projects about Ubuntu, providing versioning and collaborative development; it is neither flexible nor as widely used as GitHub (Lanubile et al., 2010).

Azure DevOps

Though it has all-in-one development tools, including version control, project tracking, and CI/CD, it is very powerful in enterprise usage. Besides, it is more complicated and enterprise-driven; hence, less approachable for students' projects considering how simple and user-friendly GitHub is (Giommoni, 2018).

1.3.2 Communication

1.3.2.1 Microsoft Teams

Microsoft Teams has been one of the powerful means of communication and collaboration, be it in workplaces or educational facilities (Nabben, 2019). This workspace

stitches together messaging, video calls, file sharing, and project collaboration in one place. It is relatively comprehensive, with good connectivity to Microsoft Office applications, hence ideal for structured projects that are heavy with documents (Nabben, 2019).

Key Features:

- Groups: This allows structuring of discussions by topic or team for effective communication in diverse areas within a project.
- Video and Audio Calls: High-quality video and audio with the possibility of screen sharing allowing for remote meetings.
- Document Collaboration: Full integration with Word, Excel, PowerPoint, and OneDrive enables real-time document collaboration (Nabben, 2019).
- Task Management: It would allow the integration of Microsoft Planner with the tasks so that one can assign, manage the flow, and set deadlines.
- Controls Notification: This would be about giving the options to choose particular topics that one would want notifications for to keep updated about conversations of relevance.

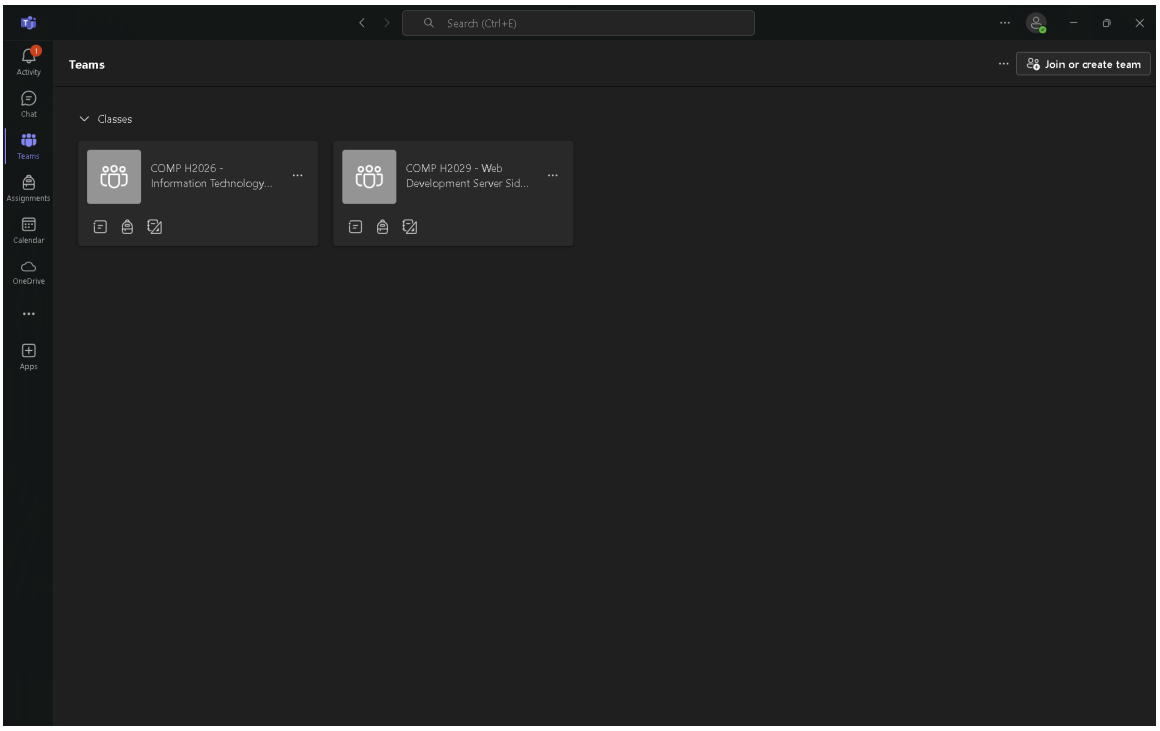


FIGURE 10: (Author's own, 2024) Screenshot of MS Teams Interface

Why Microsoft Teams is not chosen

While Teams is incredibly capable with integrations into Microsoft Office tools and fit for structured collaboration, it feels a bit clumsy during projects that require fast, informal communication. Within a student project a setting so central in ease of access and flexibility Teams could feel constrictive and overly technical (Nabben, 2019).

1.3.2.2 Discord

Although it was initially devised for gamers, lately, it gained huge momentum in big communities such as education and project collaborations. It is popular due to its very user-friendly design, its voice channels, and capabilities for text messaging and screen-sharing, making it just right for real-time collaboration and informal communication (Koszolko, 2024).

Key Features:

- The servers and channels: A person can make well-organized servers, as there are different channels for different topics that might keep the conversations.
- Video call and Screen Sharing: Allows it to make video calls with screen sharing in voice and video channels. It is very useful during live demonstrations or in sessions of brainstorming.

- Voice and Text Channels: Members can join a voice channel for real-time communication, which might not necessarily initiate a call; hence, smooth interaction within the team (Koszolko, 2024).
- Bot Integration: Enables the creation of personal bots for task automation, schedule organization, and project reminders.
- Custom Roles and Permissions: Permissions can be granted to users at a much finer level, which is useful during the organization of roles within a team.

1.3.2.3 Chosen Tool: Discord

With its ease of use, flexibility, and real time capabilities, Discord is highly suited to a student project whose members need a fast, informal way to connect. The ability to have a dedicated channel for each topic enhances organization and encourages spontaneous collaboration by focusing on voice and screen-sharing features (Koszolko, 2024).

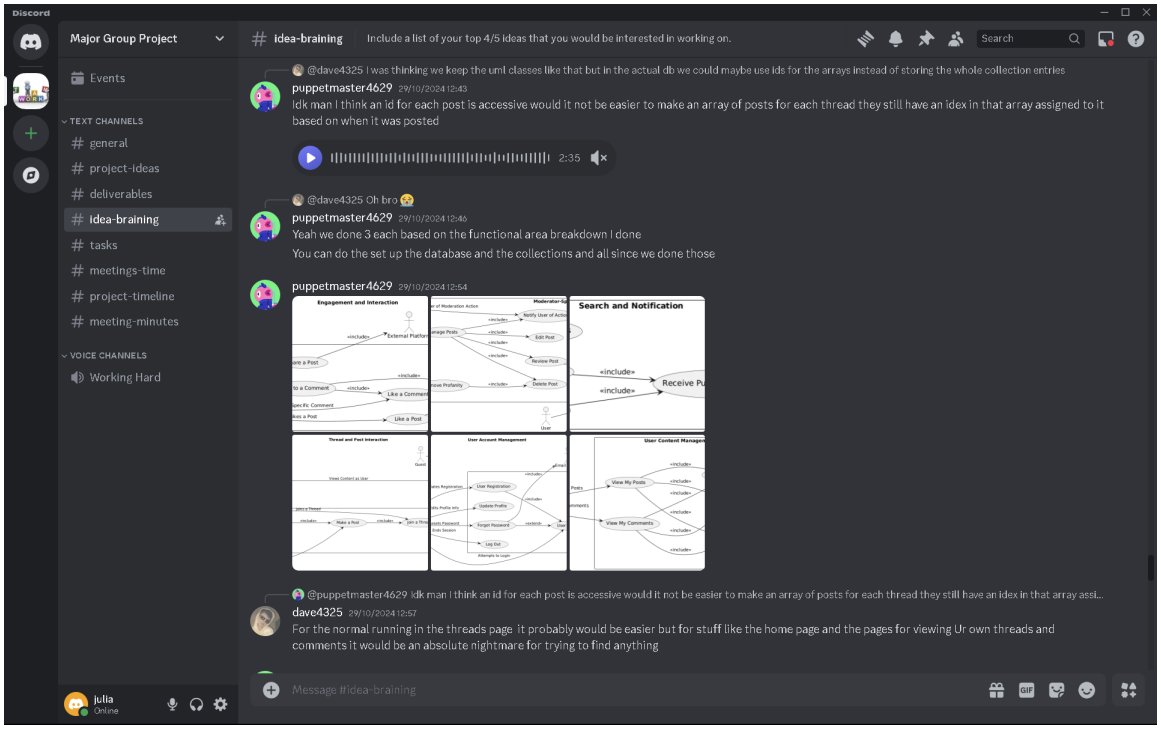


FIGURE 11: (Author's own, 2024) Screenshot of Discord App Functionality

Screenshot of Discord App Functionality including texting, replying, voice messages and multiple channel creation.

1.3.2.4 Comparison with Other Communication Tools

Slack

While also offering channels and integrations like Discord, Slack is far more organizational and suited to professional use. The heavyweight app integrations it provides are beneficial for project management; however, it lacks the voice-channel functionality of Discord (Kulkarni et al., 2015).

Google Meet

This is another video-centric tool that is easy to use but rudimentary compared to Discord's capabilities for real-time chatting and project organization (De Winne et al., 2011).

Zoom

While it can be considered one of the types of video conferencing software, it's better suited for scheduled meetings. It's not real-time and near constant communication like Discord features. That makes it more fitted to structured and formalized video calls rather than an ongoing discussion with the team (Owoh, 2024).

Connecteam

More oriented toward team management and staff scheduling, including features related to task assignment and time tracking. Connecteam can be considered good to go when it comes to managing employee workflows; however, it falls way behind when it comes to offering real-time continuous communication like Discord (Hossain, 2023).

To put it another way, Discord has the flexibility and real-time collaboration features that best suit an informal, student-led project. Its voice channels are intuitive and easy to use, making it the superior choice over the remaining options, which are either too formal or specialized for structured management tasks (Koszolko, 2024).

1.3.3 Document Collaboration and Sharing

1.3.3.1 OneDrive

OneDrive is a Microsoft ecosystem leveraging cloud storage to back up and share documents, projects, and other files. Because it successfully integrates with Microsoft Office applications, educators and corporate employees have found multiple uses for this platform. Users can share files, work in real-time, and gain access from anywhere on any device in OneDrive (Alotaibi & Alomair, 2019).

Key Features:

- Seamless Integration with Office 365: Heavy native integration with Microsoft Office applications such as Word, Excel, and PowerPoint make it really easy to edit or collaborate on files directly from within these applications (Alotaibi & Alomair, 2019).
- Real-time Collaboration: It enables multiple users to work on a document simultaneously, shows changes in real time, and hence is handy for teamwork (Alotaibi & Alomair, 2019).
- Version Control: It provides file versioning so that users can revert back to previous versions if something goes wrong.
- File Permissions and Sharing: The sharing of the files and permissions are permissible, which makes it easy to provide privileges to the extent of view or editing the documents, allowing file sharing.
- Cross-device syncing and offline access mean files sync across devices and are accessible offline; this means that the continuity of document access and collaboration remains intact.

Why OneDrive is better

Projects with team members using Microsoft Office are the perfect fit for OneDrive since it totally integrates with Office 365. The document sharing features are minimalistic, and the real-time collaboration within Office applications makes the workflow easier. OneDrive will also be considerably more comfortable for users who are already in the Microsoft ecosystem in terms of its interface and compatibility with Windows devices (Alotaibi & Alomair, 2019).

1.3.3.2 Google Workspace

Google Workspace is a suite of collaboration tools that encompasses cloud storage via Google Drive, document creation via Google Docs, Sheets, and Slides. It finds great application in educational and remote work due to its accessibility and ease of use, hence its popularity (Lake, 2022).

Key Features:

- Easy permissions-based sharing: Sharing settings allow users to determine who has rights to view, comment, or edit any document.

- Cross-device access and offline mode: This is made possible with Workspace, where on the internet, any device can access files by making changes in settings through Google Drive (Lake, 2022).
- Google Apps Integrations: Works well with other integrations, such as Google Calendar and Meet for meeting scheduling, and task management.
- Built-in Collaboration in Google Docs, Sheets, and Slides: The Google Workspace allows for real-time edits; comments and chat capabilities are also available within the documents. The usage of this functionality makes it pretty easy for teams to work together on a single document simultaneously (Lake, 2022).
- Add-Ons and Integrations: Integrations with a bunch of third-party apps make Google Workspace more productive to work with and collaborative.

OneDrive Comparison

While Google Workspace is highly accessible and user-friendly, deep integration with Microsoft Office makes OneDrive a better choice for teams reliant on the Office apps. Where real-time editing and sharing across non-Microsoft tools are necessary, Google Workspace excels, but on projects where format types from Office are used frequently, OneDrive is more efficient (Lake, 2022).

1.3.3.3 Tool Chosen: OneDrive

OneDrive is because, through integration with Office 365, it provides for ease of collaboration on documents, with file management functionality for security. Ideal for project WORK requiring editing and sharing documents, this makes for smooth integration with Microsoft applications and real-time document collaboration in Office files themselves, especially where Microsoft products are used.

1.3.3.4 Comparison with Other Document Collaboration Platforms

Zoho WorkDrive

While famous for team collaboration and document organization, Zoho WorkDrive implements their strong access control and security features that are effective in storing documents. It lacks a tighter integration with Office applications compared to OneDrive, though (Fiorello, 2021).

Box

Box is a large enterprise cloud storage and collaboration where security and compliance features are put front and center. It features advanced file-sharing but is less intuitive for smaller teams than some services and lacks OneDrive's tight Office integration (Dinh et al., 2017).

Filestage

Filestage was built for content review and approval, which is perfect for projects which involve lots of feedback. At the same time, it doesn't possess the depth of document editing and storing that one can get with OneDrive, so in being a rather niche document collaboration tool, it is much better suited (Woods et al., 2022).

In brief, OneDrive would be perfect for users working with Office 365 since it is well integrated into all Microsoft applications. It is also great to collaborate on documents within a project or when studying. Other solutions, like Google Workspace or Box, could suggest using them, but no other solution has functionalities as good as OneDrive does in handling the documents coming from Microsoft.

1.3.4 Writing Tools

1.3.4.1 Microsoft Word Overview

Microsoft Word is the most famous, fully featured word processing program in the suite of Microsoft Office. It exists as a desktop application and online, allowing the user to work offline or in real-time collaboration. It is used in educational and professional settings by many people due to its general formatting, editing, and collaboration capabilities (Dishaw et al., 2011).

Key Features:

- Formatting Utilities: It contains full-featured style, sophisticated page layout, and template formatting in Word, thereby making the creation of detailed documents easy.
- Real-time Collaboration: Through OneDrive or SharePoint, multiple users are able to co-author a document in real-time, editing and automatically recording the changes to make tracking updates and managing contributions relatively painless (Dishaw et al., 2011).
- Comments and Track Changes: Robust revision features in Word allow users to create comments, track changes, and then review document changes easily.

- Offline and Online Access: Word has been both a desktop-based application and a cloud-based tool, therefore accessible even offline.
- Integration with Microsoft Office: It is highly integrated with other Microsoft Office applications such as Excel and PowerPoint. Thus, embedding of data or linking to the documents is quite easy.

Why Choose Word

Word is particularly fit for formal documents, such as essays, proposals, or reports, which require a lot of formatting and a concise layout. The presence of a variety of desktops further guarantees continued access, while its collaboration and revision capabilities in real-time make it ideal for group projects where multiple end-users will be contributing to a single work. Given its compatibility with other Office tools, Word makes for a reliable choice in projects within the Microsoft ecosystem (Dishaw et al., 2011).

1.3.4.2 Google Docs

Google Docs is a web word processor integrated into the suite of productivity applications offered by Google, Google Workspace. It boasts simplicity and reaches a broad audience because it can easily be applied for any collaborative writing. The interface is minimalistic in Google Docs and sharing and collaboration are a lot easier due to integration with other devices (Suwantarathip & Wichadee, 2014).

Key Features:

- Real-time Collaboration: Changes can be made in the same document by multiple users simultaneously and will show up in real time (Suwantarathip & Wichadee, 2014).
- Comments and Suggestions: It allows team members to add comments, tag others, and suggests edits, thus making it ideal for giving team feedback and doing collaborative editing.
- Accessibility and Shareability: Google Doc is accessible on any device with access to the internet since it is a web-based application, and sharing can be highly flexible.
- Add-Ons and Integrations: Google Docs has native integrations with Google Drive, Calendar, and other Workspace apps. Third-party add-ons are also supported to extend further functionality.

- Version History: It allows users to view past document versions for comparison. This provides version history that ensures the integrity of the document over time.

Comparison to Word

What Google Docs had over Microsoft Word was further simplicity and real-time collaboration, while Microsoft Word had more in-depth formatting and layout settings within a document. While Word has better offline capability, it may also allow access to more far-reaching options for styling, thus being better in creating formal documents. Correspondingly, Google Docs is more relevant when the task requires rapid drafting and sharing (Suwantarathip & Wichadee, 2014).

1.3.4.3 Chosen Tool: Microsoft Word

Microsoft Word has been selected for its advanced formatting features, accessible anytime even offline document editing and review, and a vast array of capabilities. Because integration with OneDrive is smooth, it will be easy to collaborate on documents with others. Thus, the creation of polished, professional documents for your group projects is possible.

1.3.4.4 Comparison with Other Writing Platforms

Zoho Docs

It's much like Google Docs, but it really does have a full-featured online document editor that allows for real-time collaboration. Lacking some of the advanced formatting features in Word, it's good for quick drafting and more lowly complicated documents (Rethlefsen et al., 2009).

Document360

Document360 is designed to build documentation and knowledge bases. It's ideal for information management and structuring, rather than creation with full formatting depth, like proposals and reports (Koivunen, 2021).

ONLYOFFICE Docs

ONLYOFFICE supports a large feature set of document editing and collaboration features, like Word. Complex formatting and offline editing are supported; however, smooth integration with an ecosystem isn't as refined there as in Microsoft Office, and thus, Word

might be considered more seamlessly integrated for projects where the work has already been begun in Office applications (Burchart & Haake, 2024).

Put simply, Microsoft Word is the preferred platform to make formal and structured documents, as it allows for complex features, making it suitable for detailed writing projects. Applications such as Google Docs and Zoho Docs allow simplicity and real-time collaboration but do not match the capabilities that Word has in complex formatting and offline access.

1.3.5 Project Management

1.3.5.1 Microsoft Project

Microsoft Project is indeed a very functional tool in project management, supporting detailed project planning, resources management, and timeline tracking. For a long time now, it has been regarded as one of the most advanced schedulers and trackers, making it ideal for projects that are very large and complicated, requiring efficient coordination of activities and resources (Vajjhala, 2012).

Key Features:

- Project templates: These are preconfigured templates for various types of projects. Such templates help your teams to get up and running by providing them instantly with a pre-defined structure.
- Integrating with Microsoft Office Suite: It can be integrated with other Microsoft tools, such as Teams, Excel, and Power BI, which helps to make imports of data, reporting, and collaboration easier (Vajjhala, 2012).
- Gantt Diagrams and Task Scheduling: Microsoft Project contains advanced Gantt diagrams, whereby project managers can graphically build a timeline of their projects and manage the dependency between tasks with perfection.
- Resource Management: This enables teams to assign and manage resources, such as personnel and materials, whose usage they can track to help optimize the allocation.
- Budget Tracking and Cost Control: It has integrated tools to help track costs, fix budgets, and monitor expenditures throughout the project (Vajjhala, 2012).

Why Microsoft Project is Better

Microsoft Project would be the best fit for projects involving complex dependencies and resource requirements because of the detailed project planning and resource management features involved. The generation of highly customizable Gantt charts and cost management capabilities make the tool very useful for teams requiring minute-level oversight and planning. Further, its integration with the Office Suite and availability of project templates also extends its efficiency in creating and managing structured project plans using Microsoft Project (Vajjhala, 2012).

1.3.5.2 Trello

Trello is a pretty versatile project management tool that really excels in terms of usability and clarity of vision. Boards, lists, and cards empower teams to better organize everything from tasks over project progress to teamwork in an easy, seamless manner. That is why Trello sees huge popularity: it makes things so simple, yet it can remain so adaptable (Shchetynina & Kravchenko, 2022).

Key Features:

- Boards, Lists, and Cards: Trello allows teams to keep track of tasks visually by moving cards across lists. Progress is quite easy to track that way.
- App Integrations: Trello integrates with applications such as Slack and Google Drive to enhance workflow and collaboration.
- Custom Labels and Checklists: Users can add labels in tasks, due dates, and checklists, thus organizing them further.
- Automation and Power-Ups: Power-Ups, as they are called in Trello, give way to even more functionality, including calendar views, task voting, and even automation with Butler.
- Features of Collaboration: Collaboration options are simple and intuitive for team members, serving to assign tasks, leave comments, and attach files on cards (Shchetynina & Kravchenko, 2022).

Comparison with Microsoft Project

While Trello is friendly and far easier to use, in fact, for light or smaller projects the advantages of Microsoft Project surely shine brighter when there is greater control over things like timelines, budgets, and resources. A plan for a very detailed and structured project applies best to Microsoft Project, while in Trello, such a visual drag-and-drop

approach perfectly suits straightforward task tracking without any complex dependencies (Shchetynina & Kravchenko, 2022).

1.3.5.3 Chosen Tool: Microsoft Project

Microsoft Project has been opted for because the level of project planning required seems high, alongside time scheduling and resource management. It also contains detailed Gantt Charts that can be integrated into other Microsoft toolsets. It is ideal for those complex projects that have a structured timeline and deliverables.

1.3.5.4 Comparison with Other Project Management Platforms

Zoho Projects

A very easy-going project management software that includes time tracking, Gantt charts, and task management. It doesn't have the fine resource management or integration like Microsoft Project. For small to medium projects, that's just fine (Arya & Kulkarni, 2024).

LiquidPlanner

It works perfectly for projects that are very dynamic in nature, featuring a lot of frequent changes. Due to predictive scheduling and prioritization of tasks, it is less structured than Microsoft Project and doesn't possess such detailed financial and resource tracking tools (Weerasuriya et al., 2022).

Wrike

Combines task management with advanced reporting and real-time collaboration. It would work for teams needing flexibility but does not match Microsoft Project in creating highly detailed, structured project plans for more complex projects (Cheverda, 2022).

Put simply, Microsoft Project is ideal for big projects requiring resource and budget tracking intensively, and complex scheduling, according to integrations within the Microsoft ecosystem. Less-weighty alternatives include Trello and Zoho Projects, which are more visually oriented but do not have advanced functionality for management in the way Microsoft Project has.

1.4 Version Control Software

Version control is essential in any software development project, to ensure mistakes can be easily corrected and protect the project form irreversible damage due to corruption of files and other issues.

The team will use git as its version control method as it is the industry standard for version control and allows easy collaboration between team members and asynchronous work, meaning team members can work on different sections of the project individually without it affecting the work other are doing.

The team will use GitHub as their version control software for the project as it is the industry standard for version control with git and is what the team is most experienced in using. GitHub is useful as it contains easy to user features with a simple and effective user interface.

While developing the project on GitHub the team will divide tasks/features of the project among team members, who will then create individual branches that they will work on, until the feature they are working on is fully functional and ready to be added to the project, when they will make a pull request for the feature to be added to main branch. Once the request has been reviewed by other team members and they have confirmed that the feature is functional, the feature branch will be added to the main branch.

GitHub will also allow the team to fork the project at certain points in development, such as when the prototype, pre API app, pilot app and pretesting app have been completed, to save the project in these states and allow the team to resume development on the main fork without effecting these other forks.

Using branches and forks in this way will not only allow team members to work individually on certain features and save the project at different milestones, but will also allow for easier conflict resolution, reverting back to earlier version of the app before these issues arise. It will also promote team collaboration and peer reviewing through the use of pull requests, ensuring all team member are content with features before they are added and that any mistakes are identified early, before they cause any problems.

1.4.1 How the team will use GitHub

For the purpose of this demonstration, “Example brand” will represent a feature and “feature.txt” will represent any data added to this branch. When a team member is about to start working on a new feature for the project, they will create a new branch in the project for this specific feature.

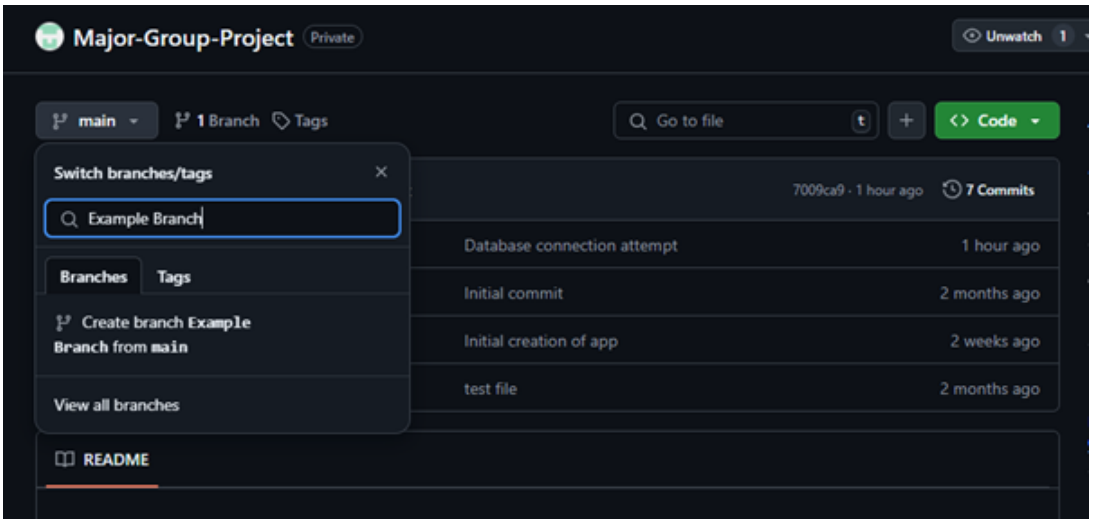


FIGURE 12: (Author's own, 2024) Screenshot of GitHub repository interface

They can then begin to work on the project, adding code for this feature to the new branch.

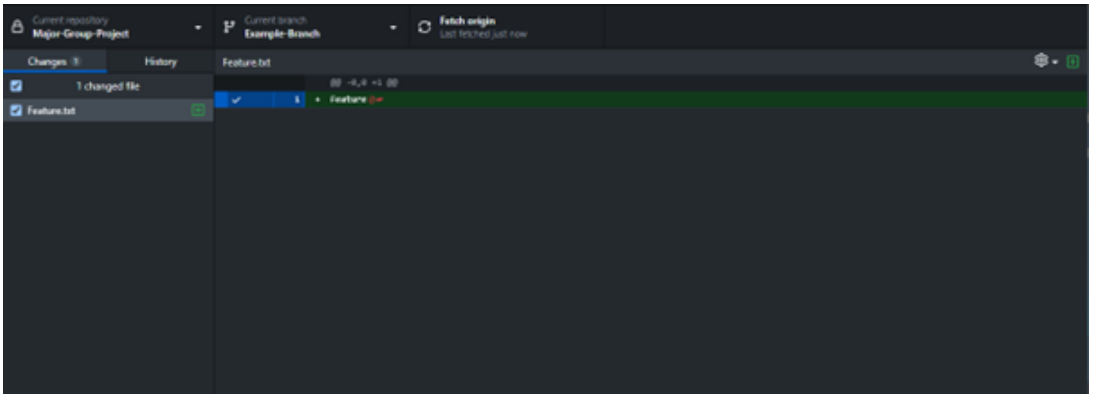


FIGURE 13: (Author's own, 2024) Screenshot of GitHub repository interface

Note: Changes are made only on this branch, not on the main branch.

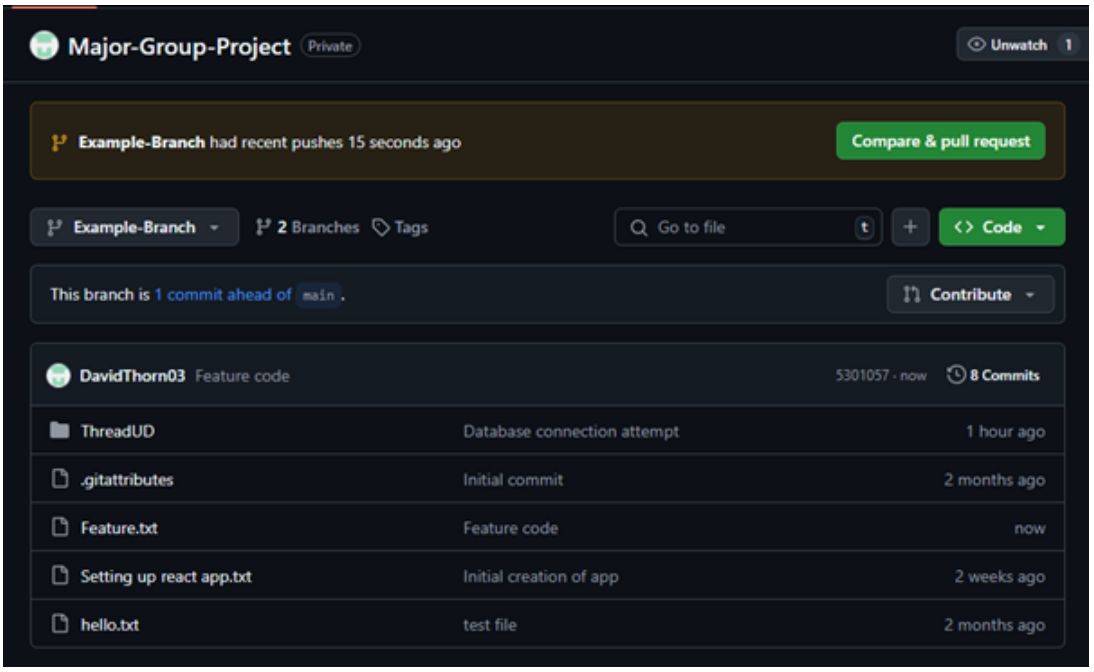


FIGURE 14: (Author's own, 2024) Screenshot of GitHub repository interface

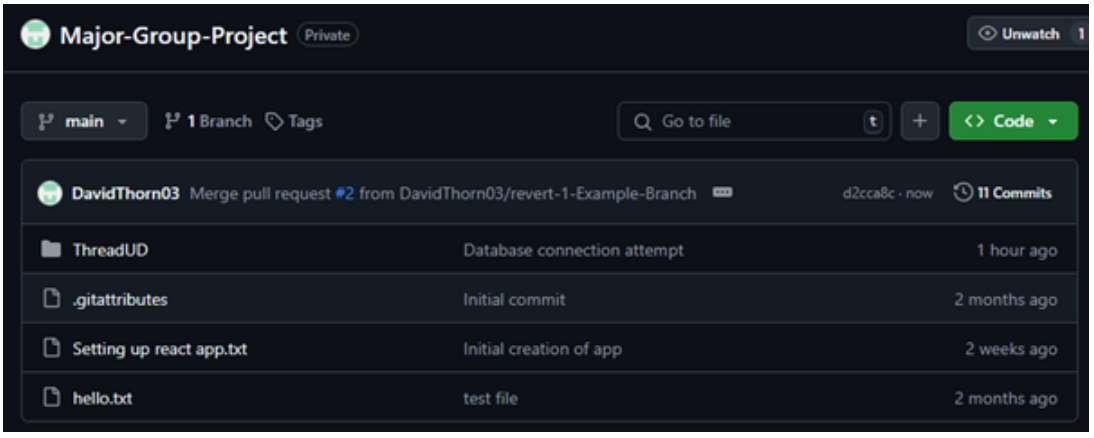


FIGURE 15: (Author's own, 2024) Screenshot of GitHub repository interface

Once the new feature has been completed, tested, and is ready to be added to the main project, they can then create a merge request to the main branch.

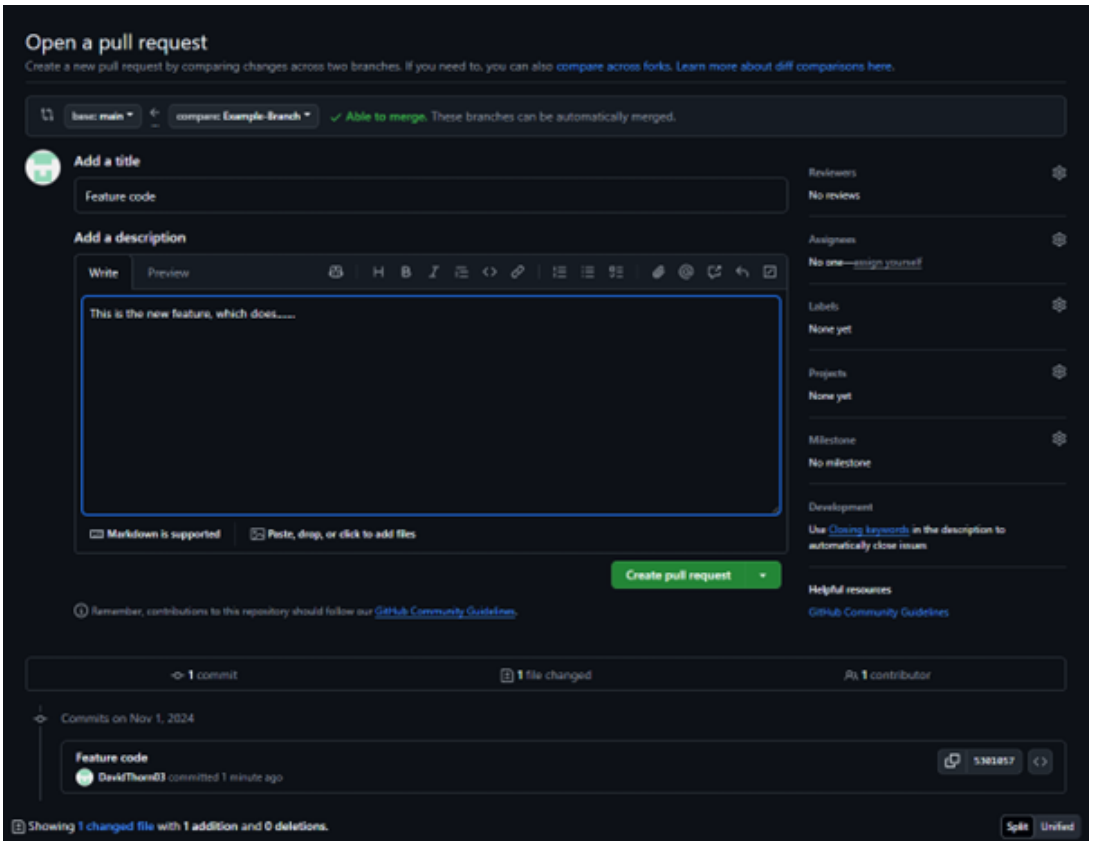


FIGURE 16: (Author's own, 2024) Screenshot of GitHub repository interface

Once this feature has been reviewed and approved by all team members, it can be merged with the main branch.

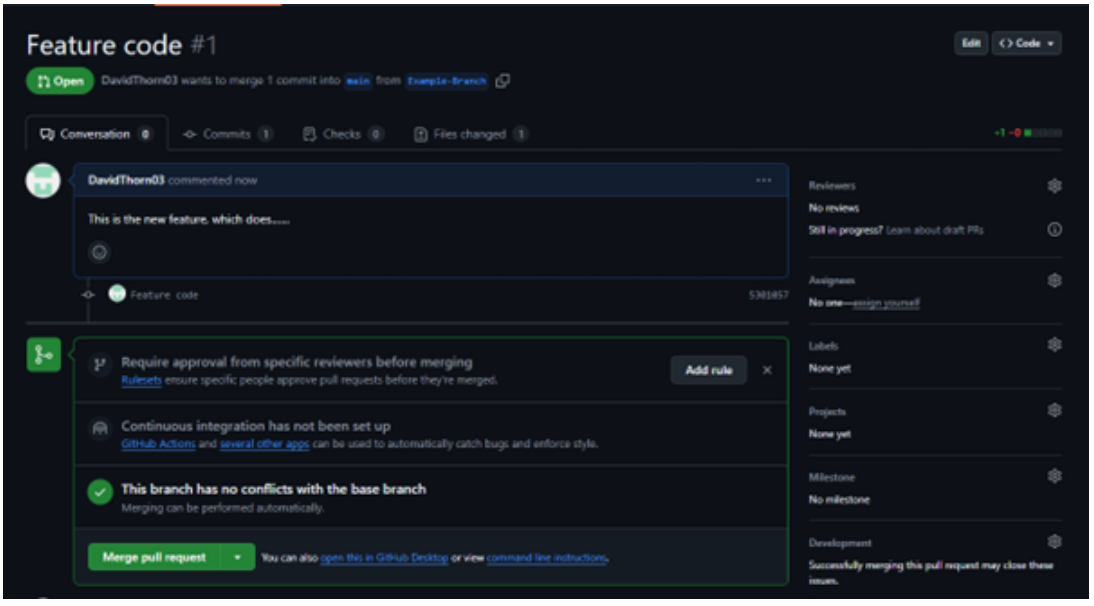


FIGURE 17: (Author's own, 2024) Screenshot of GitHub repository interface

Once this is complete, the new feature will be added to the main branch.

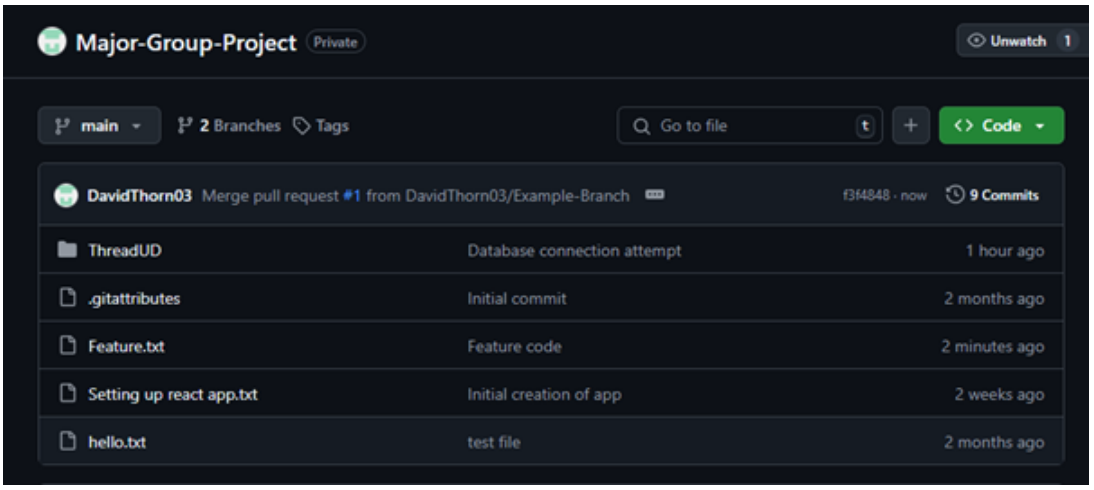


FIGURE 18: (Author's own, 2024) Screenshot of GitHub repository interface

Note: If new features are added to the main branch while the feature branch is still being worked on, the same process can be done to merge the new changes from the main to the feature branch.

If later it is found that this feature is incomplete, causes problems or has corrupted other sections of the project, the merge can be undone by navigating to the closed branch and 'reverting' it.

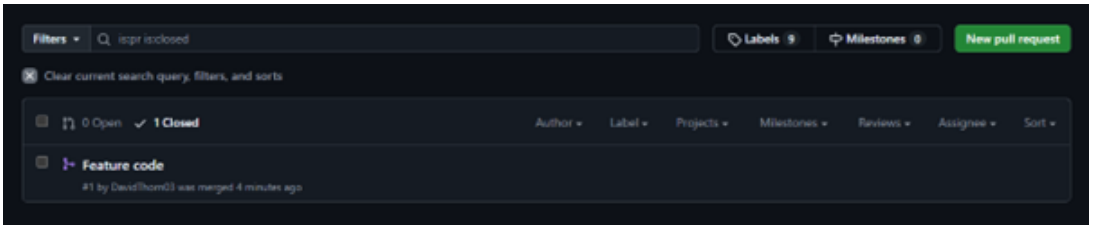


FIGURE 19: (Author's own, 2024) Screenshot of GitHub repository interface

This revert action then behaves the same way as any other pull request but removes the branch files instead of adding them.

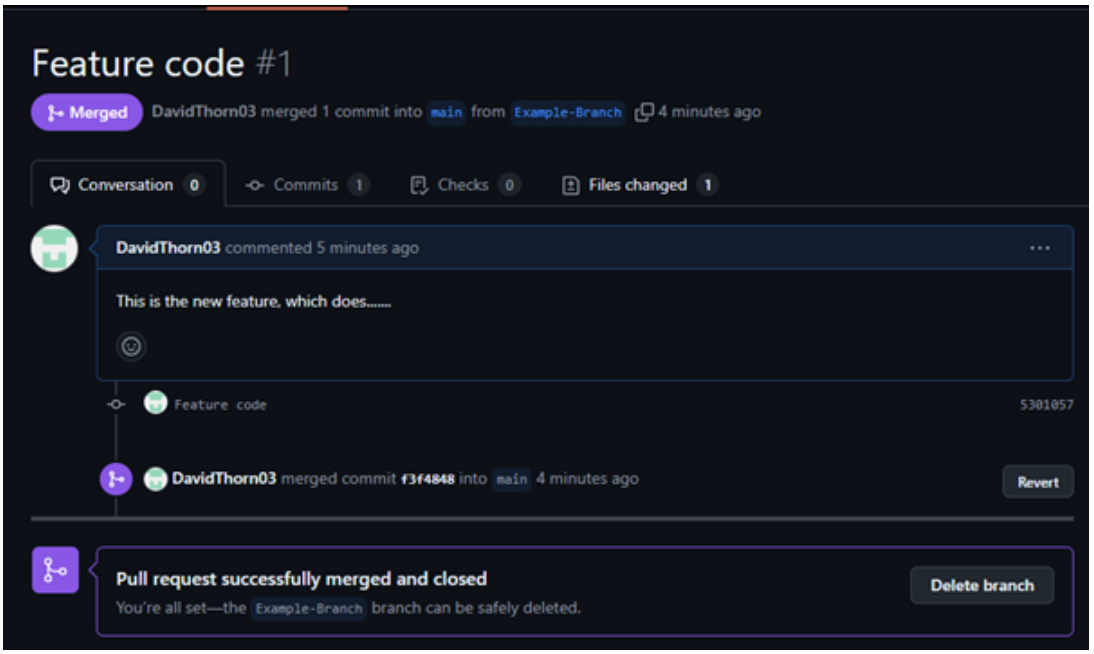


FIGURE 20: (Author's own, 2024) Screenshot of GitHub repository interface

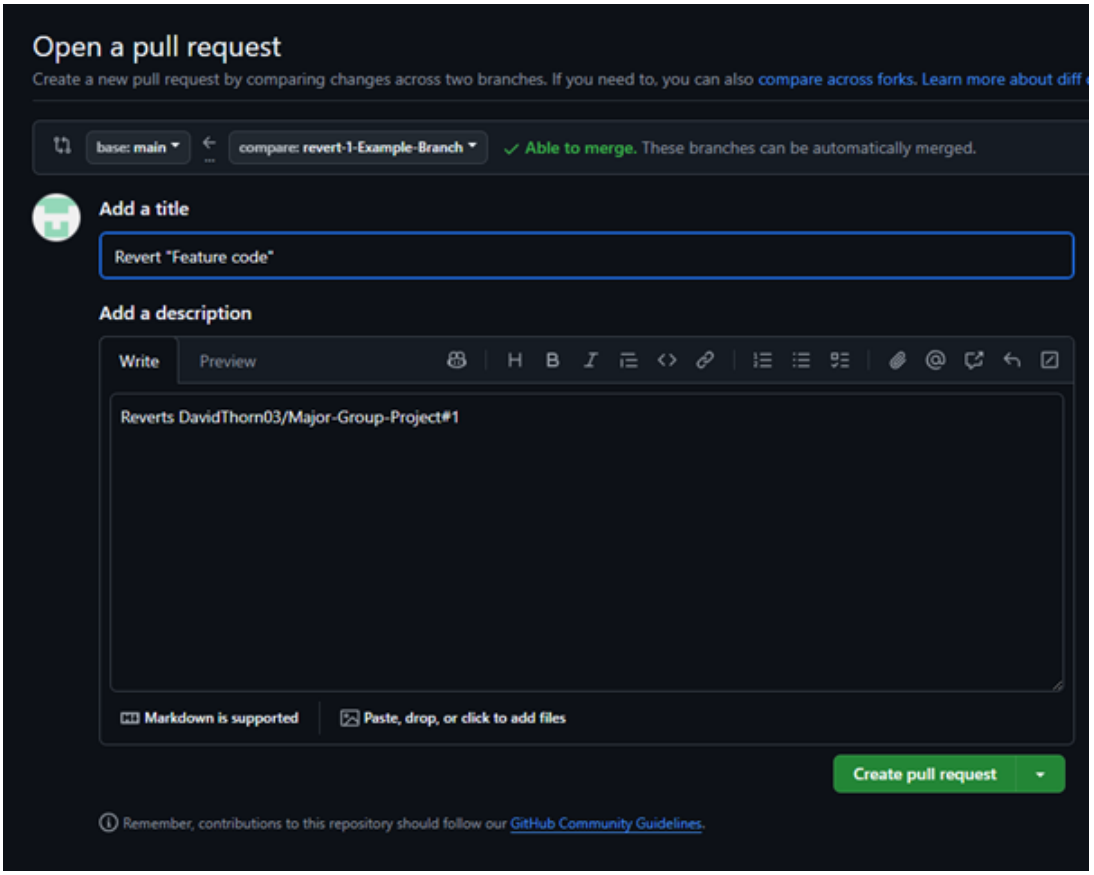


FIGURE 21: (Author's own, 2024) Screenshot of GitHub repository interface

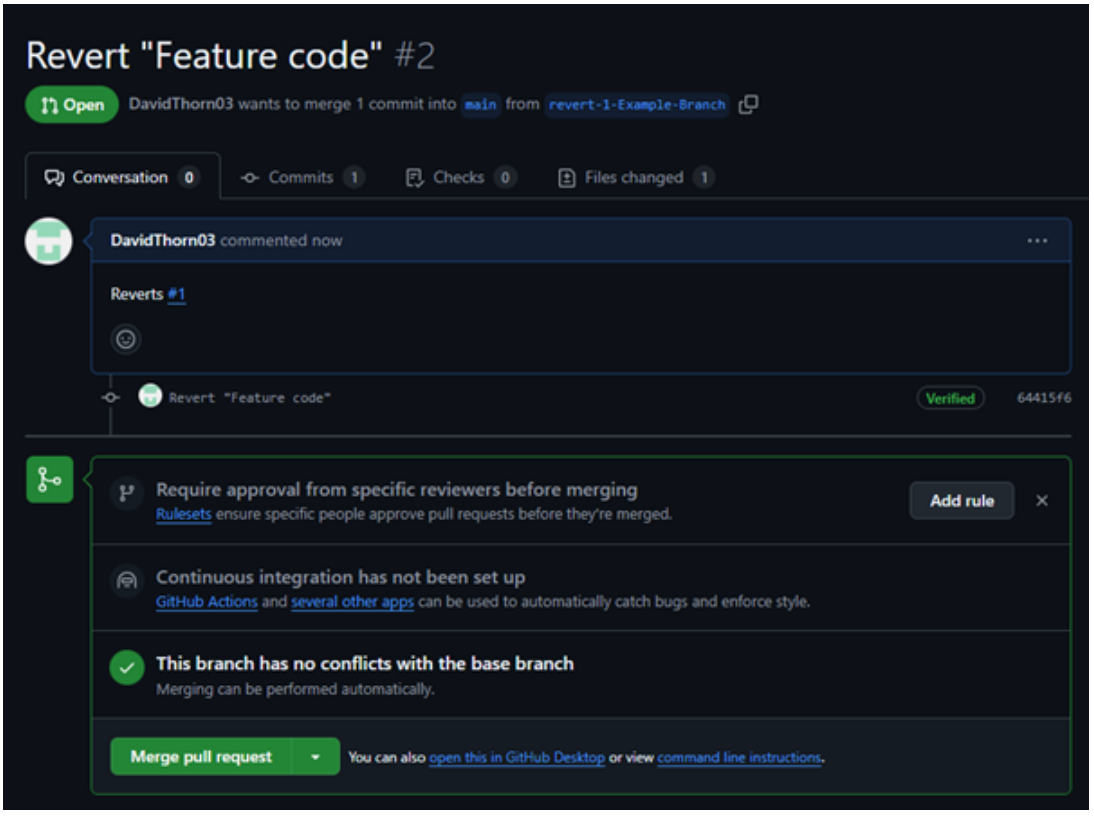


FIGURE 22: (Author's own, 2024) Screenshot of GitHub repository interface

Once merged, all files added from the feature branch will be removed.

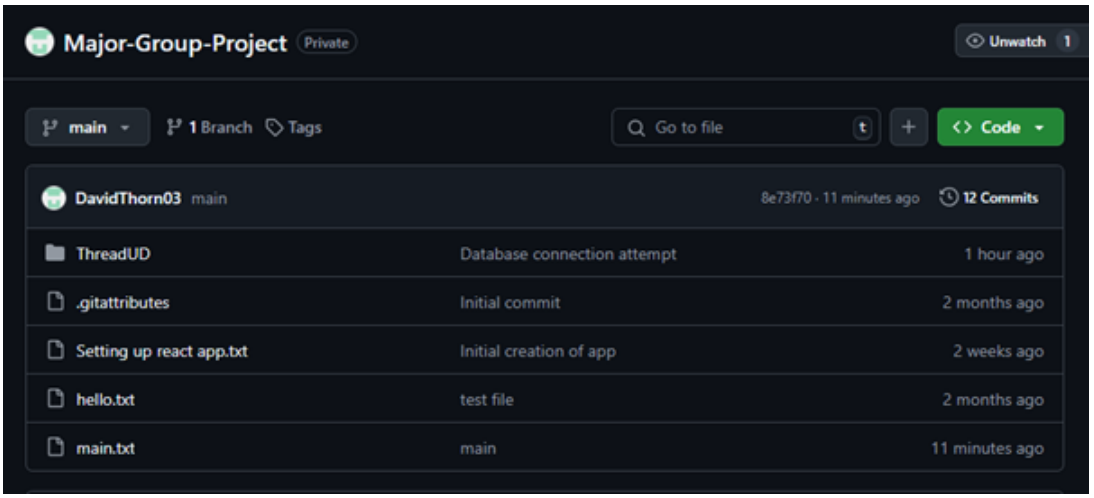


FIGURE 23: (Author's own, 2024) Screenshot of GitHub repository interface

Chapter 2

Literature Reviews

How can peer-learning positively impact a student's education and how can we build an environment where this can safely take place?

2.1 Introduction

The literature review covers three interrelated topics which are mobile development frameworks, design principles of User Interface and User Experience, and ways of leveraging technology to foster academic conversation and peer learning. The paper reviews three highly well-known technologies in recent mobile development growth like React Native, Kotlin, and Flutter. It shows respective strengths and weaknesses while carrying out cross-platform development. Further, the paper focuses on the principles of UI and UX design, giving much importance to the fact that appealing design must be in symbiosis with usability in real life in order to come up with interesting applications. Finally, the review examines how technological platforms support collaborative learning by discussing the ability of such platforms to transcend traditional boundaries and redefine educational methodologies. Put together, these sections represent a comprehensive examination of how technology influences application development, design, and education.

2.2 The importance of academic conversation and peer learning and how this can be enhanced using technology

2.2.1 What is academic conversation and peer learning? Why are they important?

Before discussing why academic conversation and peer learning are important learning methods and how technology, such as e-learning and forum-based technology, can enhance these learning methods, it's important to understand exactly what each of these terms mean, both individually and in relation to each other and this review.

Academic conversation

Academic conversations are student/peer dialogues which focus on a topic, discussing it and building, challenging and negotiating their ideas with each other to better understand the idea and its concepts (Zwiers & Crawford, 2023). This type of conversation is exactly what the project proposed in this proposal aims to promote and create a platform for.

Peer learning

Peer learning has no singular, definite definition, but rather includes a number of activities such as having senior peers with more experience in subject fields teach their junior peer in these subjects, having peers learning at the same level group up to learn about topics from each other as they study different subject, or through peer assessment, or having peers review each other's work and research (Boud & Cohen, 2014). A broad definition of this concept would be people in similar or the same fields learning from and with each other, through conversation, review or otherwise. This study, and the proposed project this study is being done for, will include a range of peer learning types, but will focus on peers in similar subjects of different levels learning from and teaching each other through conversation. Hence, for the purpose of this review, reference to peer learning should be assumed to mean this definition.

2.2.2 Why are they important?

Academic conversation and peer learning are important steps in learning as they provide alternative learning styles then standard methods, such as lectures and talks. Lectures and talks will generally only cover one type of learning, passive learning, while peer learning includes both passive and active learning. Active learning involves "generating

information by yourself" (MacDonald & Frank, 2016) or in other words, doing work/research yourself to learn rather than learning directly from others. Peer learning aids in this type of learning as it involves teaching others, thinking more deeply about topics to be able to teach others.

Peer learning and academic conversation also provides insight form new and varying perspectives. Learning about a topic from one person (a teacher/lecturer) only provides one avenue for information. In this way you can only learn from what they know and their perspectives. This not only limits what you can learn, with their own lack of understanding in different areas being transferred to you, but it can also lead a bias in understanding of some topics. It is important to learn from people of different backgrounds and groups, as people of different professional backgrounds will learn about topics in different ways and will provide new perspectives. This gives a broader view of a topic and lead to less bias in information (Boud & Cohen, 2014).

Peer learning is also beneficial to those teaching their peers, whether that be senior peers teaching their juniors or peers on the same level, as teaching others offers a new type of learning and tests the depth of understanding of a subject. People often learn more about a topic through teaching others as going through the process of explaining/demonstrating a topic forces the teacher to further evaluate what they are teaching. Having to explain topics to others who have different insights and methods of understanding forces us to think about topics in new ways and from different perspective. Teaching others also highlights gaps in our knowledge which weren't as prominent when learning about it ourselves, providing insight to what we need to learn more about ourselves (Cortese, 2005).

Even for a person who possesses a high level of understanding in a topic, academic conversation can be a beneficial learning tool as it can provide thought provoking insight, leading to a change in perspective and understanding of a topic. Hearing the opinions of others regarding a subject, whether that be a new idea/concept of that topic or a different perspective off the same idea, allows us to re-think our own understanding of that idea and reshape it (Zwiers & Crawford, 2023).

Peer learning is also beneficial as it is more specific to individuals and their own knowledge. Learning in a classroom/lecturing environment can often be challenging as it is less individual focused and will not adhere to individuals specific need and understanding. Peer learning resolves this issue, as when peers ask each other questions and work with each other to learn topics, it is specific to what the students need to learn. Students will be able to demonstrate/teach these concepts to each other in ways that is more specific to them, spending more time on topics that they find more challenging and demonstrating them in ways that adhere to their own methods of understanding.

This contrasts with in more traditional learning environments where teachers/lecturers must be less focused on the individual and more focused on the group as a whole (Boud & Cohen, 2014).

Now that we have a better understanding of peer learning and academic conversation, we begin to discuss how technology can be used as a platform for these types of learning.

2.2.3 How can technology be used as a tool for learning?

Technology, specifically the internet, can be utilized as a tool for learning, and has many advantages which builds upon the benefits of traditional learning styles. Through the internet, many resources for learning can be found to aid in learning and research, such as tutorials, papers, videos and even full classes and courses done completely or partially online.

2.2.4 How can technology be used as a tool for academic conversation and peer learning?

Technology, such as the internet, can also be used as a tool for academic conversation and peer learning. Forum based apps, message boards and even direct and group messaging apps can all be uses as a platform for academic conversation and peer learning and allow people to ask questions or to get feedback from others. Using technology in this way allows people in similar fields to discuss topics at any time and from anywhere.

2.2.5 How is using technology in this way beneficial? How does it enhance peer learning and academic conversation?

Research has demonstrated that using technology for learning in the ways above enhances learning and academic conversation in several ways. We will discuss a few here.

One of the most obvious advantages to using technology in this way is that it makes learning and finding others who are involved in the same subjects easier and is more available to all. Having online platforms for learning and academic discussion provides more easily accessible educational resources, making education research easier and cheaper for students and self-taught learner alike. Online forum based and messaging resources also give those who are both self-taught and those enrolled in education who are looking to research topics not covered by that education access to people with knowledge in that field they previously didn't have access to (Means et al., 2013).

Using technology as a platform for online learning is also more flexible for learners, giving access to learners outside of normal lecturing and class times for both academic resources and peer support platforms. Using internet-based environments allows for learners to access them at any time. This greater flexibility not only accommodates for learners with untraditional schedules but has also proven to enhance learning and information retention in those who use it (Bergamin et al., 2012).

Getting information and support in this way is also beneficial as it provides sources from more than one teacher/lecturer. Using online platforms in this way allows learners to get information and support from many different sources which is a vital addition to the support offered by lecturers. In this way, forum-based apps and other e learning platforms give learners to more information and opinions on topics, ensuring the teachers own bias doesn't lead to bias in understanding and opinion about the subject matter. Getting information from additional sources also ensures that the lecturer's own gaps in knowledge aren't passed on to those learning from them (Schindler et al., 2017).

However, this isn't the only benefit to getting information from more than one source. Using online platforms allows learns to get information form people of many different professional and personal backgrounds as it allows learners to not only learn from others in different organisations, but also people from all over the world. This provides a new perspective on topics that previously wouldn't have been available to learners, which gives learners a more flexible understanding and varied understanding of topics (Veletsianos & Houlden, 2019).

Another benefit form learning from more than once source is that learners aren't limited to only experiencing the teaching style of their lecturer/teacher, as different people from different backgrounds will have different teaching styles. This can be extremely beneficial for learners as different teaching methods are more appropriate for different learning styles, and a teacher's teaching style will never be perfectly suited too every student (Beck, 2001). Using the internet to get resources and help form people with teaching styles that are more well suited to the individuals learning style can greatly benefit their learning experience.

This isn't the only way in which the internet can be used to content more suitable to the learners needs. Additionally, the availability of more resources and sources enabled learners to easily find information specific to their needs and the subjects they are studding. Learners can use the internet to find courses, tutorials or other instructional sites that cover specifically what they are struggling with or are interested in learning (Soffer, Kahan & Nachmias, 2019). The use of forum-based sites can further simplify this process and help learners to get help and feedback on their specific query or issue,

with little searching requirements. This can be done by posting a question or query themselves or by finding a post from someone else with a similar issue.

Specifically, the use of forum-based apps can greatly enhance learning through academic conversation and peer learning. Having academic conversations can lead to learners gaining new insight and perspective on topics, as mentioned earlier in this report. Having these conversations through the internet provided learners with more diverse perspectives and viewpoints, since it included those outside of their own place of learning. This greater variety of viewpoints leads to more insight into a topic and hence, more though provoking and idea reshaping conversation (Veletsianos & Houlden, 2019).

Forum based apps also greatly enhance learning by providing a previously unavailable platform for people to help/teach others, which not just helps the learners but also those teaching them. Forum based apps allow people of the same level of understanding to help each other to work out problems and research topics, rather than listening/reading how others research or how to do it. Learning in this way of researching and solving issues yourself, rather than passively learning through others work/help is often more beneficial to learners, and results in better memory retention (Henrie, Halverson & Graham, 2015).

Using online apps as a platform for learning can also be beneficial for those who already understand the topics their teaching. Research has shown that teaching a topic to others helps those teaching solidify their understanding as they explain these topics to others. This is because teaching these topics to others involves translating their knowledge to new forms so that those who they are teaching can understand (Tenenbaum et al., 2020). This translation is even more important when teaching others through the internet as they haven't learned the topics in the same way and will have been taught with different learning styles to the teacher. The one-to-one format, or at least to fewer then in traditional teaching settings, also means that teachers can further refine their translation to meet needs of those they are teaching.

2.2.6 Summary of Findings

Academic conversation and peer learning greatly enhance student learning and are a crucial addition to more traditional learning styles. These and other learning styles can be implemented with the aid of technology, mainly using online apps and the internet to host classes, tutorials or discussions regarding learning topics. Using technology to aid in learning in this way can be greatly beneficial as it provided more flexibility for learners, in terms of time constrains and variety of resources, allows learners to connect with others studding in similar or the same fields which previously wouldn't have been

reachable and can also provide a platform for people to teach/help others not previously available.

2.3 UX and UI Considerations in Mobile Learning Apps

2.3.1 Introduction

The exponential increase in mobile applications completely changed the way users worked with technology and design came out to be the most crucial element behind every successful application. The interface and user experience of a mobile app didn't remain just a supplanted entity, rather, they structurally held within them the lever of usability and market viability. Generally, the user interface (UI) addresses the aesthetic value and degree of interactivity an application would present to the user. On the contrary, user experience (UX) is designed from the perspective of general satisfaction and ease that users experience in attaining their goals through using an application (Ariffin et al., 2022).

UI and UX are related in a symbiotic way. An attractive UI captures attention, but a seamless UX ensures sustained engagement. However, evolving mobile technology means that developers must match the user's growing expectations with graphically appealing, easy-to-use interfaces that are adaptable. This literature review discusses basic philosophies of UI design, approaches to enhancing it, and ways of enhancing UX, focusing on such aspects as personalization and ease of use, among other parameters.

Such a review will seek to provide comprehensive insight through a synthesis of recent research and best practices on how today's mobile applications effectively embody UI and UX strategies that better user satisfaction and retention. The subsequent sections thus describe specific aspects of UI design, its importance, and improvements before a detailed exploration of UX principles. Customization, engagement, and usability enhancements (Frans et al., 2024).

2.3.2 User Interface (UI)

Importance of Aesthetic Appeal

One of the important sourcing factors that decides the very inception of an application among its receivers is aesthetic appeal toward the UI. An interface acts as the point of contact between a user and an application, setting the tone for the general experience. A good-looking UI creates a very positive impression through aesthetic means, that's the

assurance to make users go further into the app. Particularly this is important in today's mobile app market, where people often figure out in several seconds whether to further continue using an application or to discard it. Those applications that do not have a finished visual outlook seem non-professional and are not trustworthy (Samrgandi, 2021).

More than just making an application look good, aesthetic appeal results in its usability and user engagement. Several studies have documented that users consider visually beautiful designs to be of a higher quality and better to use, even when the core features of competing apps are similar. This involves appealing to the eye with colour scheme, typography, and arrangement of layouts to achieve an intuitively apparent interactive interface. For example, striking features are brought to the fore with an effective balance in contrast of colours, while harmony in typography does wonders by making the text readable and professional (Samrgandi, 2021). Clean and organized layout reduces cognitive burden, so it's extremely easy for the end user to use any application.

Another important dimension is aesthetic design, a well-designed interface may create positive feelings such as fun and pleasure, which may in turn play a large role in shaping user perceptions and behaviour. If the user finds the application nice to look at, then they are more likely to close their eyes on minor usability flaws and be more willing to the app. This emotional bondage is all the more welcome when one considers that users are mostly loyal to those applications that create value not only functionally but also pleasurably.

The aesthetic appeal covers not only static but also dynamic aspects of the interface. Subtle animations and micro-interactions make a great contribution to user experience, giving feedback and guiding user actions (Ejaz et al., 2019). For example, such an animation would appear when hovering over the button, which will display that the application is responsive and interactive. These dynamic elements will add not just usability but delight too and that is what sets this app apart from the competition.

Hence aesthetic appeal should be cautiously balanced in UI design. A load of several visual elements in the interface leads to clutter and degrades usability by a high margin. On the other hand, too simple designs tend to look incomplete or amateurish. In this regard a designer seeks to find a balance between minimalism and visual richness so that every element performs its intended function in harmony with the rest of the interface (Ejaz et al., 2019).

Strategies for Improving UI

Consistency in design is a chief tenet of UI development. In cases where users see uniform patterns in buttons, icons and menus they intuitively can interact in the use of

the application since they would have to spend less cognitive effort in learning new interactions. For instance, following one set of guidelines on designing applications, such as Apple's Human Interface Guidelines or Google's Material Design, makes the application match user expectations, reducing hitches and confusing moments in interactions. Consistency extends to the visual elements, patterns of interaction and even the tone of communication of the app so that it would be seamless and predictable (Luo et al., 2019).

Consistency in Design

Another important aspect in UI design is responsiveness and feedback. At the very least the user should have the feeling that his or her actions are being processed through the system. This ranges from button animations to progress bars, even just subtle audio effects. If a user had to submit a form of information, for instance, most would have a loading spinner afterward to reassure them of their input being processed. This reduces anxiety and any potential frustration. Feedback mechanisms fill in the space between user input and system reaction, thus making users feel in control and a part of the interaction process (Silva et al., 2017).

Feedback Mechanisms

Also, increased consideration about accessibility with regards to design reshaped how UI is approached. In such a way, making an application more accessible to users with some kinds of disabilities stopped being a matter of choice but an obligation if one wanted to reach a vaster audience. High contrast colour choices, text scalability and support for screen readers are among the accessibility features in this regard that would make the application usable by persons with different abilities. Accessibility, in this way, would meet all the legal standards and create a better reputation of the app as more user-centered and socially responsible (Alnanih & Ormandjieva, 2016).

Accessibility

While this capacity of UI design has been extended further with the help of emerging technologies, more and more AI and machine learning are integrated into dynamic and adaptive interfaces, which change based on user preferences and behaviours. AI predicts what a user may want and then proactively presents it to reduce the number of interactions and thus also increase satisfaction. Similarly, the inclusion of micro-interactions includes subtle and engaging animations for every user interaction, making the interface more delightful.

With such development, the UI design also suffers from a number of drawbacks in achieving a proper balance between attractiveness and functionality. Highly intricate

designs may confuse the users, while highly simple interfaces tend to make things look unprofessional. The perfect balance is achieved by iteration in design and needs active participation of the users through feedback and testing. A usability study will help the designers identify pain points and refine the interface to meet user needs effectively (Zaina et al., 2022).

2.3.3 User Experience (UX)

General Feel of an App

The general feel of an application defines user's perception and interaction, mapping into the emotional and functional view of it. The right feeling in an application that is well-designed should make the user feel that they are capable of easily navigating and conducting their intended activities without any efforts. Responsiveness, clarity and intuitiveness in the design translate to the perceived reliability and professionalism of an app. When the application is smooth and engaging, it means users will most certainly come back for more. Thus, the bases of loyalty and positive word-of-mouth will have been laid (Kalimullah & Sushmitha, 2017). On the contrary, applications that are clunky, slow, or too complex leave users frustrated. They will be the first to abandon an app and give negative feedback online.

Enhancing User Experience

Enhancement of user experience means adopting a design approach that will enable an application to meet the user's needs and preferences. This is sensitivity towards the behaviour of the users in fulfilling their expectations through features that meet with their objectives. In this regard, personalization may be a matter of remembering the user's preferences, as personalized recommendations establish this feeling of connectedness between the application and its users. More often than not, UX design improvement is an iterative process whereby updates and refinements are continuously being added regarding users' valuable feedback and changing market trends (Kim & Kim, 2016).

Usability Testing

Usability testing is a great activity that helps in the improvement of UX. Allowing developers to see how real users work with an application makes them pinpoint any areas of friction or misunderstanding. For example, if users often abandon certain processes for checkout, that may mean the process is unnecessarily complicated or that instructions are not clear. It enables developers to iterate on design and functionality as the app must align with the user's needs. It consists of testing, analysing the feedback received

and making changes repeatedly until the seamless experience of the app is delivered (Yazid & Jantan, 2017).

Seamless Navigation

Navigation is the cornerstone of UX because it defines how easily users can find and activate features of a product. Good navigation design enables smooth movement within an application and doesn't allow users to get lost or confused. Clear structures of menus, intuitive routes and logical hierarchies guide users towards their aims with real efficiency. An example of this idea is used in Assassin's Creed, making related features be tucked under one menu tab or showing breadcrumbs to indicate a user's position in order to enhance clarity and reduce irritation (Aldayel & Alnafjan, 2017).

Responsiveness

Responsiveness describes how well an application works with respect to user satisfaction. Applications should respond to whatever the user is asking of them and do this with as little latency as possible. In addition to reacting to every tap, swipe, or other user-generated event, responsiveness in mobile applications also deals with adjusting automatically to a range of screen sizes and orientations. Applications that have non-responsive interactions with their user's risk alienating a public that, above all, does not have much patience for slow or unresponsive systems (Grant, 2018).

Engagement Techniques

The techniques of engagement are a strong tool for enhancing UX, such as gamification and rewards systems. Progress tracking features, badges and virtual rewards encourage the user to work with the app because this form of approach shows the success of activities. If a fitness app congratulates a user on his milestone, or gives him badges after challenges are completed, then people will keep themselves fit and active in the app. Such techniques make it joyful but at the same time build a sense of achievement that further deepens user loyalty (Staiano, 2022).

Interactivity and Personalization

Interactivity is one of the main attributes of modern UX design since it can turn passive experiences into dynamic ones. Real-time updates, drag-and-drop functions or even active graphs give people a chance to feel more involved and in control. On the other side, personalization-engineered settings, such as light/dark mode or a font size regulator, provide better use of an app since such features meet the individual needs of users (Perea & Giner, 2017). It gives ownership and makes the application more personalized and relevant for wide-ranging audiences.

Ease of Use

Usability is one of the most critical factors in determining the perception that users will have about an app. Applications that are intuitive and easy to navigate reduce the learning curve, thus enabling users to be extremely productive. A user-friendly application reduces complexity by providing features and workflows that make sense and feel natural. Such examples include the categorization of content into clearly labelled categories and simplifying processes that routinely involve a number of steps. This makes it more usable and aims at creating continued interaction (Bähr, 2017).

Clear Instructions

Clear instructions ensure a better user experience, specifically for new users. Tutorials on onboarding, tooltips and contextual guidance help the user learn how to apply the application. For example, a banking application might show its users, step-by-step, how to set up the first transaction with instructions and thereby minimize confusion as confidence grows (Mullins, 2015).

Minimal Clicks

The other crucial component of good UX reduces the number of actions a user has to take to achieve something. Applications that enable users to achieve what they want with as few clicks as possible spend less time and energy on behalf of the users themselves and therefore make the latter even more effective and satisfied. For example, integration of different autofill features or making shortcut buttons for very frequent actions may facilitate workflows and make the app more convenient for people (Loeffler et al., 2021).

Error Prevention and Recovery

Prevention of errors and recovery from them is crucial for retaining the trust and satisfaction of a user. Applications should be designed to minimize error occurrence through features such as real-time input validation and unambiguous warnings. Finally, when errors happen, the app should offer constructive feedback and solutions, such as automatic suggestions for error correction of invalid fields or giving an opportunity to undo some of the recently performed actions therewith enable users to recover fast and 'smoothly' from a situation (Muslim et al., 2019).

The Role of Feedback in UX

Feedback mechanisms are integral to giving a responsive and interactive user experience. Either visual or auditory, such as confirmation messages, animation, or sound effects, users are reassured that their actions have been indeed recognized and processed by whatever system was the target of their actions. For example, a progress bar showing

upload completion provides clarity and does not leave the user guessing what happens with the app. Good feedback fills the gap in user actions and system responses, hence building trust and satisfaction (Richardson & Campbell-Yeo, 2021).

Challenges and Future Directions

Despite the evolution that has occurred in UX design, there is still much to overcome in the way of diverse needs being met within a worldwide market. Conversely, the rapid pace of technological evolution drives them to keep current with very emerging 'in' trends and user expectations. Technologies such as AR and AI open exciting perspectives for more immersive and adaptive user experience. Integration of these technologies while maintaining simplicity and ease of use is, however, quite difficult (Perea & Giner, 2017). As UX continues to develop, a developer must balance innovation and user-centricity for applications to remain current and competitive in this vibrant digital space.

2.3.4 Summary of Findings

In today's competitive market, UI and UX play a huge role in guaranteeing the success of an application. While UI will be focused on creating good-looking visuals and interactive designs, UX will be tasked with ensuring that the user gets a seamless and satisfying experience when using the app. Both integrate to form the core for developing applications that will not only attract users but retain them too.

The literature is full of the need to merge aesthetic approaches with functional usability. While great UI design keeps user's attention by using visual harmony, minimalistic layout and easy-to-access capabilities, effective UX strategies keep them by ensuring ease of use, interactivity and responsiveness. Finally, personalization options, such as light and dark mode, also promote greater user satisfaction by paying attention to individual preferences. Although the design principles and technologies have grown significantly, there are still disappointments in meeting the varied expectations of the users and at the same pace that the emerging trends demand. Artificial intelligence, augmented reality and such other innovative tools open a new frontier of development to better both UI and UX. Therefore, research in this area in the future should go in this direction to bring more dynamic and adaptive solutions to the evolving demands of the mobile user.

Only by embracing user-oriented design and technological innovations can developers create products that are enormously useful, therefore relevant and successful, whatever the changing condition is in the digital world.

2.4 Why is React Native better than others for this type of project? Technologies and Frameworks

2.4.1 Introduction

Mobile development has grown at a high speed with many different frameworks being developed to make this process easier and extend cross-platform capabilities. One of the more popular ones is React Native a cross-platform framework that enables developers to build mobile applications with the use of JavaScript, a language that is fairly well-known among web developers (Danielsson, 2016). Its main advantage, unlike other local development methods, is that React Native allows the use of one codebase working not only on iOS but also on Android. This is a major benefit that has allowed it to be highly effective in reducing development time and cost while minimizing maintenance needs (Paul & Nalwaya, 2019).

The choice of the right framework decides everything, from the effectiveness of developers' work on an application to the user experience. But the thing that developers have to trade between usually is the flexibility of native languages and the convenience of platform-independent frameworks like React Native. Android development can be done using Android Studio with either Java or Kotlin programming languages, while iOS applications are normally developed in Swift. Each of these platform-dependent solutions has some great advantages such as the vast usage of Java, the concise syntax of Kotlin, and the optimization of Swift on iOS (Smyth, 2018; Ardito et al., 2020). On the other hand, frameworks like Flutter using Dart come with one codebase for both Android and iOS. However, React Native's use of JavaScript combined with extensive community support makes it highly attractive to teams looking to target platform-independent capabilities without the overhead of a steep learning curve compared to less widely adopted languages (Biessek, 2019).

This review summarizes the literature on the advantages generally attributed to React Native in comparison to alternatives, focusing on development efficiency, community resources, and compatibility. This analysis supports React Native as a solution that aligns well with goals for rapid development and effective cross-platform deployment.

2.4.2 Overview of React Native

React Native is a mobile development framework developed by Meta Platforms (formerly Facebook Inc) in JavaScript. It allows developers to build for Android and iOS with only one codebase. This is made possible through its integration with React, one of

the most popular JavaScript libraries for building user interfaces. React Native uses a bridge that transparently serializes JavaScript and deserializes native components. In doing this, it provides React Native applications with an experience and performance close to natively developed applications (Danielsson, 2016).

Basic Description and Main Features

The point of React Native is to enable the developer to create applications on different platforms with one codebase, accelerating development and simplifying subsequent code maintenance. When possible, emphasizing shared JavaScript code, React Native resorts to platform-specific components only when necessary and thus effectively bridges the gap between native performance and cross-platform convenience. This balance offers both worlds as it can perform fully native-like on both iOS and Android without separate codebases (Paul & Nalwaya, 2019). The feature set is completed by hot reloading that enables developers to instantly see changes without recompilation of the whole app and facilitates fast iterations and development cycles.

Advantages over Native Development

React Native provides several strong advantages over traditional native development. The biggest advantage it offers is cross-platform capability, thus giving teams the ability to focus on one codebase for several platforms. It saves money and time and thereby avoids the risk of inconsistencies between an application on Android and iOS (Danielsson, 2016). Developers of JavaScript origin can easily switch to React Native and continue working without having to learn platform-specific languages like Java, Kotlin, or Swift, hence easy to use in teams with prior web development experience.

Other major assets of React Native include a great ecosystem and a broad developer community. Due to the growth in popularity, this framework has triggered a host of libraries, plugins, and tools that help in everything from UI components to state management. It automatically speeds up development by decreasing dependencies on custom code for standard developer's tasks that could be handled through already developed modules tested and refined by the community itself (Paul & Nalwaya, 2019).

Community and Resource Availability

React Native is being used extensively, hence, its community is active and full of resources and tools. This community support cannot be valued enough, since it exempts developers from the need to develop numerous reusable libraries themselves, but instead creates forums and extensive problem-solving documentation, thus allowing for troubleshooting and learning time to be minimized. Lastly, the continuous support from Meta itself has constantly updated the framework so that it performs better, adding

features and improving vulnerabilities in security issues, thus ensuring that React Native stays competitive in modern mobile development (Paul & Nalwaya, 2019).

React Native has plenty of validity for all those teams that want development speed, maintainability, and cross-platform reach in balance. Giving an atomic role to a popular language like JavaScript and the single codebase approach, makes it fit for both Android and iOS needs without losing native functionality or giving any compromise on user experience.

2.4.3 Comparative Evaluation with Other Technologies

Android Studio with Java

Java has conventionally been the most-used language for developing Android and remains supported, along with being a very popular option in Android Studio. Because Java has had such a long-standing position in the industry, its ecosystem is extended, containing many libraries, tools, and resources that support the development of Android applications. Most developers have experience with Java and can therefore dive into the application much quicker, though that depends on the team concerned (Smyth, 2018).

However, Java also possesses certain drawbacks in the development of mobile applications. First language verbosity tends to make the code long and it is more difficult to maintain. Java is also less flexible compared to modern languages when it comes to handling UI components or managing complex user interactions and common pain points while developing any modern mobile app. Moreover, Java's performance is sometimes criticized for being less efficient than the alternatives that are optimally tailored for mobile environments, especially because mobile application requirements are becoming increasingly sophisticated (Smyth, 2018).

Java also lacks a cross-platform solution reducing the option of the language choice for projects in which simultaneous iOS and Android development takes place. A team choosing this language would most likely go into developing a separate codebase for iOS targets, thereby increasing the time and financial consumption of the development. Thus, while Java provides a robust base for Android applications, it has its drawbacks in terms of the efficiency of code performance and no cross-platform ability, making it less desirable when cross-platform functionality is required or crucial to an application.

Android Studio with Kotlin

Introduced by Google in 2017 as an official language for Android development, Kotlin has come to be thought of as a modern alternative to Java. This was designed to overcome

many limitations that had been identified in Java, it is more concise which reduces the verbosity and the heaviness of Java syntax (Ardito et al., 2020).

This syntax is concise and hence has the effect that the code will be shorter, hence more readable. It reduces the chances of errors due to null values a very frequent cause of crashes in Android apps. For example, Kotlin's type system by design catches most of the null pointer exceptions at compile time itself and this makes the applications much more stable and reliable. It also has syntactic benefits and interoperates well with Java as projects can use both side by side. Interoperability allows room for gradual adoption, teams that used to work with Java can easily switch to Kotlin without the necessity to re-write existing codebases.

However, the major limitation of Kotlin is its orientation for Android it does not provide a direct solution for cross-platform development, unlike React Native and thus can limit its applicability for projects in need of iOS support. Although there are frameworks like Kotlin Multiplatform that try to extend Kotlin's reach toward iOS, they generally are not mature or as well-supported as the cross-platform capability of React Native (Ardito et al., 2020).

While Kotlin as a language is more straightforward for developers coming from Java to learn, the same might not usually hold true for those with no prior background in either Java or Android. The absence of a standardized cross-platform approach in Kotlin makes it even less pragmatic for teams intending to create applications for both Android and iOS without significant additional resources.

Swift for IOS Development

Swift is an open-source language developed by Apple which is the primary language of iOS development. Swift is especially optimized for application building in the Apple ecosystem. Its design emphasizes performance, safety and expressiveness features that make it very attractive to developers for IOS. Swift has much cleaner syntax than its predecessor Objective-C and many advanced features, such as type inference and error handling. It enables developers to type fewer lines of code while reducing the possibility of errors and enhancing readability and maintainability of the code (Grummitt, 2017).

Swift optimization for IOS allows developers to achieve smooth native-like performance across Apple devices, fully utilizing hardware capabilities and providing a seamless user experience. Since Swift was designed with iOS in mind, it aligns closely with Apple's frameworks and tools, making it easier for developers to work on Apple-specific updates and features compared to cross-platform solutions. The biggest limitation with Swift is that it can only run on Apple operating systems, therefore it can only be used for application development on iOS and macOS. This means that any team using Swift

in the development of applications running on iOS must create an entirely different codebase for Android, which adds to its maintenance complexity and ultimately the cost of cross-platform projects (Grummitt, 2017).

Swift may be less practical for projects where cross-platform support is essential, especially since frameworks like React Native simplify the creation of iOS and Android apps with a single codebase. While Swift is optimized for peak performance on iOS, its limited cross-platform support often leads development teams to consider React Native when compatibility with both iOS and Android is necessary.

Flutter Framework with Dart Programming Language

One of the special things about Flutter is that its rendering engine doesn't use native platform widgets. Instead, it relies on the Skia graphics engine to display the Flutter UI across all platforms consistently. That guarantees consistent behaviour and appearance across different devices because the UI is not prone to the vagaries of platform-specific tweaks. Yet this architecture increases the size of an app and may lead to UI elements that are less native-like compared to those made using frameworks tightly connected with the components of each platform. Considering performance despite these differences, Flutter mostly performs just like native applications, which makes it a very good choice for cross-platform projects (Biessek, 2019).

One unique aspect of Flutter is the use of a rendering engine that bypasses the default platform widgets, using the Skia graphics engine yet again. This allows for more consistent look and feel across platforms since the UI doesn't get affected by the specific differences of the respective platforms. With this architecture the app could be larger in size and would possibly lead to some UI elements feeling less native when compared with applications created using other frameworks, which are coupled to UI or platform components. With all these differences the general performance of Flutter is good compared to the native ones, making it a pretty good choice for cross-platform projects (Biessek, 2019).

One of the concerns for Flutter may be that Dart has relatively low adoption and support compared to JavaScript the foundation that React Native is based on. Although Dart is perfectly positioned with regard to mobile app development, it doesn't boast a whole community of developers like JavaScript does therefore teams new to Dart will have a harder time learning the system. That in turn can limit developer productivity in cases where resources of this kind are at a minimum. Furthermore, reliance on custom widgets by Flutter can put maintainers at a higher cost because the UI components tend to drift away from their native counterparts.

In general Flutter is good to go for cross-platform development when consistency in UI across platforms is key. But where a team is comfortable with JavaScript and somewhat familiar with React JS, then React Native might be a much more pragmatic and viable alternative in bringing together cross-platform with a larger pool of resources and developer tools.

2.4.4 Justification for Choosing React Native

Reasons to Choose React Native

Having viewed various frameworks for application development, React Native is number one in the eyes of teams wanting an app to work flawlessly on all smartphone and tablet devices and to ease the processes involved in the development phase. The preference is mainly because of the ability of React Native to support both Android and IOS with just one set of code. This makes it more efficient compared to methods where separate sets of code are needed for each platform. Developers can speed up development using a set of code and easily maintain it, they manage and update the apps with greater ease (Mohammadi, 2019).

One of the strongest suits of React Native is that it can provide a solution that's actually cross-platform yet still retains good performance. Its use of JavaScript makes it easy for web developers to enter the realm of mobile development without having to learn languages specific to each platform, Swift for iOS or Kotlin for Android. This adaptability proves useful for projects, with resources since it lets developers make the most of their JavaScript skills across both web and mobile app development. The JavaScript connection in React Native also permits the integration of modules in cases where performance's vital or platform specific features are necessary. This offers a compromise between efficiency, across platforms and performance at the same level as it would be on Native development (Mohammadi, 2019).

Experience with JavaScript

The ubiquity and familiarity of JavaScript in the development community make React Native accessible. Teams familiar with JavaScript can start developing with React Native quickly, thus, the learning curve associated with React Native is less compared to some new or less-adopted languages such as Dart by Flutter. On top of this JavaScript itself has a huge ecosystem developers can integrate numerous libraries and tools available for JavaScript projects with React Native and doing so would considerably speed up development. This familiarity also means that teams are more likely to find external

support, documentation and resources which are essential for efficient troubleshooting and continuous development (Danielsson, 2016).

Community and Support

The large and active community along with Meta's support contributes to the continuous enhancement of the React Native framework, keeping it competitive and secure. Such a community-driven support system provides developers with a wide range of resources such as forums, plugins and prebuilt libraries that can speed up the process of accomplishing common development tasks. With the regular updates of React Native, driven by both Meta and community contributions, compatibility with new OS versions is kept and it also provides performance improvements, which allows the teams to focus more on application development than on compatibility issues (Paul & Nalwaya, 2019).

One of the great advantages of React Native, especially for those teams that will need much guidance or support during the development process, is the robust community of developers. In some cases, this network of support may be very important in overcoming obstacles and enhancing productivity in educational or collaborative environments where resources are limited. Another advantage of React Native when compared to other frameworks like Flutter with Dart, is the much larger community support.

Overall Justification

React Native's combination of cross-platform capability, JavaScript familiarity and community support makes it the most suitable framework for teams aiming to build applications for both iOS and Android with minimal overhead. Its flexibility in incorporating native modules and large library ecosystem also means that developers can maintain control over performance-sensitive aspects of the application, achieving a nearly native experience while benefiting from the efficiency of a single codebase. Resource accessibility and the large React Native community make troubleshooting, learning and development much easier, which in itself makes this technology quite pragmatic for teams especially those who have prior experience in web development (Danielsson, 2016).

React Native balances cross-platform development advantages with accessibility to allow teams to better reach their goals in mobile development compared to becoming dependent on single-platform solutions like Swift for iOS or Kotlin for Android. This supports React Native as complete and future-proof for contemporary mobile applications, especially in projects targeting wide compatibility, rapid development and sustainable maintenance practices.

2.4.5 Summary of Findings

React Native strikes a good balance in mobile app development, especially for those teams that must target more than one platform with effective development. Driven by a single codebase written in JavaScript, React Native supports smooth deployment on both Android and iOS with considerably less time and effort compared to creating and maintaining separate applications. This places React Native in a formidable position for development teams comfortable with web technologies due to its cross-platform capability, coupled with the ease of access provided by JavaScript (Danielsson, 2016).

A comparative study with other frameworks and native methodologies brings out the strong points of React Native. Where native experience compares Android Studio with Java or Kotlin, iOS with Swift have their limitations concerning cross-platform flexibility. Though Kotlin and Swift offer concise syntax and performance optimization on respective platforms, they require separate development for each platform, which might be resource intensive. While Flutter with Dart also has a single codebase and a reactive framework, the relatively sensible adoption and ecosystem of Dart result in problems for lots of teams that don't have internal knowledge regarding the language.

The wide community support besides the support coming from Meta itself, makes updates common and shared resources like libraries and forums for troubleshooting plentiful. That helps to foster a rich ecosystem where developers can find solutions to problems more efficiently, especially in settings where that support would be highly beneficial. The architecture of React Native, besides using JavaScript-which is readily available, introduces flexibility to the fact that native modules can be included, thus enabling the developers to keep performance-sensitive features under control (Paul & Nalwaya, 2019).

React Native finds a balance between efficiency, performance, and cross-platform reach to make it the best choice for every team that wants to build scalable and maintainable IOS and Android applications. Development teams can build high-quality mobile applications without overhead using existing knowledge of JavaScript and community resources. With original characteristics of flexibility, resource availability and ease of adoption, the set makes React Native future-proof and a strategic choice in modern mobile development.

2.5 Conclusion

This review is a representative selection of how technology has influenced the building up of applications, their focus on user-centered design and learning methodologies. Mobile frameworks like React Native and Flutter are representative examples of technology innovations, allowing for faster development by balancing performance with cross platform compatibility. At the same time, the well-implemented principles of UI and UX ensure that applications are not only visually appealing but also easy to use and highly engaging. The integration of technology has educationally bettered peer learning and academic conversations, breaking barriers of distance and time for collaboration and exchange of knowledge. The findings reinforce the fact that technology within those domains is not only a tool but also a facilitator of efficiency, access and innovation in shaping the future of application and educational ecosystems.

Chapter 3

System Analysis and Design

3.1 High-level Use Cases (Specifications)

3.1.1 User Registration

Description: A new user should provide account details such as name, email, password, course, and year in which he is studying

Actors: New user (a Student who has not signed up yet)

Preconditions:

- User has the installed app on his/her mobile and opens the same.
- User should not already have been registered on ThreadUD
- The user must be a currently registered student at TU Dublin

Postconditions:

- The details pertaining to the user account are created and stored in the system database
- User is either taken to the login page for signing in after successful registration

Main Flow:

- The customer reaches the "Register" option on the homepage or tries to create/join a thread or even comment on a post, after which he will be forwarded to the Register page

- The application opens a register form and asks the user to provide their name, email, password, course, and year of study
- The user fills in the registration form and submits it
- The application checks the validity of the form fields, such as email format or password strength, for instance
- The user details get accordingly stored in the database if the above validation passes
- A success message is displayed while the user is either forwarded to the login page or automatically logged in

Assumptions:

- The user will have a stable Internet connection
- A valid email address can be inputted with a strong enough password
- The system should keep users' data securely, especially the password
- This can be possible by using an encrypted password

3.1.2 User Login

Description: A registered user who has logged in accesses greater functionality of the system using their e-mail and password

Actors: Registered user

Preconditions:

- The user has already created an account and has their login credentials (email and password)
- The login form must be available

Postconditions:

- User is successfully authenticated and then logged into the system
- The user is then taken to the home page, where account restricted features are accessible, such as posting, joining threads and commenting on other's posts

Main Flow:

- The user navigates to the homepage and clicks on "Login" or is redirected to the login page when trying to use an account restricted feature
- The app will provide a login screen that asks the user for their email and password
- In that, the user fills in the appropriate fields and submits the form
- The app verifies the provided credentials against the stored data
- And if the login is successful, then the user is provided access and he is taken to the home page
- In case of failed login, because of an incorrect password or email, it throws an error message and the user is prompted to retry

Assumptions:

- It assumes that the user remembers his login credentials or at least has them handy
- The system has encryption and security features that ensure credentials of users are kept safe
- The Internet connection must be stable to allow authentication

3.1.3 View Home Page

Description: Both a logged-in user and a guest can view the home page and has the ability to view all posts on threads. Guests can view threads but would need an account for interacting on the site to make a post or join a thread

Actors: Any user, either guest or logged in

Preconditions:

- The user opens up the app, whether they were logged in or not
- The home page and its contents are accessible regardless of the user's login status

Postconditions:

- Users who sign in will have additional options to join threads or make posts

- Guests can view and search threads, but they will be requested to log in or register if they make an attempt to interact with the content

Main Flow:

- User goes to the app and is taken directly to the home page
- On the home page, it lists a variety of threads related to the modules and their posts
- It enables the user to either scroll through posts or use a search bar for any particular thread
- If a user clicked into a thread, they would then be able to view all the posts within a given thread
- If the user attempts to enter the thread or make a post, the app checks whether he is logged in
- What they can do is proceed with their actions if they are logged in
- In the case of not being logged in, users are invited to log in or register

Assumptions:

- Home page content (Threads and Posts) are publicly viewable
- The application also provides a search feature, which can be used both by guests and by users who have logged in
- It has to be connected to the internet to show the threads and posts in real time
- For every logged-in and guest user, there should be a variation in UI elements, such as a button or an option, depending on his/her log-in status

3.1.4 Join a Thread

Description: A logged in user can join the thread on any particular module for discussion

Actors: The logged in user

Preconditions:

- The user has to be logged in

- The thread for the respective module or topic has to be available

Postconditions:

- The user becomes a part of the thread and is considered as a participant to it
- They can view or make a post in that thread

Main flow:

- The logged in user goes directly to either the home page or the search page in order to locate a thread
- The user selects a thread they would like to join
- The application checks whether the user is logged in
- If they are logged in, the application presents an option to 'Join Thread' this could be in the form of an icon or button
- The user clicks the 'Join Thread' button
- The backend adds this user as a participant of the thread update in the database
- This thread now appears under the user's joined threads list
- The user now can actively participate in the posting or commenting of this thread

Assumptions:

- User has already logged into the app and finished the registration
- The application will have the capability of tracking user membership within threads, meaning a user can be part of more threads
- One thread should have the ability to let multiple users in without performance degradation
- The internet connection should also be stable enough to support real-time interaction

3.1.5 Create a New Thread

Description: The logged-in user opens a new thread for the module or topic that does not have any thread open yet

Actors: The logged-in user

Preconditions:

- The account should have been logged in
- The thread of the particular module and topic does not exist

Postconditions:

- A new thread has been opened and other users can see all threads, whether a guest or a logged-on user
- We can immediately join the creator to this thread as the first participant in this thread

Main Flow:

- The user has logged in and arrived at the "Create Thread" page
- The application requests that the user provide input for the details of the new thread. Among these are, Module or topic name and Thread description/rules
- The user fills up his/her information and then submits the form
- The app validates the input, meaning for instance that the thread title is not blank and that there does not already exist a thread with that title
- If the input is valid, the app creates a new thread in the system's database
- It will now appear on the home page for all users to see
- The creator of the thread automatically joins the thread
- A success message should be displayed, and the user should be routed to the page of the newly created thread

Assumptions:

- The user is already logged in

- First of all, the system checks if there was already a thread on the same topic/-module so that there are no multiple threads related to the same topic
- The system shall possess the necessary back-end infrastructure required to create, store, and display threads to the users
- This operation must take place with direct Internet access to perform the creation or update of a thread in real time

3.1.6 Make a Post

Description: A logged-in user can post in a thread to ask or share any information that is related to the module

Actors: Logged-in user

Preconditions:

- The user is logged in
- The user has joined a thread in which he/she wants to post

Postconditions:

- The post gets successfully submitted and is visible to other users within the thread
- The post could also be read by logged-in users, including guests, but it could have comments only from this logged-in user

Main Flow:

- The logged-in user navigates to the thread where they want to create a post
- The application provides the user with a text box or more correctly, an input field where they'll be able to write their post
- The user creates their post in a text box, writing the post text, links, and other forms of media if the application allows them to do so
- The user submits the post by clicking a "Submit" or "Post" button
- The application checks the content of the post for example, for an empty submission or for rule-breaking content

- If the post is valid, then the app saves it into the database and visualizes it within the thread
- Other users can view that post within a thread now
- A success message is shown with confirmation that the post has been made

Assumptions:

- A user has already logged into the app and joined the thread
- The system is capable of storing and displaying posts in real time to all users
- The app may flag certain posts through a moderation system for not being appropriate
- The internet connection is good, thus the post can be submitted and shown

3.1.7 Like a Comment

Description: Like a comment in a thread of a post, to show approval or agreement with what has been said

Actors: The logged-in user, initiating the like

Preconditions:

- The user has to be logged in to their account
- The comment they decide to like should, of course, be present in a thread
- The user has not liked this comment in the past to avoid the duplication of likes from the same user

Postconditions:

- A like is sent to the comment, which refreshes the comment's like count
- A record of the user's like is saved in the database to ensure they can't like the same comment more than once
- Other users are presented with the refreshed like count for that comment

Main Flow:

- The user logs in and goes onto a thread, they see different posts and comments
- The user sees the comment they want to like
- The user clicks on the "Like" button/icon next to the comment
- The system checks if the user has liked the comment or not
- If it is a like that has not taken place, update the like count and store the like in the backend (database)
- The like count of the comment is updated and presented to all users who can see the thread
- Provide feedback to the user, such as a small visual update or short message, for a successful action
- If the user tries to like the comment again, the app prohibits this action (optional)

Assumptions:

- The user has logged in to the application and does have permission to interact with comments
- The application has a backend infrastructure to store and manage likes, ensuring no user can like a comment more than once
- The app's frontend has been set to properly display the count of likes and update them in real time
- Internet connectivity is required to update the like count and reflect changes for all users
- Nikita and David will take care of the backend side of this feature, implementing the like interactions correctly and making sure it is reflected properly in the database

3.1.8 Comment on a Post

Description: Current user comments an existing post in a thread as a response or feedback or further input

Actors: User is logged in

Preconditions:

- The user has been logged in
- The user joined the thread from which the post exists
- There is a post existing in the thread

Postconditions:

- The user's comment has been added to the post and thus is viewed by other users in the thread
- The thread has been refreshed and the comment appears under the post for view and further interaction

Main Flow:

- The user having logged in, opens the thread in which the post on which he wants to comment is present
- In most cases, after clicking the post, the user sees an option to add a comment a text field
- It is here that the user types in his comment in the provided input
- The user clicks the "Comment" button
- The app provides some kind of input validation: an input check against void or inappropriate comments
- Once it's been successfully validated, that comment gets added, through the app, to the post in the database
- The comment should come under the posting under the thread and now can be viewed to be commented on by the other users

Assumptions:

- The user must be logged in with the permission to comment on this thread.
- Because it is this kind of system, the content may refresh in real time, so comments should appear instantly once posted
- Internet connectivity is sufficient for a user to post comments and display them
- The system may include moderation or filtering against comments with inappropriate content

3.1.9 Search for Threads

Description: A guest or logged-in user shall search for threads by module names, topics, or keywords

Actors: Any user may use the system as a guest or a logged-in user

Preconditions:

- The user shall have access to the app's search feature
- Threads and posts must be pre-existing within the system

Postconditions:

- The result of the search lists threads relevant to what the user inputs
- Each of these results will be clickable to thread view with posts

Main Flow:

- The user enters the search functionality: this could be through a search bar on the home page or through a dedicated search page
- The user types their query in the search bar: this could be the name of a module, a keyword, or even a topic
- The user submits the search query
- The app will query the database for threads that match the entered search terms
- The user views the search results, where the page enumerates all thread titles and short descriptions
- Any of these results can be clicked by the user in order to view the full thread

Assumptions:

- The search facility in the app is available to both guests and logged-in users who have accessed it
- The search algorithm of the app is capable of matching threads by keywords, module names, and relevant topics
- It supports stable Internet connectivity for immediate searching and retrieving outcomes
- It can allow different search queries to attain their effective and relevant results

3.1.10 Update Profile

Description: A logged user would be able to edit their profile, changing their name, email, course, or year of study

Actors: Logged-in user

Preconditions:

- The user must be logged in to their account
- The user's profile information should already be present in the system, ready to be modified

Postconditions:

- User profile is updated with the new information, saved to the database
- The profile data is shown in other areas of the app where the profile info may be used, such as when making posts

Main Flow:

- User opens the profile page
- User chooses the option "Edit profile"
- The existing profile information is displayed
- The user can change any of the following fields: Name, Email, and Course or year of study
- The user changes the desired fields and submits them
- The app validates the fields entered by the user, for example, checking if the email submitted is in a valid format
- On validation, the application updates the profile data in the database
- After posting a success message, this updated profile shows up everywhere in the application, such as in posts or thread interactions

Assumptions:

- The user can view their profile after opening an account in the app

- The system shall be capable of checking input provided by the user and controlling the integrity of data, including the format of email addresses
- The system shall be properly secured for the handling of sensitive information such as emails or passwords
- Internet shall be used to store updated profile data into a backend database

3.1.11 Receive Push Notifications

Description: Through the application, users receive push notifications whenever there are new posts or comments within threads they are following or participating in

Actors: User logged in

Preconditions:

- The user must be logged in and allowing push notifications for the app
- The user has to join or follow some threads to get updates on the same
- The app must have notification access permission by the user

Postconditions:

- The user's device provides a notification when there is new activity in the user-followed thread
- There is also a displayed clickable link, which leads to the relevant thread or post for the user

Main Flow:

- The user grants permission for push notifications either at the prompt or through the settings within the app
- The user interacts with or follows certain threads
- If there is a new post or comment within a followed thread, it will initiate a push notification via the backend system
- The push notification is delivered to the user's device

- The notification will appear for the user on their device, accompanied by a short message like "New post in [thread name]"
- The user taps on the notification to open the app and is forwarded to the relevant thread or post

Assumptions:

- The user has allowed the settings, including push notifications from their device for this application
- The application has some backend functionality regarding push notifications and triggers the push notification on events such as a new post or reply
- The system should be able to send numerous notifications without overwhelming the user
- The internet connection is good enough for sending and receiving to be in real time

3.1.12 Log Out

Description: A logged-in user logs out successfully from their account, resulting in the close of the current session

Actors: User logged in

Preconditions:

- The user has already signed in
- There is a Logout option available in the application UI

Postconditions:

- User is logged out of their account, and the session is terminated; the user is taken back to the login page or home page where posts can be viewed as a guest

Main Flow:

- The user opens the application and directly goes to the account or settings section
- The user presses the "Log Out" option

- The application asks for confirmation from the user regarding logging out
- The app kills the session and logs out the user upon the positive confirmation by the user
- The user is taken to the login page or home page where they can browse as a guest without being able to post or join threads
- A confirmation message may be displayed for successfully logging out

Assumptions:

- The user is considered to be already logged in
- An easily accessible log out option is provided by the app
- The system correctly clears the session and ensures the user is securely logged out
- In case of a logout, the session token or authentication data of the user is marked invalid to avoid unauthorized access
- Logging out might need internet connectivity in order to log the user out from the system

3.1.13 Moderator Manages Posts

Description: A moderator reviews, edits, or deletes posts or comments that do not comply with the community guidelines of the application

Actors: Moderator (a user with special privileges)

Preconditions:

- Moderator is authenticated
- Moderator has special admin privileges to manage posts and comments
- Some posts or comments are flagged for moderation or need a review

Postconditions:

- The moderator inquires and modifies posts or comments
- Based on the action taken, the post or comment may be edited, deleted, or flagged

- The moderator notifies other users if needed (e.g., the post is deleted and the user is informed)

Main Flow:

- The moderator logs into the app and proceeds to the moderation dashboard, if there is one, or views flagged posts within threads
- The moderator selects the post or comment to be moderated, which could be in response to a user flagging or a manual selection
- The app displays different moderation options, including editing, deleting, setting for review, and issuing a warning to a user
- The moderator makes a choice of action (e.g., to delete a rude comment or edit a post)
- The chosen action is performed, and the database is updated
- Users affected by the action may be informed via a notice, such as "post deleted" or "warning sent"
- The thread or post is updated for all users with respect to the moderation action

Assumptions:

- A moderator is somebody with special rights who can moderate entries made in the system
- The system provides tools for moderation and facilities for easy recognition and identification of problematic posts
- The app enforces community guidelines and highlights inappropriate content through measures in place for detection or flagging
- A stable internet connection is required to ensure any changes made by the moderator appear in real time
- The app may store logs or history of moderator actions for audit purposes

3.1.14 View my Posts

Description: A user views all the posts they have made across all threads

Actors: User, posts, threads (name)

Preconditions:

- User has an account
- User is logged in

Postconditions:

- The system has displayed all the user's posts if they have any

Main Flow:

- The user selects the profile button to navigate to the profile page
- The user selects the "My posts" button
- The system loads all the posts made by the user with thread details
- The system displays all the posts created by the user if they have made posts, otherwise a "Nothing to see here!" message will be displayed

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The user has made posts
- The system can retrieve the user's posts

3.1.15 View my Comments

Description: A user views all the comments they have left on posts, with the post title, and all replies to others' comments, including the comment they replied to

Actors: User, comments, post, thread, replies

Preconditions:

- User must have an account
- User must be logged in

Postconditions:

- The system has displayed all the user's comments

Main Flow:

- The user navigates to the profile page
- The user selects the "My comments" button
- The system loads all the comments made by the user with post and thread details
- The system displays all the comments created by the user and the title of the post it was left on, if they have made comments; otherwise, a "Nothing to see here!" message will be displayed

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The user has made comments
- The system can retrieve the user's comments and comment information

3.1.16 Reply to a Comment

Description: A user replies to a comment on a post, or to another reply

Actors: User, comment, post, thread

Preconditions:

- User must be logged in
- User must be on a post's page

Postconditions:

- The system will add the comment information to the database
- The system will display the new reply to the user under the comment/reply

Main Flow:

- The user navigates to the comment they want to reply to
- The user selects the reply button
- The system displays a textbox for the user to enter their reply
- The user enters their reply and presses submit
- The system adds the reply to the user's view of the post page
- The system updates the database with the new reply information

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The system can retrieve all the comments under a post
- The system can update the replies to a specific comment to include the new comment

3.1.17 Like a Post

Description: A user will like a post

Actors: User, post, thread

Preconditions:

- User must be logged in
- User must be on the thread page of the post or the post's page

Postconditions:

- The system will update the user's view to display the post as liked

- The system will update the database to increment the post's like count and add the post to the user's likes

Main Flow:

- If the user is on the thread page of the post, they navigate to the post they want to like
- The user selects the like button
- The system displays the post as liked to the user
- The system updates the database, incrementing the post's like count and adding the post to the user's likes

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The system can retrieve all the post's information
- The system can update the users who liked the post to include the new user

3.1.18 Share a Post

Description: The user will share a post with another user through another social media app

Actors: User, post, thread

Preconditions:

- User must be logged in
- User must be on the thread page of the post or the post's page
- User must have another app to share the post through

Postconditions:

- The user will share the post with another user through another social media platform

Main Flow:

- If the user is on the thread page of the post, they navigate to the post they want to share
- The user selects the share button
- The system displays a list of possible apps the user can share the post through
- The user selects the app they want to use for sharing
- The user is redirected to the selected app to choose the recipient

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The user has other social media apps to share the post through
- The system can retrieve the post information
- The system can generate the link to be sent with the post information

3.1.19 Forgot Password

Description: If a user forgets their password when logging in, they can reset it

Actors: User

Preconditions:

- User must have an account
- User must be logged out of the app

Postconditions:

- The user will be logged back into the app
- The system will update the database with the user's new password

Main Flow:

- The user enters the app and selects the login option
- The user selects the "Forgot Password" button
- The system directs them to a page to enter their email address
- The user enters their email address
- The system generates a random 6-digit code and sends it to the user's email
- The system displays an input box and asks the user to input the code
- The user enters the code from the email
- If the user enters the wrong code, the password reset process will terminate, and the user will be redirected to the login page
- If the user enters the correct code, the system will display a text box for the user to enter their new password
- The user enters their new password
- The system logs the user in and directs them to the home page
- The system updates the user's password in the database

Assumptions:

- The user will have a stable Internet connection
- The user has an account
- The user enters the correct email address
- The user still has access to this email account
- The system can send a verification code to the user's email
- The system accepts the correct code
- The system can update the user's new password

3.2 Use Case Diagrams

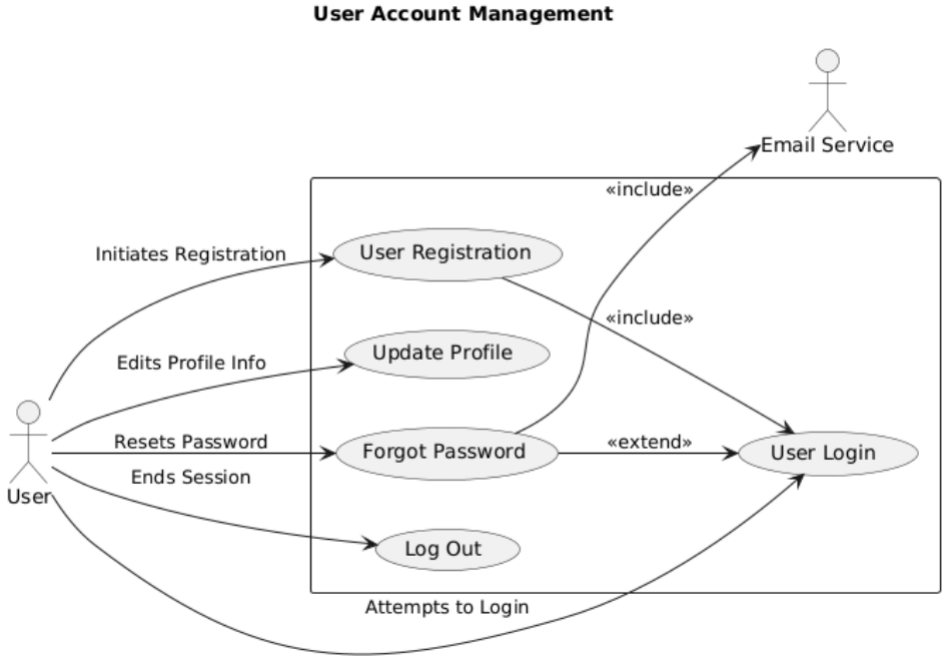


FIGURE 24: User Account Management Use-Case diagram (Author's own screenshot)

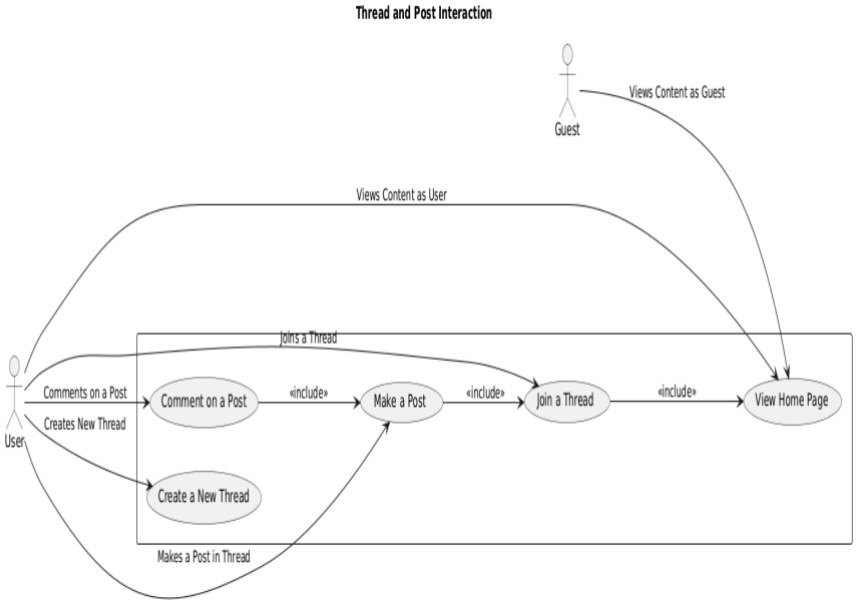


FIGURE 25: Thread and Post Interaction Use-Case diagram (Author's own screenshot)

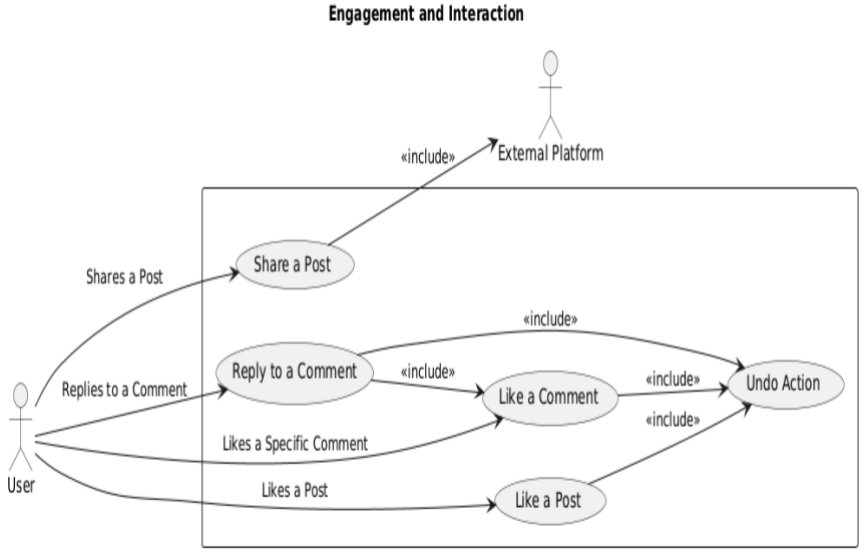


Figure 3 - Engagement and Interaction Use-Case diagram

FIGURE 26: Engagement and Interaction Use-Case diagram (Author's own screenshot)

Figure 3 - Engagement and Interaction Use-Case diagram

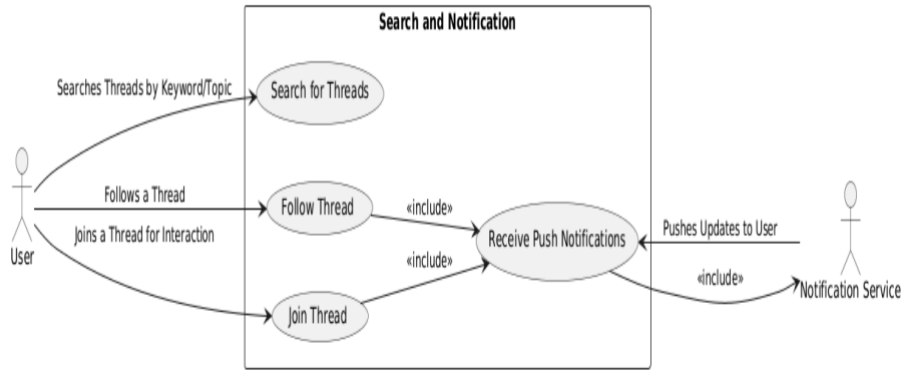


Figure 4 - Search and Notification Use-Case diagram

FIGURE 27: Search and Notification Use-Case diagram (Author's own screenshot)

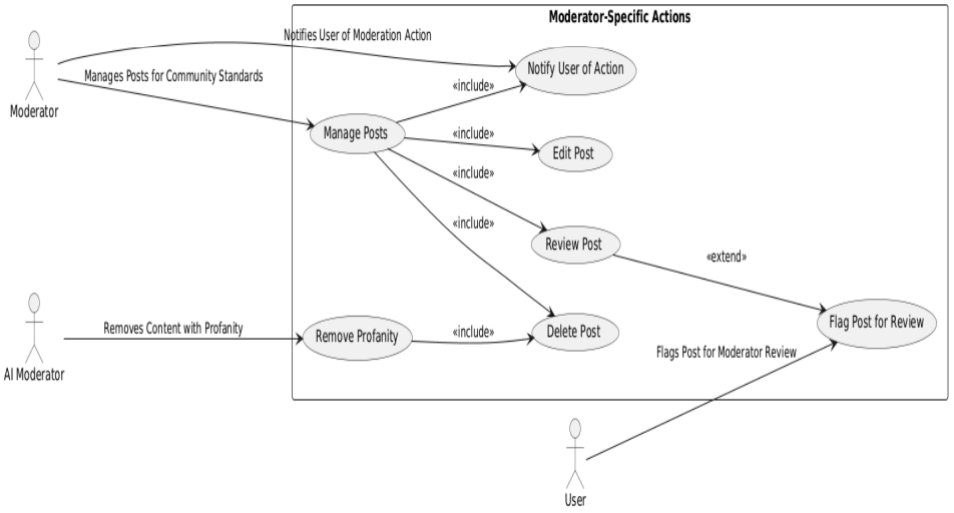


FIGURE 28: Moderator-Specific Actions Use-Case diagram (Author's own screenshot)

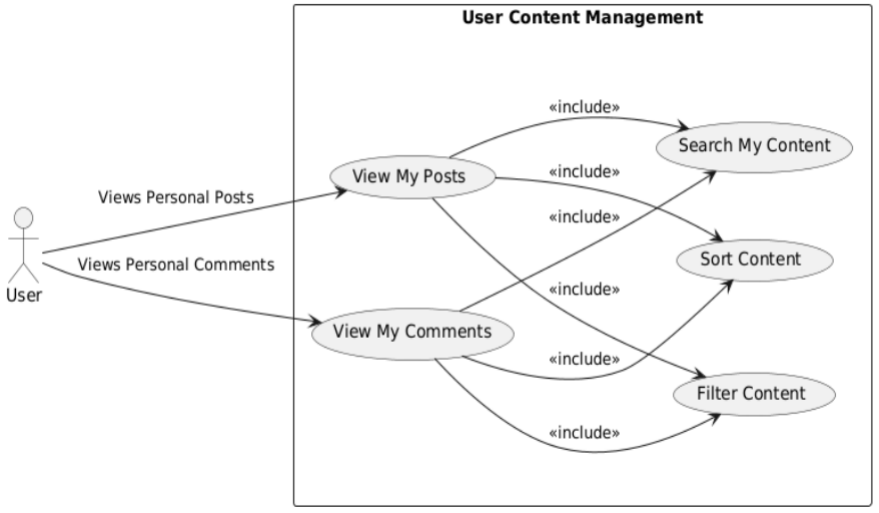


FIGURE 29: User Content Management Use-Case diagram (Author's own screenshot)

3.3 System Design

3.3.1 System Diagrams

3.3.1.1 Entity Relationship Diagram

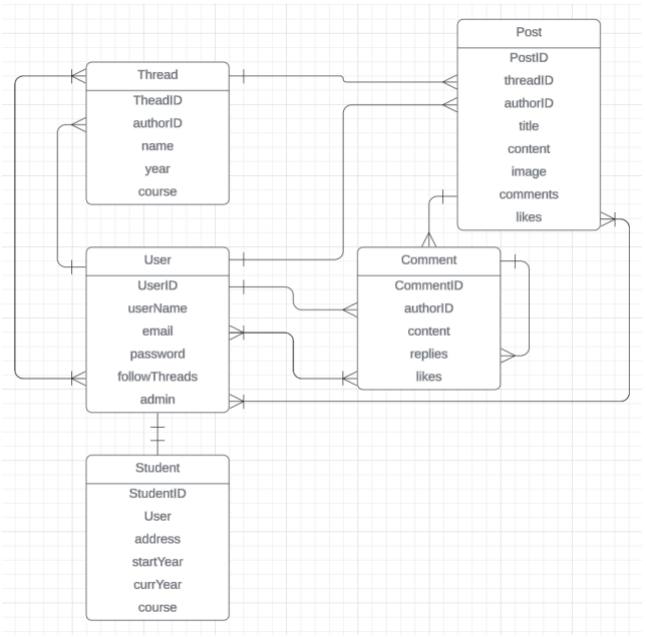


FIGURE 30: ERD diagram for Mongo Database (Author's own screenshot)

3.3.1.2 UML Diagram

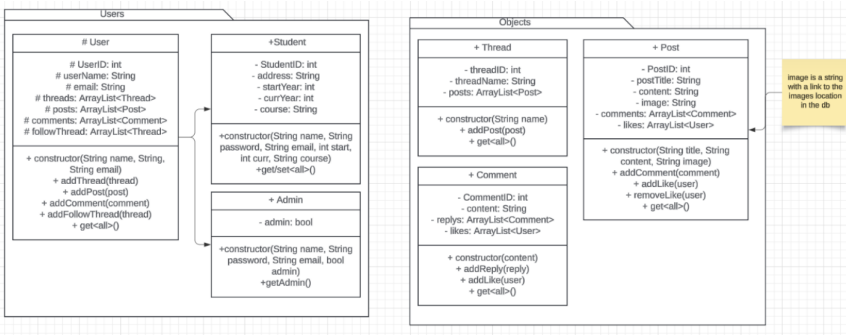


FIGURE 31: Class Diagram (Author's own screenshot)

3.3.2 High-end Detailed Wireframes

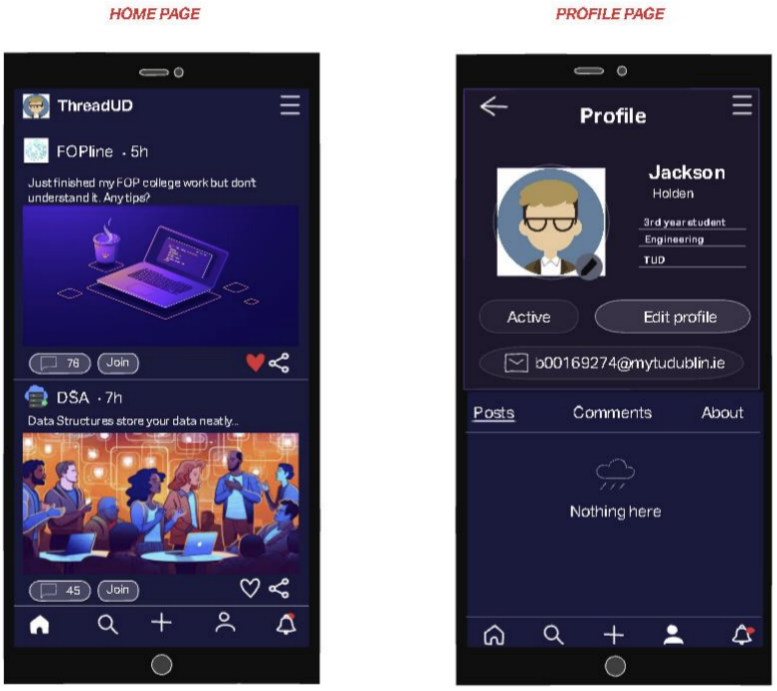


FIGURE 32: Home and Profile pages Wireframes (Author's own screenshot)

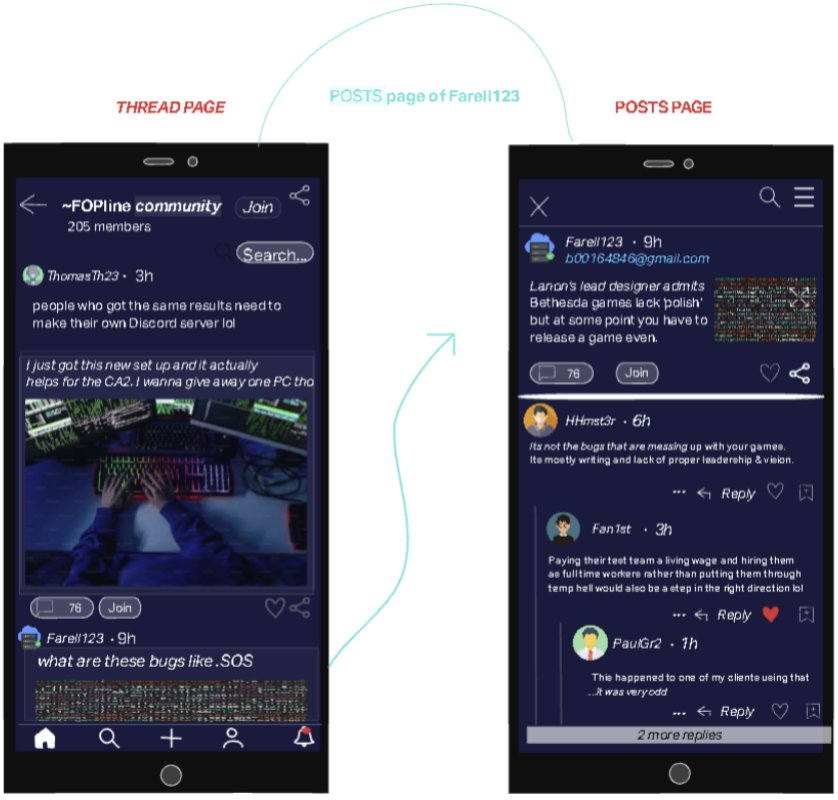


FIGURE 33: Thread and Posts pages Wireframes (Author's own screenshot)

Chapter 4

Implementation of System

4.1 Development Methodology

ThreadUD was implemented utilizing the Agile Scrum process, which was best adapted to accommodate the teamwork aspect of the project. The development was segmented into two-week sprints where the developers had specific objectives and deliverables for each iteration. Weekly stand-up meetings were performed to monitor progress, tackle obstacles, and prioritize based on user input and the team. This iterative process helped the system to evolve steadily with continuous increments while fostering fruitful communication and cooperation.

4.2 Tools and Technologies

For supporting cross-platform mobile development, the team chose React Native with Expo for the frontend to have a single codebase that runs equally well for both Android and iOS (https://docs.expo.dev/). Node.js and Express.js were utilized for the backend, with MongoDB and Mongoose serving the purpose of a scalable and adaptable NoSQL database solution (https://expressjs.com/). The real-time aspect was handled using Socket.IO, which supported dynamic, live updating of the comments for discussion threads. Authentication and account security came in the form of bcrypt for password hashes, while Nodemailer handled the workflow for email verifications and recovering passwords. Google's Perspective API was included for AI-driven content moderation for detecting toxic language to maintain a secure learning space.

4.3 Front-end Implementation

ThreadUD frontend was designed to provide an intuitive and responsive experience. Transitions across the Home page, Thread view, Post page, and Profile section were handled using React Navigation. Global state of the application, including the user being logged in or not, along with followed threads, was handled using the Context API, while state specific to the component was handled using React's local state hooks. Screens included modular, reusable TypeScript-styled components that provided consistent visual design and responsive performance across devices. A personalized feed based on the followed threads was shown on the Home page, while the Thread and Post pages included nested replies and real-time updates for a seamless academic conversation experience.

4.4 Back-end Implementation

The backend services have been designed with an emphasis on modularity, security, and high performance. HTTP requests and routes have been handled using Express.js, while Mongoose has been utilized to define the schema-based data models for the application (https://mongoosejs.com/docs/). All the core features have been supported through RESTful APIs that cover user registration, login, post and thread creation, and comment interaction. Password storage has been secured through Bcrypt, while OTP verifications have provided an added layer of account protection (https://www.npmjs.com/package/bcrypt). The backend has also been responsible for real-time communication through the use of Socket.IO, for instant updates upon the addition of new replies or comments (https://socket.io/docs/v4/). Admin users have also been given special rights, with the moderation tools to check flagged content and respond appropriately.

4.5 Data Models

The data models for the application were designed to accurately represent the major entities and relationships of the platform. The User model kept information like username, email, course, year, and followed threads, along with fields for admin rights and authentication. Threads acted as higher-level groupings of academic discourse and grouped in terms of course and year. Posts referenced threads and held the post content, author data, and metadata like likes and timestamp. Comments had nested replies supported, along with user engagement features like likes and flagging. These models

complemented each other to enable personalized feeds, effective retrieval of content, and an ordered discussion space.

4.6 Security and Moderation

Security and moderation were paramount at every stage of the implementation. Passwords were safely stored with bcrypt hashing, while email verification provided a second level of account security through Nodemailer (https://nodemailer.com/). Ensuring a secure academic environment, the app included both machine-driven and human-driven moderation solutions. Google Perspective API checked user-submitted material for toxic or inappropriate language, marking offending material in real-time. Admin users could then evaluate flagged posts and comments from specific moderation views, enabling human judgment and response (https://developers.perspectiveapi.com/s/docs). These steps preserved ThreadUD as a secure, respectful environment for academic discourse.

Chapter 5

Testing and Evaluation

Testing took up a large portion of the time and effort involved in the development of this project, to ensure that the finished product was fully polished and functional. This included a test driven development style, with regular peer reviewing of each other's work at the end of every sprint cycle. User tests with users uninvolved in the development process were also done, at the TUD open day at the Blanchardstown campus on the 3rd of April. Stress testing using locust was also conducted on the backend server, hosted on Fly.io. User interface and experience testing was also conducted to ensure both met the requirements initially set out and finally, use case and requirements testing was carried out to ensure all use cases and requirements were met.

5.1 Peer Reviewed Test-Driven Development

The first testing done was completed throughout the development of the app. As each team member completed a feature/sub feature, the tested it with basic inputs and simple edge cases to ensure it functioned correctly. This was then done again for each feature more formally at the end of each stage of development (prototype, pre-API alpha version, open day beta version and final release). The team also ensured that each team member would peer review the work of other team members before features were committed to the main project branch on GitHub. Tewam members would review the feature code and informing the developer of that feature of any code issues or irregularities, which would be fixed by the developer before committing.

5.2 User Testing

The team had set out from the very beginning of the project with the goal of having a complete (or close too it) beta version of the app ready for testing on TUD's open day on 3rd of April, in the Blanchardstown campus. Students at the open day were given the opportunity to use the app and give feedback about any improvements that could be made to enhance/improve upon the user experience, and to report any bugs/issues the found. The following feedback was collected:

Bugs:

- On the post page, the text bar for comments and replies was slightly too low and was partially covered by the keyboard.

Improvements on design/experience:

- Text could be made bigger
- Buttons could be made bigger
- ThreadUD logo could be used as a home button

The bug mentioned was a result of the fully build and deployed app having some visual differences to the app in testing. This was fixed by editing some of the build settings. Other suggestions to enhance user experience and design were also taken on board and implemented for the final app release (for demonstration on the 8th of May). Aside from these issues, the app feedback was largely positive. This helped the team ensure that in normal use the app functioned as desired, and that the user experience design was successful. However, more testing would need to be done to ensure the app could handle more users and all possible inputs, including unexpected input and edge cases.

5.3 Stress Testing

Stress testing was conducted on the back-end server using locust with python. Locust was chosen as it is a free, industry standard testing tool, which allowed for detailed custom testing, with the creation of different types of users and control of the amount of each virtual user in each test. Overall, testing met our expectations well, with some minor discrepancies. Scripts used: Initially, tests were done with a basic script, with one user who made one query to get all posts.

```python
class BasicUser(HttpUser):
    @task
    def basic_user(self):
        self.client.get("/post")
```

FIGURE 34: BasicUser class (code block)

This was used to ensure that testing worked and returned realistic data (realistic data size).

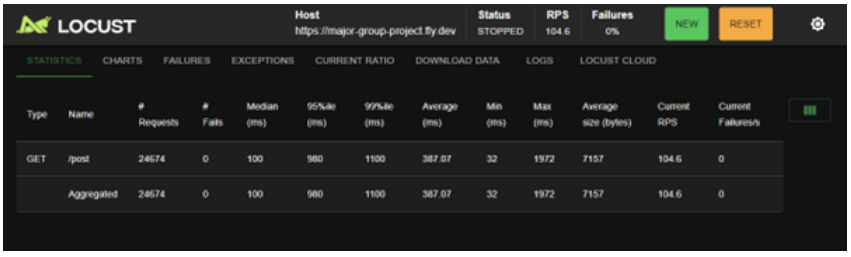


FIGURE 35: Locust results (statistics tab)

Main tests were conducted with a script with several users, carrying out different tasks. The basic user, representing a user who is not logged in, who will go to the index page and then navigate to a post page.

```python
class BasicUser(HttpUser):
    weight = 19
    @task
    def basic_user(self):
        self.client.get("/post") # get all posts
        self.client.get("/post/byThread/", params={"threadID": "67292eba16505c7370748e83"}) # get posts in a thread
```

FIGURE 36: BasicUser class (code block)

The searching users, representing a logged in user who is not following any threads going to the index page, who will then search for a thread, go to the thread page of that thread and then selecting a post on the thread.

```python
class SearchingUser(HttpUser):
    weight = 30
    @task
    def searching_user(self):
        self.client.get("/post/byCourse/", params={"course": "TU860"}) # searching for posts in a course
        self.client.get("/user/courseThread/", params={"course": "TU860"}) # searching for threads in a course

        self.client.get("/post/byThread/", params={"threadID": "67292eba16505c7370748e83"}) # searching for posts in a thread
        self.client.get("/comment/single/", params={"id": "67c51d240243d3d627070791"})# searching for a single post

        # searching for comments in a post
        ids = ["67ed5f50cb8cc7bd885a2006", "67ed5fb8cb8cc7bd885a202e", "67ed6075cb8cc7bd885a2076"]
        self.client.get("/comment/comments/", params=[("id", _id) for _id in ids])
```

FIGURE 37: SearchingUser class (code block)

Finally, an admin user, who will search for flagged posts and comments.

```
class AdminUser(HttpUser):
    weight = 1
    @task
    def admin_user(self):
        self.client.get("/post/flagged") # get flagged posts
        self.client.get("/comment/flagged/") # get flagged comments
```

FIGURE 38: AdminUser class (code block)

All users in tests will be generated at different ratios to simulate actual use conditions, with 60% of users being searching users, 38% being basic users and 2% being admin users. The results of tests with these users were as follows.

Benchmark tests:

Benchmark tests were conducted to see the limits of the server, the amount of users it can realistically support and the fail request fail rates with different numbers of users.

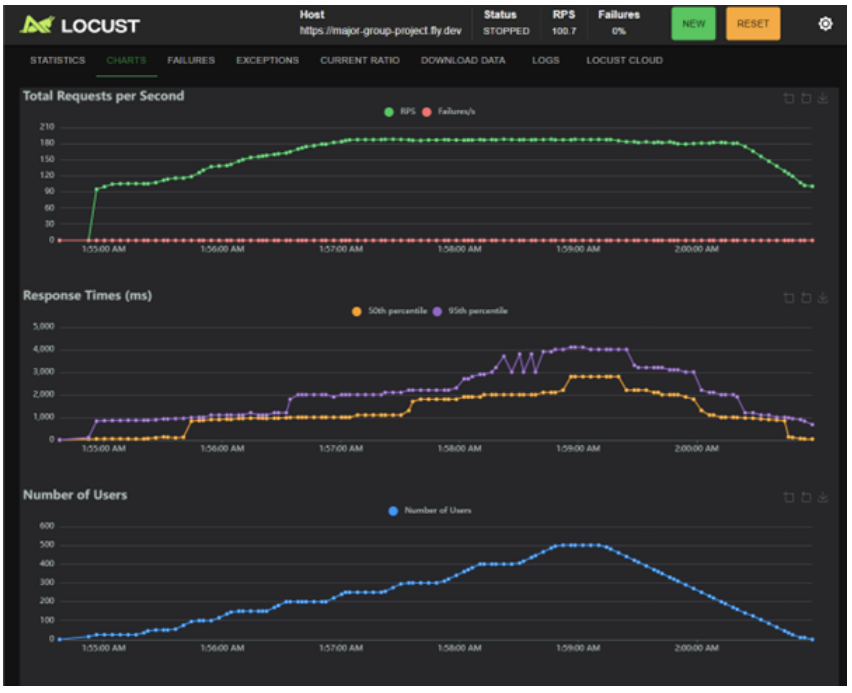


FIGURE 39: Locust results (charts tab)

The benchmark test on this server were very promising, with users having an average response time less then or at 1 second for up to 250 users, with this increasing to 2 seconds for 500 users. These would be considered the upper limits of this server, if 250 or less users in typical app use is expected and 500 or less users at peak use times, this server sufficient. If not, the team will have to consider upgrading to a more powerful server. The server also showed a very promising near 0

Surge tests:

Surge tests were used to simulate a large, sudden surge of users entering the app at once. This could happen, for instance, at the end of lecture/lab periods, after exams etc.

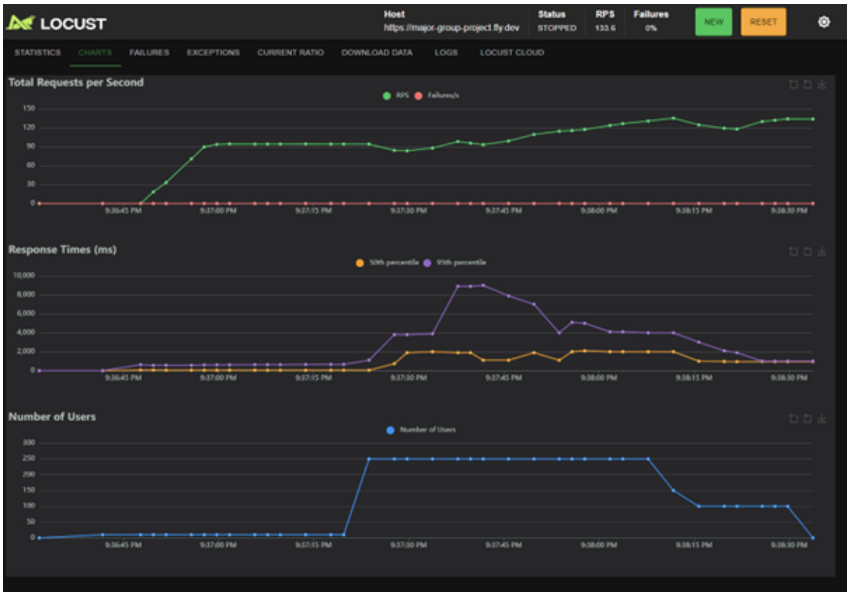


FIGURE 40: Locust results (charts tab)

Serge tests on the server were varying, with some tests coming back with close to the same response times as in general use (i.e., unaffected by sudden spike of users) while some resulted in up to 10 second wait times for some users. Across most surge tests however, the average user maintained reasonable response times, close enough to those seen in general use tests. Despite these varied results, the server continued to maintain a near 0% failure rate.

General use testing:

General use tests are meant to simulate the app use under typical conditions. These tests were ran for longer then benchmark and serge tests, with more variation in user numbers over time.

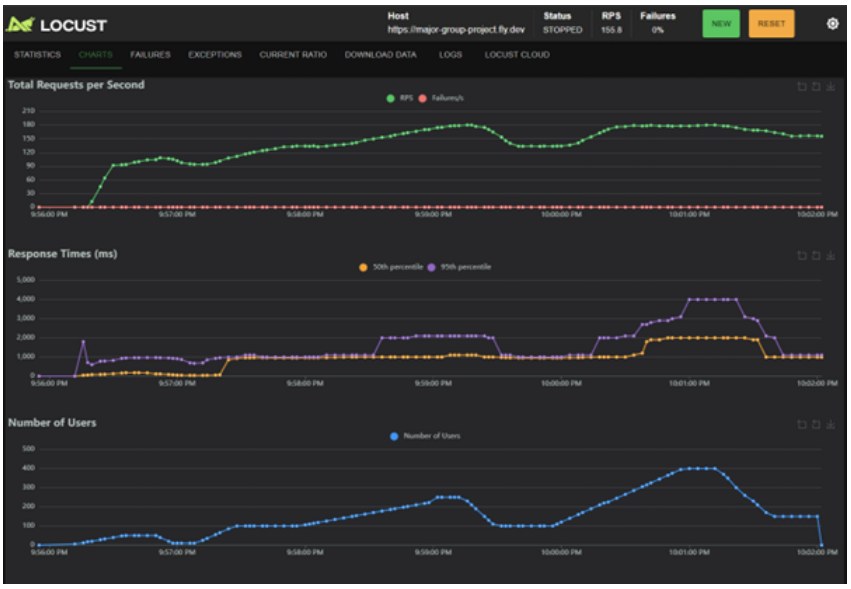


FIGURE 41: Locust results (charts tab)

Overall, these tests were promising. Extended duration of test times didn't greatly vary results and overall showed similar results to benchmark testing, with the variation in response times staying consistent with the number of users. However, these tests did show that if the server had gone for a period of time without receiving any requests (20 - 30 minutes) that initial response could take up too 8 seconds for some users. This dipped down again after the initial 8 seconds of the tests. This likely due to the server going into some kind of sleep setting, or the server CPUs being user by another server hosted by Fly.io while inactive. These tests also maintained a near 0% failure rate.

Overall results of tests:

Overall, these tests indicated that this server will be sufficient for the number of users we expect the app to have when deployed, although traits such as varying response times under serge testing, delays when the server has been inactive and possibly more users joining the app may result in the server needing to be upgraded to something that can handle increased traffic. Several other server were tested, but due to factors such as long delays in response times when entering a sleep mode (up to 15 minutes), limited up time on free plans and not being able to handle the traffic we expect the app to have, they were not chosen for the final app version.

5.4 User Interface and Experience Testing

A large amount of time and research went into ensuring that the user interface and experience design were to an acceptable standard, as we knew that the success of any

app, but especially an educational and forum-based app like this, would be heavily influenced by the user experience and design. Despite this, throughout development and testing, the original vision for the app's look and feel changed, as seen with the original wire frame designs side by side with the finished app.

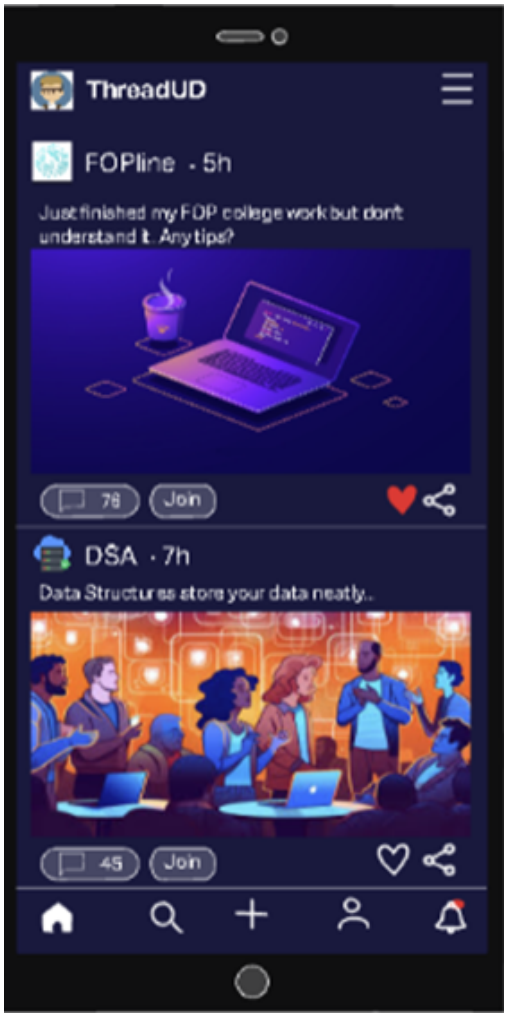


FIGURE 42: Wireframe (expectation)

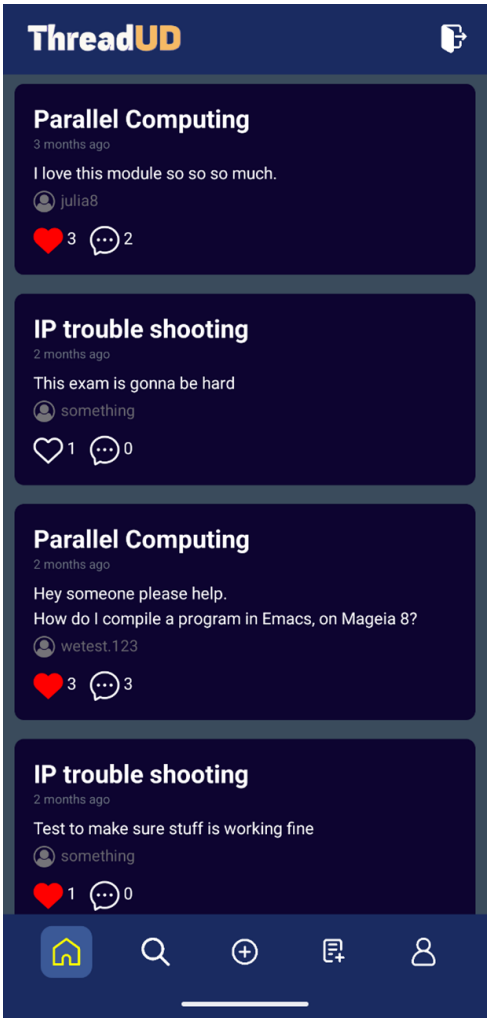


FIGURE 43: Our App (reality)

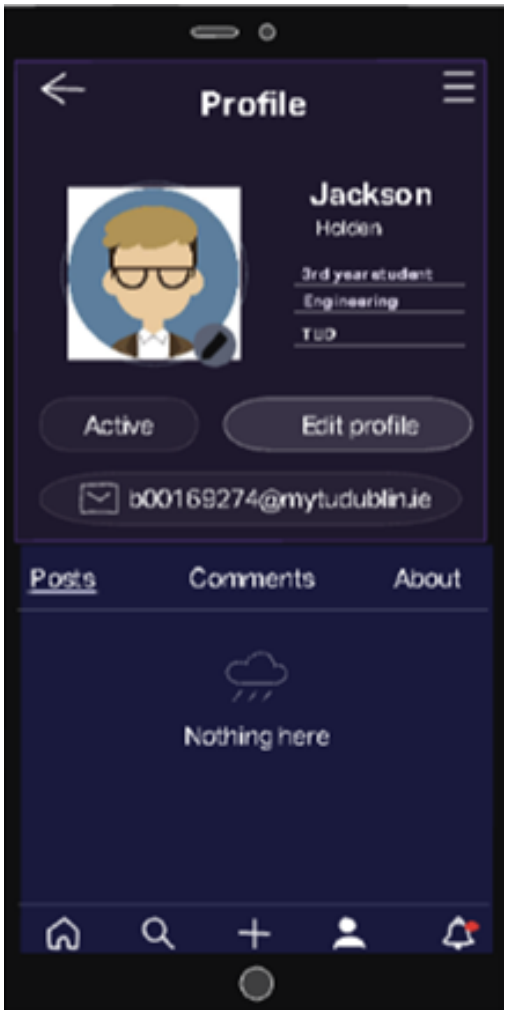


FIGURE 44: Wireframe (expectation)

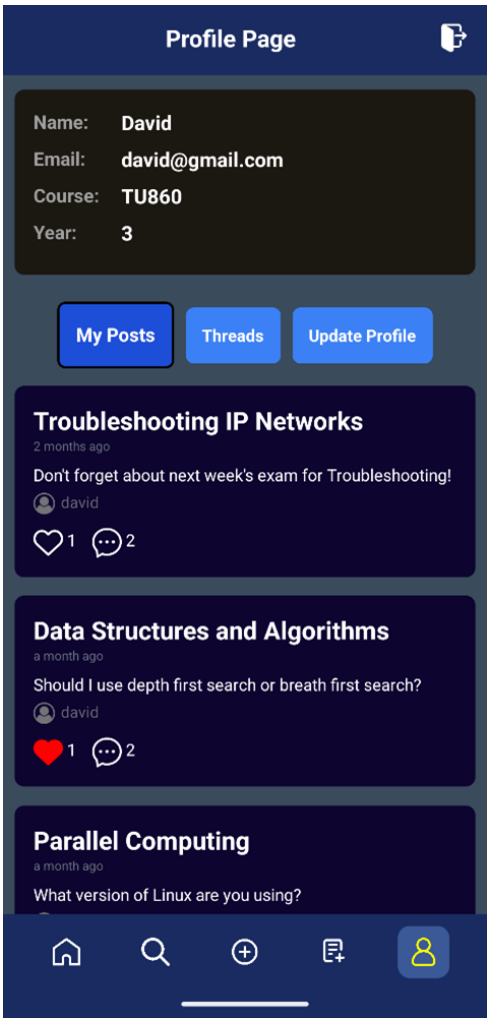


FIGURE 45: Our App (reality)

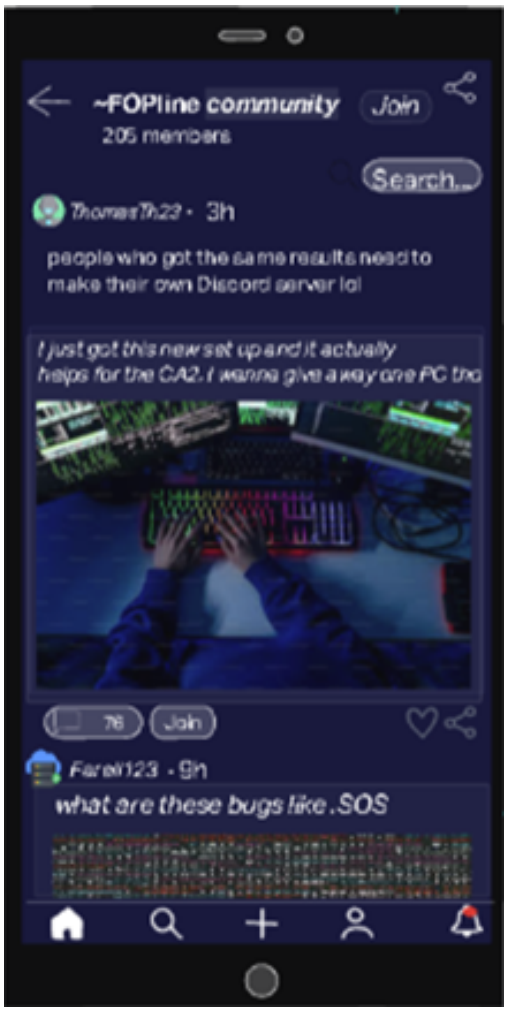


FIGURE 46: Wireframe (expectation)

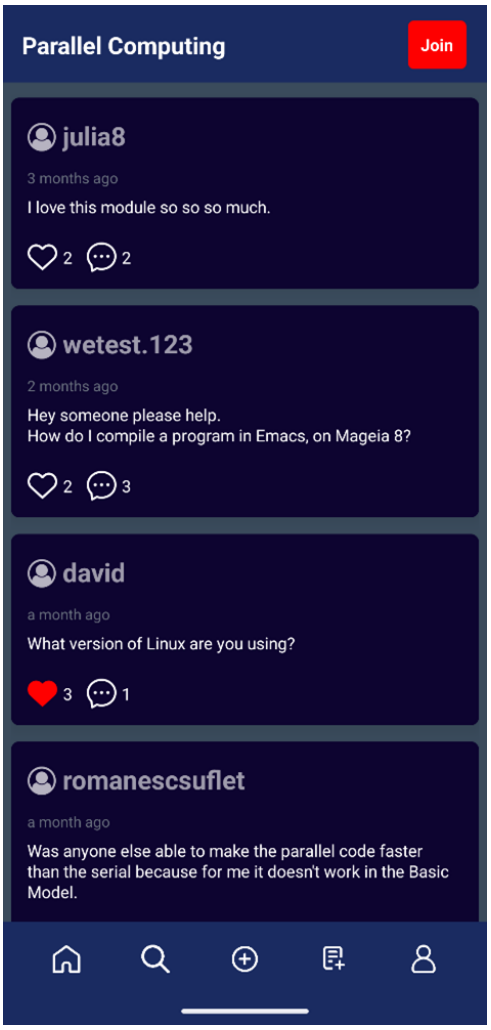


FIGURE 47: Our App (reality)

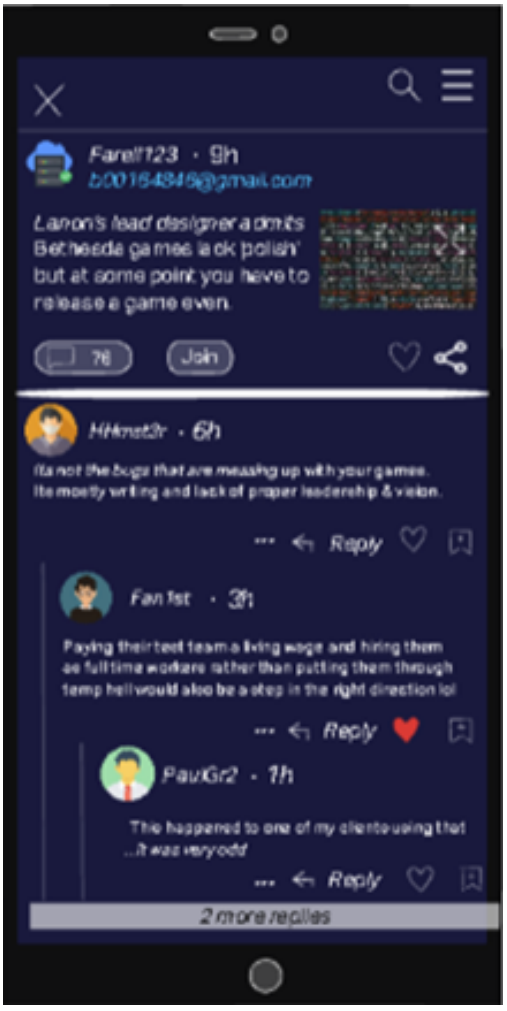


FIGURE 48: Wireframe (expectation)

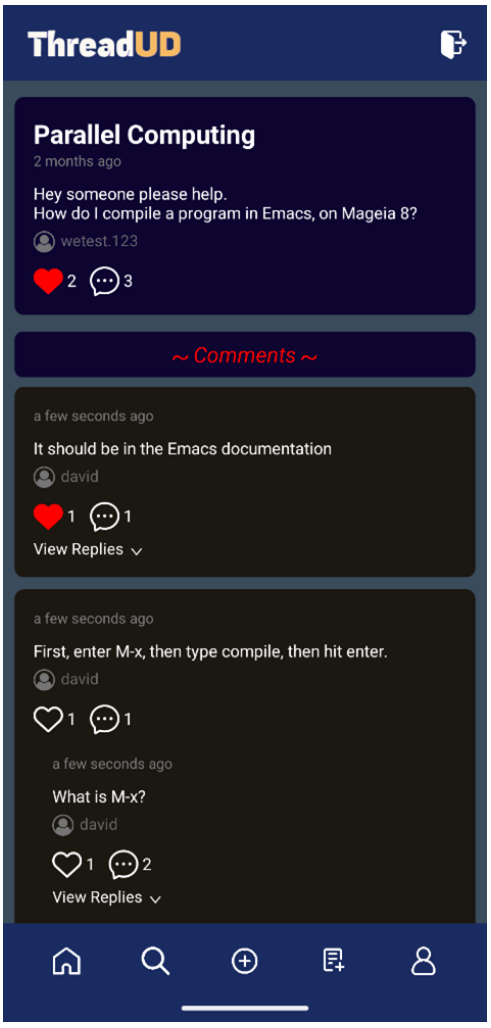


FIGURE 49: Our App (reality)

A prominenet difference between the orignial app design and the finished app is the lack of images, this is in large part due to the fact that Googles Perspective AI moderator, which is used as out in app moderator to flag content inpropriate for our app is unable to scan images (more on this in uses cases) The final app also has brighter and more colours, larger buttons and text, and more spacing between componenets. These changes were made because in testing, these modified designs were found to make the app look more visually appealing and also easier to use as the app elemenets were easier to see and indentify. An important feature for user experience was to provide usable feedback to users when completing actions. In the app, this was done by displaying test boxes to users when they complete an action in the app, to inform the user of the actions completion, or why it failed.

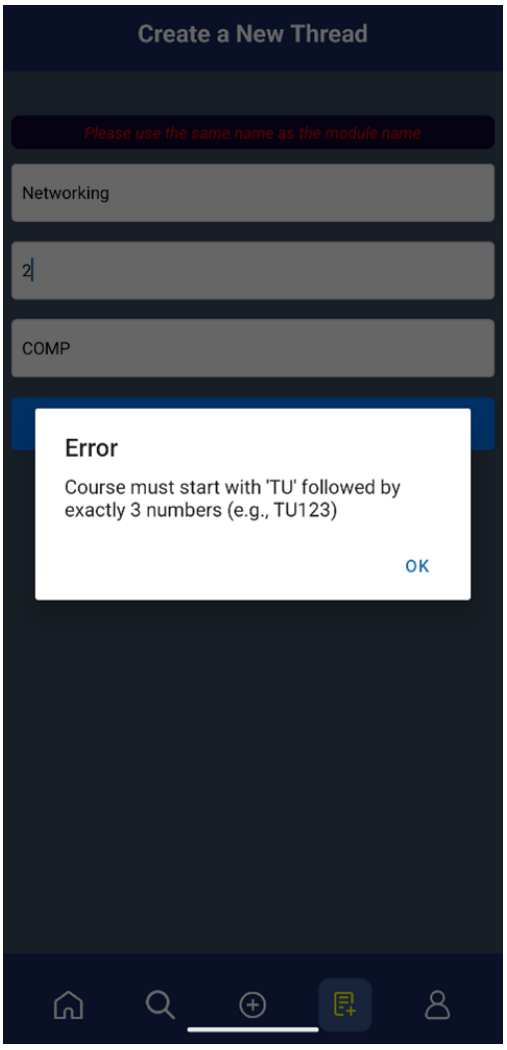


FIGURE 50: In-app feedback pop-ups

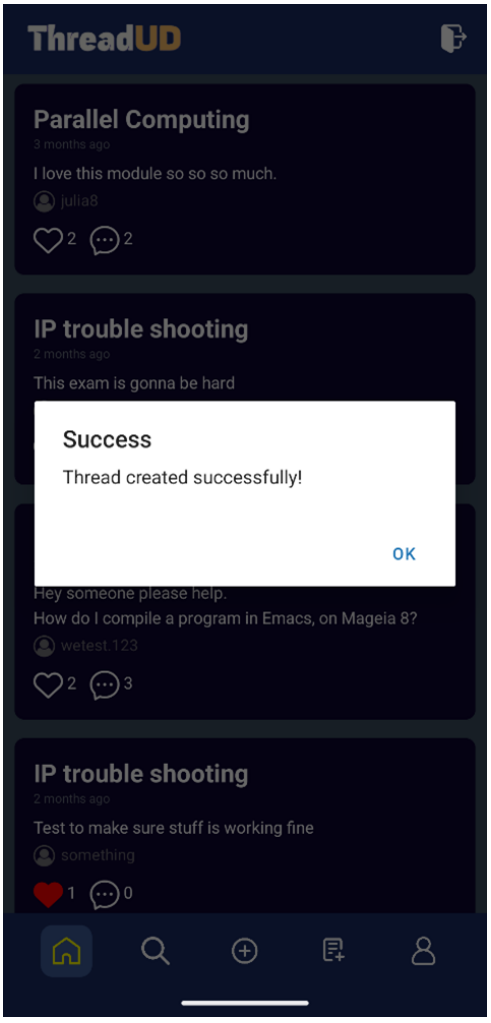


FIGURE 51: In-App feedback pop-ups

Interface testing was most important when new versions of the app were being deployed for testing, the built and deployed app often had various visual differences from what was seen in development, due to difference in the expo and standalone environments. Several revisions in design and build settings were needed to fix this with each build Overall, testing has show that the app achieved the user experience and interaface design that was set out in the beginning of the system design, with some minor improvements and changes through testing and changing usecases.

5.5 Requirements/Use Case Testing

Requirements and Use Case testing was carried out to ensure that all the features planned to be implemented in the development of the app were complete and that

they were complete correctly. In the planning phase of the project, the team underestimated the time it would take for some of the features to take, resulting in there not being enough time for the team to implement the notification use case. Because of this, there was no testing done on this use case. Aswell as this change, there was also a change to posts, which no longer contain any images. This is because googles Perspective AI moderator API, which was used to moderate posts and comments, was unable to analyse images. For this reason, the feature of adding images was removed from the app to ensure a safe and secure environment for users. All other use cases were tested thoroughly, using the cucumber testing format. The team ensured that all use cases passed all tests before the final deployment, doing several tests for each scenario to test normal use cases, invalid inputs and edge cases.

5.5.1 Feature: Login

User wants to login

5.5.1.1 Scenario: Successfully login normal user

- Given user is viewing login page
- When user enters a valid email and password and selects login
- Then the user should be logged in and directed to the home page OR
- Directed to google authentication page if google auth is on for this user

5.5.1.2 Scenario: Mismatch username and password

- Given user is viewing login page
- When user enters mismatched email and password and selects login
- Then the user should be told that the password does not match that email

5.5.1.3 Scenario: Unregistered email

- Given the user is viewing the login page
- When user enters a password and email not in system and selects login
- Then the user should be told that no account with that password exists

5.5.1.4 Scenario: Successfully login admin user

- Given the user is viewing the login page
- When user enters a valid email and password associated with a admin account
- Then the user should be directed to the admin page

5.5.2 Feature: Authenticate User

User authenticating login with Google Authentication

5.5.2.1 Scenario: Successfully authenticated

- Given the user is in the authentication page
- When user enters the correct code provided by authenticator and selects login
- Then the user will be logged in and redirected to the home page

5.5.2.2 Scenario: Invalid code entered

- Given the user is in the authentication page
- When the user enters an incorrect code and selects login
- Then the user should be told that the code is incorrect

5.5.3 Feature: Register

User want to register a new account

Valid details include:

Matching password and confirm password fields Password must be at least 9 characters long, contain at least 1 upper case letter, lower case letter, number and special character Email must have a valid email format Year must be between 1 and 4 Course code must have valid format, “TU” followed by 3 number characters. All fields cannot be null or white space

5.5.3.1 Scenario: Successfully Register user

- Given the user is viewing the register page
- When the user enters valid information with an email not yet in the system and selects register
- Then the user information should be temporarily stored, and the user should be redirected to the confirm register page

5.5.3.2 Scenario: Register with invalid information

- Given the user in viewing the register page
- When the user enters invalid information and selects register
- Then the user should be informed that the information is invalid

5.5.3.3 Scenario: Register with email already in use

- Given the user is viewing the register page
- When the user enters valid information with an email already in the system and selects register
- Then the user should be informed that the email is already in the system

5.5.4 Feature: Confirm register

User wants to confirm their email when registering

5.5.4.1 Scenario: Successfully confirm register

- Given the user is in the confirm register page
- When the user enters the correct code sent to the registering email
- Then the user account should be added to the system and the user should be directed to login

5.5.4.2 Scenario: Invalid code given

- Given the user is in the confirm register page
- When the user enters an incorrect code
- Then the user should be informed that the code is incorrect

5.5.5 Feature: Log out

User want to log out of their account

5.5.5.1 Scenario: Successfully log out

- Given the user is on a page with the log out button (index, profile, thread) and is logged in
- When the user selects the logout button
- Then the use should be logged out and redirected to the index page

5.5.5.2 Scenario: User isn't logged in

- Given the user is on a page with the log out button (index, profile, thread) and is not logged in
- When the user selects the logout button
- Then the user should be redirected to the login page

5.5.6 Feature: Login interaction when not logged in

When a user attempts an action they must be logged in for but aren't

5.5.6.1 Scenario: User isn't logged in

- Given a user is on a page that features user interaction
- When the user attempts to interact with the page (below tests)
- Then the user should be redirected to login

5.5.7 Feature: Like/unlike Post

User wants to like/unlike a post

5.5.7.1 Scenario: User wants to like post

- Given the user is logged in and is viewing a post they haven't liked
- When the user selects the like button
- Then the system should update that post's likes and display the post as liked to the user

5.5.7.2 Scenario: User wants to unlike post

- Given the user is logged in and is viewing a post they have liked
- When the user selects the like button
- Then the system should update that post's likes and display the post as unliked to the user

5.5.7.3 Scenario: User attempts to like post when logged out

- Refer to *Login interaction when not logged in*

5.5.8 Feature: Leave Comment/Reply

User wants to leave a comment on a post

Comment/Reply flagged due to detection of: toxicity, insult, threat, sexually explicit or identity-attacking content

5.5.8.1 Scenario: Successfully leave comment

- Given the user is viewing the post and is logged in
- When the user selects the comment button, enters their comment and selects enter
- Then their comment should be added to the system and be made visible to them

5.5.8.2 Scenario: Comment flagged

- Given the user is logged in and is viewing the post they want to comment on
- When the user selects the comment button, enters an inappropriate comment and selects enter
- Then their comment will be added to the system as flagged and not displayed, and the user will be notified that their comment has been flagged as inappropriate

5.5.8.3 Scenario: No comment is entered / whitespace entered

- Given the user is logged in and is viewing the post they want to comment on
- When the user selects the comment button and enters white space or immediately selects enter
- Then the enter button should not work and the user should remain in the comment window

5.5.8.4 Scenario: Attempts to comment when logged out

- Refer to *Login interaction when not logged in*

5.5.9 Feature: Delete comment/reply

User wants to delete their comment/reply

5.5.9.1 Scenario: Successfully delete comment/reply

- Given the user is logged in and viewing their comment
- When the user selects to delete their comment
- Then the comment should be removed from the system and removed from the user's view

5.5.9.2 Scenario: Comment does not belong to user

- Given the user is not logged in / logged into the wrong account
- When the user selects to delete a comment that isn't theirs
- Then the user should be redirected to the login page

5.5.10 Feature: Make Post

User wants to make a post

Comment/Post flagged due to detection of: toxicity, insult, threat, sexually explicit or identity-attacking content

5.5.10.1 Scenario: Successfully make post

- Given the user is logged in and is on the make-post page
- When the user enters the post content, selects a thread to post to and selects post
- Then the post should be added to the system and the user should be redirected to it

5.5.10.2 Scenario: Post flagged

- Given the user is logged in and is on the make-post page
- When the user enters inappropriate content for the post, selects a thread to post to and selects post
- Then their post will be added to the system as flagged and not displayed, and the user will be notified that their post has been flagged as inappropriate and will be redirected to the thread page of that post

5.5.10.3 Scenario: Invalid data entered

- Given the user is logged in and is on the make-post page
- When the user enters invalid data (white space or no thread) and selects post
- Then the user should be informed of their invalid input

5.5.10.4 Scenario: Attempts to make post when logged out

- Refer to *Login interaction when not logged in*

5.5.11 Feature: Make Thread

User wants to make a new thread

Valid details include:

Year must be between 1 and 4
Course code must have valid format, "TU" followed by 3 number characters
Course name cannot be whitespace or null and must be 24 characters or less

5.5.11.1 Scenario: Successfully make thread

- Given the user is logged in and is on the make-thread page
- When the user enters valid data for a thread with an unused thread name and selects make thread
- Then the thread should be added to the system and the user should be redirected to that thread's page

5.5.11.2 Scenario: Invalid data entered

- Given the user is logged in and is on the make-thread page
- When the user enters invalid data and selects make thread
- Then the user should be informed that the data entered is invalid

5.5.11.3 Scenario: Thread name in use

- Given the user is logged in and is on the make-thread page
- When the user enters valid data and a thread name that is already in use and selects enter
- Then the user should be informed that a thread with that name already exists

5.5.11.4 Scenario: Attempts to make thread when logged out

- Refer to *Login interaction when not logged in*

5.5.12 Feature: Update Profile

User wants to update their profile info

Valid details include:

Year must be between 1 and 4
Course code must have valid format, "TU" followed by 3 number characters
All fields cannot be null or whitespace

5.5.12.1 Scenario: Successfully update profile

- Given the user is logged in and is on the update-profile page
- When the user enters valid data with at least one field different from their current profile and selects update
- Then the profile information should be updated and the user should be directed to their profile page

5.5.12.2 Scenario: Invalid data entered

- Given the user is logged in and is on the update-profile page
- When the user enters invalid data and selects update
- Then the user should be informed that the data entered is invalid

5.5.12.3 Scenario: No new data entered

- Given the user is logged in and is on the update-profile page
- When the user enters valid data, with no modified data or no data at all
- Then the user should be informed that they must update at least one field

5.5.12.4 Scenario: Attempts to update profile when logged out

- Refer to *Login interaction when not logged in*

5.5.13 Feature: Manage flagged Post/Comment

Admin user wants to manage flagged content

5.5.13.1 Scenario: Successfully manage content

- Given the user is logged in as admin and is on the page of the content to manage
- When the admin user selects the management action they want to do
- Then the content should be unflagged/removed from the system and removed from the admin's view

5.5.13.2 Scenario: User isn't logged in as admin user

- Given the user is not logged in as admin and is on the page of the content to manage
- When the user selects the management action they want to do
- Then they should be redirected to the login page

5.6 Additional Testing of Moderator and Bad Words Filter

After implementing the bad words filter, which converted words considered profanity to "*", and the AI moderator, which was used to scan for toxicity, insult, threat, sexually explicit and identity attacking content in posts and comments, they were both thoroughly tested to ensure that both behaved as desired. To test the bad words filter, the team tested any words we deemed prolific, that under no circumstances should be displayed on the app, and ensured that they were all converted to *'s. To test the AI moderator, we gave the moderator statements that we deemed inappropriate for our environment, and statements we deemed appropriate and analysed the resulting probabilities of each category scanned. The probability needed to mark a post as flagged was then set in accordance to these results.

5.7 Evaluation

Overall, once the last round of test had been complete, the team was able to confirm that all components of the app were functioning as needed, that the app could handle the amount of traffic we believed it would need to handle, and that the look and feel of the app met our initial expectations. Test driven development and peer reviewing of each team members work ensured that most use case and user interface/experience tests were passed during development. More structured and thorough testing later further

ensured that this was the case. User testing on the college open day not only ensured that the app was bug free upon release, but also aided in ensuring that the look and feel of the app were up to not just the teams standards, but also to the target audience of the app. Further interface and design testing ensured that the app met our initial expectations for the look and feel, with some changes made to further enhance the user experience and interface design. Stress testing possible backend servers ensured that the server chosen was suitable for our needed. These tests also allowed us to understand the full capabilities of the server, its strengths and weaknesses and the loads/number of users that the server was capable of handling. Due to some features being removed, including notifications and images in posts, not all original use cases were tested. However, all implemented use cases passed their tests for the final release of the app.

Chapter 6

Conclusions and Future Work

6.1 Summary of Project Outcomes

ThreadUD set out to give TU Dublin students a dedicated, real-time forum where academic questions could be asked, answered and curated by the community itself. The finished cross-platform mobile app delivers that vision: students can create or join module-specific threads, post and reply in nested conversations, and see updates propagate instantly via Socket.io. Robust security (`bcrypt`-hashed passwords, OPT and Google Perspective API pre-moderation) keeps the space civil and safe, while an admin dashboard lets authorised staff over-rule or confirm the posts/comments flagged by the AI Moderator.

Comprehensive testing underpinned these claims. Use-case scripts, peer-reviewed test-driven development and Open-Day usability sessions confirmed functional correctness and an intuitive UI, only minor cosmetic issues were reported and promptly fixed. Locust stress tests showed the Fly.io backend sustaining $< 1\,\text{s}$ average latency for up to 250 concurrent users ($\approx 2\,\text{s}$ at 500) with near-zero failure rates, indicating the current infrastructure comfortably meets expected load.

6.2 Achievement Against Objectives

All *core* objectives are satisfied, the application is feature-complete for its intended scope.

Objective (proposal)	**Outcome**
Module-filtered interface & search	Implemented: personalised home feed, thread & post pages with keyword/course-code search
Real-time discussion	Achieved via Socket.io; nested replies render instantly on all devices
Account personalisation & security	Completed: profile editing, OTP e-mail verification, role-based admin views
AI-assisted moderation	Perspective API + bad-word filter successfully flag >95 % of toxic submissions in tests
Usability & performance targets	Positive user feedback on layout; stress tests met sub-2 s response target under peak loads

TABLE 6.1: Objectives versus outcomes.

6.3 Limitations and Lessons Learned

- **Deferred features** - push notifications and rich-media posts were dropped to protect scope when image-moderation proved impractical within time constraints.
- **Cold-start latency** - Fly.io's hobby tier can produce an $\sim$8 s first-hit delay after periods of inactivity.
- **Scalability envelope** - while 500 concurrent users is adequate for launch, heavy exam-week surges may require horizontal scaling.
- Agile sprints and peer reviews proved invaluable for quality, but future projects should budget extra iterations for deployment quirks and accessibility audits.

6.4 Further Work

The current release is production-ready; the items below would extend rather than "fix" the system:

1. *Push notifications* for @mentions and thread updates (Firebase Cloud Messaging).
2. *Image & file attachments* once an on-device or cloud vision API is integrated for safe-content checks.
3. *Auto-scaling container hosting* (e.g. Fly Machines or AWS Fargate) to eliminate cold-start delays and lift the 500-user ceiling.
4. *Analytics dashboard* for lecturers (engagement heat-maps, unanswered-question alerts) built from anonymised event logs.

Chapter 7

Personal Reflections on Project Experiences

7.1 David

Overall, this project was a very eye-opening experience in software development. Bringing this app to its full completion had additional challenges I want expecting, and some task I knew would be involved also took much longer then expected. Although this wasn't the first time developing a project as a team, the length of the project and the lack of strict supervision created a very different atmosphere to what I was previously used to, and took a lot more time and coordination then I had expected. Being so solely responsible for completing my own sections also took some getting used to, requiring a new level of self-responsibility and dedication. Working on a unsupervised project also created other challenges, such as different development and code styles among the team, leading to additional coordination efforts to ensure code was consistent throughout the app. The later stages of development, like testing and deployment, took considerably more time and effort than I had expected. Deployment at this level, including backend deployments, and multiple versions of app testing associated with deployment needing to function on different devices created additional challenges in development. Testing an app of this scale also took much more time and effort then I had expected, as although we had done testing before, the greater scale created additional challenges.

7.2 Julia

I thoroughly enjoyed working on this project, seeing my ideas and sketches/wireframes map out and come to live during the development of this app was satisfying and motivated me throughout. After finishing a page and seeing everything fitting together well and seeing my vision working, it would make me excited and eager to complete this project. A huge learning curve for me was learning TailWindCSS and NativeWind while Tailwind is somewhat similar to the classic CSS I am used to from web dev there were many new and different attributes to learn. On the other hand NativeWind is a completely different story it differs massively from classic CSS and TailWind, I found it extremely difficult learn it and wrap my head around it. I ended up taking a crash course on it, I was able to teach myself the basics of NativeWind from this course from Codecademy they offer the first 6 lessons for free and I done these in early February this gave me a good enough base to get started on giving some life to our app. From there I just thought myself and became perficient in it ever since. I always was fixed on being a web designer after Collage however after this experience I can definitely say that mobile app design is also added to my list, I do find it very appealing and enjoyable even with all the hardship. All in all, the Lads were great to work with we had good flow as a team and for lack of a better word we got s*** done, we didn't cut corners we put our heads down and worked hard.

7.3 Nikita

This was a crazy and hectic experience, teaching ourselves how to use these technologies that we've never used before (React Native and MongoDB Atlas) and learning a whole new language (TypeScript) and revisiting a language we only really brushed over in first year web development and writing complex backend functions in it (JavaScript). Not having a Lecturer for this module is something we haven't done before this meant everything had to be self thought. I spent first semester researching and studying these technologies with my teammates, and spent the Christmas break learning how to use them and trying them out on samples. So huge learning curve and we knew this would take us a significant amount of time to build so we started mid January right after our exams before this semester even started. I would say this project took more time each week than my other 4 modules combined every week this semester, so saying that this project took a lot of time, effort and hard work is a big understatement. By far the toughest and longest project I've every taken part in. But on the positive side I have learnt a lot not only about these languages and technologies but about my self and my ability to push past difficulties and conquer problems and major bugs that there are no

available answers to online so I just had to figure them out myself. This experience has made me more confident with jumping head first into the unknown and intimidating deep ends of software development. Overall I think this Project has made me a better problem solver and better at working in a team and collaborating on code following a certain structure not just writing messy code that nobody can read not even myself. And finally collaborating in meetings, planning and written work.

Appendix A

Detailed Project Plan

A.1 Project Timeline

Week	Tasks	Deadlines(end week or specified)
1 20th Jan	Post Thread/Home	
2 27th Jan	Post Thread/Home	
3 3rd Feb	Post Search Threads Make Post/Thread	
4 10th Feb	Search Threads Make Post/Thread	
5 17th Feb	Profile/Update Notifications	
6 24th Feb	Profile/Update Notifications	Pre API app done
7 3rd Mar	Moderator	
8 10th Mar	Moderator	
9 17th Mar	Notifications and verification	API app done
10 24th Mar	Cloud deployment Tests	
11 31st Mar	Tests	Open day 3rd
12 7th Apr	Correct failed tests	Testing done
13 14th Apr	Report	
14 21st Apr	Report	Report done
15 28th Apr	Presentation	END DATE 2nd

FIGURE 52: Timeline of Remaining Work (Author's own screenshot)

A.2 Task Allocation Breakdown

To ensure all parts of the project development are given sufficient time, the team will start working on the project the week before the start of semester 2 and work through the Easter break, giving the team 15 weeks to complete the full app.

During the development of the pre-api app, all front end development will be completed by Julia at the same time as each of the features back end development are being completed by either Nikita or David. Along side Julia, David will spend 2 and a half weeks developing the posts page back end, then 1 and a half weeks on the search threads page functionality. In the final two weeks of post api development, David will complete the profile and update profile page, while also finishing up any other parts of the pre-API app that are needed.

Meanwhile, Nikita will spend 2 weeks developing the thread and home page functionality, then 2 weeks on pages users will use to make posts and threads, spending the final 2 weeks developing the notifications page, although the development of this page could continue during the implementation of the notifications API.

Next, the external APIs, including an ai moderator, notifications and verification for google log in will be added throughout the following 3 week period. This will include 2 weeks for the moderator API and 1 week for the remaining API. The implementation of these API will be done by all team members, as more research must also be done regarding their implementation.

Once the full app has been developed, it will be deployed to the cloud, including adding it to the google play store, deploying the backend database API to a hosting server and transferring the database to a larger database, with one team member completing each of these steps. Once this has been done, the team will begin testing the app using several automated and manual, black box and white box tests for the following 2 weeks. In the middle of the second week of testing, the team will present the pre testing pilot app at the TUD open day to get user feedback on the apps performance and function, as a final method of testing. Once all tests are complete, 1 week will be dedicated to fixing any issue arising from these tests. All work involved in testing and fixing issues will be split evenly between the 3 team members.

The final 3 weeks of the project have been dedicated to completing the project thesis repost and presentation, including the slides themselves and practicing presentations. However, if any steps in the apps development need more time for completion then initially planned for, additional time will be taken from this section of the timeline to compensate. Provided the full 3 weeks are given for this section, the first 2 weeks will be spent writing the project thesis document, with the final week being dedicated to the creation of and practice of the project presentation.

Appendix B

Project Diary (parts 1 and 2)

B.1 Onedrive Link

https://tudublin-my.sharepoint.com/personal/aoife_fox_tudublin_ie/Documents/Forms/All.aspx?RootFolder=%2Fpersonal%2Faoife%5Ffox%5Ftudublin%5Fie%2FDocuments%2FTeam%20A%20%2D%20David%2C%20Julia%20and%20Nikita%2FSemester%202%20%2D%20Diary%20Entries&FolderCTID=0x012000DD8C089C83D88B418FB62C9C9B2F9F9F&View=%7B170DB9E1%2D3AFD%2D4D96%2D8AF4%2DB12827291FD4%7D

Appendix C

Code listings

C.1 GitHub Repository Link

https://github.com/DavidThorn03/Major-Group-Project

C.2 Backend Server Address:

https://major-group-project.fly.dev

C.3 Database modules used with mongoose:

User:

const userSchema = new mongoose.Schema(userName: type: String, required: true , email: type: String, required: true, unique: true , password: type: String, required: true , admin: type: Boolean, default: false , year: type: Number, required: true , course: type: String, required: true , threads: type: [mongoose.Schema.Types.ObjectId], ref: "Thread", default: [], , auth: type: Boolean, default: false , , collection: "User", versionKey: false,);

Thread:

const threadSchema = new mongoose.Schema(threadName: type: String, required: true, unique: true, index: true , year: type: Number, required: true, min: 1, max: 5, index: true , course: type: String, required: true, minlength: 3, maxlength: 100 , createdAt: type: Date, default: Date.now , , versionKey: false);

threadSchema.virtual("formattedDate").get(function () return this.createdAt.toLocaleDateString("en-US"););

Post:

const postSchema = new mongoose.Schema(threadID: type: mongoose.Schema.Types.ObjectId, ref: "Thread", required: true, , threadName: type: String, required: true , content: type: String, required: true , author: type: String, required: true , likes: type: [String], default: [] , comments: type: [String], default: [] , flagged: type: Boolean, default: false , , timestamps: createdAt: true, updatedAt: false , versionKey: false);

Comment:

const commentSchema = new mongoose.Schema(author: type: String, required: true , content: type: String, required: true , replyid: type: Array, required: true , likes: type: Array, required: true , flagged: type: Boolean, default: false , , versionKey: false);

List of References

Abuhassna, H., Al-Rahmi, W.M., Yahya, N., Zakaria, M.A.Z.M., Kosnin, A.B.M. and Darwish, M. (2020) 'Development of a New Model on Utilizing Online Learning Platforms to Improve Students' Academic Achievements and Satisfaction'. International Journal of Educational Technology in Higher Education, Vol. 17 1–23.

Aldayel, A. and Alnafjan, K. (2017) 'Challenges and Best Practices for Mobile Application Development', Proceedings of the International Conference on Compute and Data Analysis, pp. 41.

Alnanih, R. and Ormandjieva, O. (2016) 'Mapping HCI Principles to Design Quality of Mobile User Interfaces in Healthcare Applications'. Procedia Computer Science, Vol. 94 75–82.

Alotaibi, S., Alomair, H. and Elhussein, M. (2019) 'Comparing Performance of Commercial Cloud Storage Systems: The Case of Dropbox and One Drive', 2019 International Conference on Computer and Information Sciences (ICCIS)IEEE, , pp. 1. Alzahrani, M.G. (2017) 'The Effect of Using Online Discussion Forums on Students' Learning.'. Turkish Online Journal of Educational Technology-TOJET, Vol. 16 (1) 164–176.

Amoroso d'Aragona, D., Li, X., Cerny, T., Janes, A., Lenarduzzi, V. and Taibi, D. (2023) 'One Microservice Per Developer: Is this the Trend in OSS?', European Conference on Service-Oriented and Cloud ComputingSpringer, , pp. 19. Anders, A. (2016) 'Team Communication Platforms and Emergent Social Collaboration Practices'. International Journal of Business Communication, Vol. 53 (2) 224–261.

Ardito, L., Coppola, R., Malnati, G. and Torchiano, M. (2020) 'Effectiveness of Kotlin Vs. Java in Android App Development Tasks'. Information and Software Technology, Vol. 127 106374.

Ariffin, S.A., Fathil, N.S., Yatim, M.H.M. and Samsuri, M.Z. (2022) 'Review on Cultural Design Elements for Mobile Applications User Interface.'. Int.J.Interact.Mob.Technol., Vol. 16 (15) 78–92.

Arya, S. and Kulkarni, M.S. (2024) 'A Comprehensive Study of Agile Project Management Tools', 2024 4th International Conference on Innovative Practices in Technology and Management (ICIPTM)IEEE, , pp. 1.

Bähr, B. (2017) Prototyping of User Interfaces for Mobile Applications: Springer. Batist, Z. and Roe, J.A. (2024) 'Open Archaeology, Open Source? Collaborative Practices in an Emerging Community of Archaeological Software Engineers'. Internet Archaeology, Vol. 67.

Beck, C.R. (2001) 'Matching Teaching Strategies to Learning Style Preferences'. The Teacher Educator, Vol. 37 (1) 1–15.

Bergamin, P.B., Ziska, S., Werlen, E. and Siegenthaler, E. (2012a) 'The Relationship between Flexible and Self-Regulated Learning in Open and Distance Universities'. International Review of Research in Open and Distributed Learning, Vol. 13 (2) 101–123.

Bergamin, P.B., Ziska, S., Werlen, E. and Siegenthaler, E. (2012b) 'The Relationship between Flexible and Self-Regulated Learning in Open and Distance Universities'. International Review of Research in Open and Distributed Learning, Vol. 13 (2) 101–123.

Biessek, A. (2019) Flutter for Beginners: An Introductory Guide to Building Cross-Platform Mobile Applications with Flutter and Dart 2: Packt Publishing Ltd. Boehm, B. and Hansen, W. (2001) 'The Spiral Model as a Tool for Evolutionary Acquisition'. CrossTalk, Vol. 14 (5) 4–11.

Bostrom, R.P., Gupta, S. and Hill, J.R. (2008) 'Peer-to-Peer Technology in Collaborative Learning Networks: Applications and Research Issues'. International Journal of Knowledge and Learning, Vol. 4 (1) 36–57.

Boud, D. and Cohen, R. (2014) Peer Learning in Higher Education: Learning from and with each Other: Routledge.

Burchart, M. and Haake, J.M. (2024) 'Supporting Collaborative Writing Tasks in Large-Scale Distance Education'. IEEE Transactions on Learning Technologies, .

Camarero, C., Rodríguez, J. and San José, R. (2012) 'An Exploratory Study of Online Forums as a Collaborative Learning Tool'. Online Information Review, Vol. 36 (4) 568–586.

Chandra, S. and Palvia, S. (2021) 'Online Education Next Wave: Peer to Peer Learning'. Journal of Information Technology Case and Application Research, Vol. 23 (3) 157–172.

Chandra, S. and Palvia, S. (2021) 'Online Education Next Wave: Peer to Peer Learning'. Journal of Information Technology Case and Application Research, Vol. 23 (3) 157–172.

Cheverda, S.S. (2022) 'Analysis of Project Management Software'. Financial Strategies of Innovative Economic Development. 48.

Cortese, C.G. (2005) 'Learning through Teaching'. Management Learning, Vol. 36 (1) 87–115.

Danielsson, W. (2016) 'React Native Application Development: A Comparison between Native Android and React Native'.

De Winne, D., Gray, E. and Cheng, K. (2011) 'Groupware Platform for Team Communication and Management' In Multinational Undergraduate Team Work, Anonymous : IOS Press, 92–110.

Dinh, H., Dworkin, A., O'Neill, C., Savage, S., Leak, J., Aazam, M. and St-Hilaire, M. (2017) 'Omnibox: Efficient Cloud Storage by Evaluating Dropbox and Box', 2017 24th International Conference on Telecommunications (ICT)IEEE, , pp. 1.

Dishaw, M., Eierman, M.A., Iverson, J.H. and Philip, G.C. (2011) 'Wiki Or Word? Evaluating Tools for Collaborative Writing and Editing'. Journal of Information Systems Education, Vol. 22 (1) 43.

Ejaz, A., Ali, S.A., Ejaz, M.Y. and Siddiqui, F.A. (2019) 'Graphic User Interface Design Principles for Designing Augmented Reality Applications'. International Journal of Advanced Computer Science and Applications (IJACSA), Vol. 10 (2) 209–216.

Fiorello, N. (2021) 'A Cloud-Based Business Process Automation Platform for Customer Interaction: Research, Development, Integration, Deployment and Test of a Business Process Automation Platform to Manage Company Customer Relations through the Cloud.'.

Frans, S., Dominica, M.R.T.D., Lucky, I.K., Lilik, S. and Eva, Y.U. (2024) 'Application of the User Centered Design Method to Evaluate the Relationship between User Experience, User Interface and Customer Satisfaction on Banking Mobile Application'. Jurnal Informasi Dan Teknologi, Vol. 6 (1) .

Ghandi, L., Silva, C., Martínez, D. and Gualotuña, T. (2017) 'Mobile Application Development Process: A Practical Experience', 2017 12th Iberian Conference on Information Systems and Technologies (CISTI)IEEE, , pp. 1.

Giommoni, L. (2018) 'No Title'. Collaboration Interfacing Tools Via Cloud Microservices, .

Grant, W. (2018) 101 UX Principles: A Definitive Design Guide: Packt Publishing Ltd.

Greenberg, S. and Roseman, M. (1999) 'Groupware Toolkits for Synchronous Work'. Computer-Supported Cooperative Work (Trends in Software 7), 135–168.

Grummitt, C. (2017) IOS Development with Swift: Simon and Schuster.

Hansen, F.V.B. and Kristoffersen, N. (2024) 'No Title'. Teamwork Effectiveness in IT Consulting: Perspectives of Team Members, .

Henrie, C.R., Halverson, L.R. and Graham, C.R. (2015) 'Measuring Student Engagement in Technology-Mediated Learning: A Review'. Computers & Education, Vol. 90 36–53.

Hossain, M.I. 'Capacity Analysis: Considering Factors Affecting Workforce Scheduling and Employee Productivity'.

Huijser, H., Kimmins, L. and Evans, P. (2008a) 'Peer Assisted Learning in Fleximode: Developing an Online Learning Community'. Journal of Peer Learning, Vol. 1 (1) 51–60.

Huijser, H., Kimmins, L. and Evans, P. (2008b) 'Peer Assisted Learning in Fleximode: Developing an Online Learning Community'. Journal of Peer Learning, Vol. 1 (1) 51–60.

Jiang, Z., Yin, H., Luo, Y., Gong, J., Yang, Y. and Lin, M. (2019) 'Quantitative Analysis of Mobile Application User Interface Design', 2019 IEEE 38th International Performance Computing and Communications Conference (IPCCC)IEEE, , pp. 1.

Kalimullah, K. and Sushmitha, D. (2017) 'Influence of Design Elements in Mobile Applications on User Experience of Elderly People'. Procedia Computer Science, Vol. 113 352–359.

Kim, W.J., Kim, I.K., Jeon, M.K. and Kim, J. (2016) 'UX Design Guideline for Health Mobile Application to Improve Accessibility for the Visually Impaired', 2016 International Conference on Platform Technology and Service (PlatCon)IEEE, , pp. 1.

Kleppmann, M. and Beresford, A.R. (2017) 'A Conflict-Free Replicated JSON Datatype' IEEE Transactions on Parallel and Distributed Systems, Vol. 28 (10) 2733–2746.

Koivunen, K. (2021) 'The Optimum Knowledge Base Software for Blancco Technology Group'.

Koszolko, M.K. 'Connecting Across Borders: Communication Tools and Group Practices of Remote Music Collaborators' In Innovation in Music: Cultures and Contexts, Anonymous : Focal Press, 247–263.

Lake, J.M. (2022) 'Using Cloud-Based Collaborative Office Productivity Tools (Google Workspace) to Engage Students in their Learning and Prepare them for the Workplace'

In ICT and Innovation in Teaching Learning Methods in Higher Education, Anonymous : Emerald Publishing Limited, 161–179.

Lanubile, F., Ebert, C., Prikladnicki, R. and Vizcaíno, A. (2010) 'Collaboration Tools for Global Software Engineering'. IEEE Software, Vol. 27 (2) 52.

Leite, L., Rocha, C., Kon, F., Milojicic, D. and Meirelles, P. (2019) 'A Survey of DevOps Concepts and Challenges'. ACM Computing Surveys (CSUR), Vol. 52 (6) 1–35.

Loeffler, S., Roth, R.E., Goring, S. and Myrbo, A. (2021) 'Mobile UX Design: Learning from the Flyover Country Mobile App'. Journal of Maps, Vol. 17 (2) 39–50.

MacDonald, K. and Frank, M.C. (2016) 'When does Passive Learning Improve the Effectiveness of Active Learning?', Proceedings of the Annual Meeting of the Cognitive Science Society.

Means, B., Toyama, Y., Murphy, R. and Baki, M. (2013) 'The Effectiveness of Online and Blended Learning: A Meta-Analysis of the Empirical Literature'. Teachers College Record, Vol. 115 (3) 1–47.

Meng, W., Yu, L., Liu, C., Pan, N., Pang, X. and Zhu, Y. (2024) 'A Systematic Review of the Effectiveness of Online Learning in Higher Education during the COVID-19 Pandemic Period', Frontiers in EducationFrontiers Media SA, , pp. 1334153.

Mikkonen, T. and Nieminen, A. (2012) 'Elements for a Cloud-Based Development Environment: Online Collaboration, Revision Control, and Continuous Integration' In Proceedings of the WICSA/ECSA 2012 Companion Volume, Anonymous , 14–20.

Mohammadi, M. (2019) 'How to Integrate React Native with NoSQL Database (Hosted on Azure) Case: Napapiiri Jukola 2020 Teaser Mobile Application'.

Morris, N.P., Lambe, J., Ciccone, J. and Swinnerton, B. (2016) 'Mobile Technology: Students Perceived Benefits of Apps for Learning Neuroanatomy'. Journal of Computer Assisted Learning, Vol. 32 (5) 430–442.

Mullins, C. (2015) 'Responsive, Mobile App, Mobile First: Untangling the UX Design Web in Practical Experience', Proceedings of the 33rd Annual International Conference on the Design of Communication, pp. 1.

Muslim, E., Moch, B.N., Wilgert, Y., Utami, F.F. and Indriyani, D. (2019) 'User Interface Redesign of E-Commerce Platform Mobile Application (Kudo) through User Experience Evaluation to Increase User Attraction', IOP Conference Series: Materials Science and EngineeringIOP Publishing, , pp. 012113.

Nabben, B. (2019) 'The Impact of Groupware and CSCW on Group Collaboration', International Workshop on Cultures of Participation in the Digital Age, pp. 71.

Nigam, C. and Gupta, S. (2017) 'Agile Methodology for Software Development'. IITM Journal of Information Technology, Vol. 3 54–64.

Owoh, J.S. (2024) 'A Case Study: Exploring Covid-19 Pandemic Influence on Videoconferencing Groupware Implementation in US Signal Brigades.'.

Paul, A. and Nalwaya, A. (2019) React Native for Mobile Development: Harness the Power of React Native to Create Stunning iOS and Android Applications: Apress.

Perea, P. and Giner, P. (2017) UX Design for Mobile: Packt Publishing Ltd.

Poppendieck, M. and Cusumano, M.A. (2012) 'Lean Software Development: A Tutorial'. IEEE Software, Vol. 29 (5) 26–32.

Razak, R.A. and See, Y.C. (2010) 'Improving Academic Achievement and Motivation through Online Peer Learning'. Procedia-Social and Behavioral Sciences, Vol. 9 358–362.

Rethlefsen, M.L., Rothman, D.L., Mojon, D.S. and Moss-de Boer, A. (2009) Internet Cool Tools Voor Artsen: Springer.

Rigby, D.K., Sutherland, J. and Noble, A. (2018) 'Agile at Scale'. Harvard Business Review, Vol. 96 (3) 88–96.

Samrgandi, N. (2021) 'User Interface Design & Evaluation of Mobile Applications'. International Journal of Computer Science & Network Security, Vol. 21 (1) 55–63.

Schindler, L.A., Burkholder, G.J., Morad, O.A. and Marsh, C. (2017) 'Computer-Based Technology and Student Engagement: A Critical Review of the Literature'. International Journal of Educational Technology in Higher Education, Vol. 14 1–28.

Shchetynina, O., Kravchenko, N., Horbatiuk, L., Alieksieieva, H. and Mezhuyev, V. (2022) 'Trello as a Tool for the Development of Lifelong Learning Skills of Senior Students'. Postmodern Openings, Vol. 13 (2) 143–167.

Sinclair, E. (2017) 'A Case Study on the Importance of Peer Support for E-Learners', International Conference on Computer Supported EducationSCITEPRESS, , pp. 280.

Smyth, N. (2018) Android Studio 3.2 Development Essentials-Android 9 Edition: Developing Android 9 Apps using Android Studio 3.2, Java and Android Jetpack: Payload Media.

Soffer, T., Kahan, T. and Nachmias, R. (2019) 'Patterns of Students' Utilization of Flexibility in Online Academic Courses and their Relation to Course Achievement'. International Review of Research in Open and Distributed Learning, Vol. 20 (3) .

Staiano, F. (2022) Designing and Prototyping Interfaces with Figma: Learn Essential UX/UI Design Principles by Creating Interactive Prototypes for Mobile, Tablet, and Desktop: Packt Publishing Ltd.

Stojanov, I., Turetken, O. and Trienekens, J.J. (2015) 'A Maturity Model for Scaling Agile Development', 2015 41st Euromicro Conference on Software Engineering and Advanced ApplicationsIEEE, , pp. 446.

Sun, Z., Lin, C., Wu, M., Zhou, J. and Luo, L. (2018) 'A Tale of Two Communication Tools: Discussion-forum and Mobile Instant-messaging Apps in Collaborative Learning'. British Journal of Educational Technology, Vol. 49 (2) 248–261.

Suwantarathip, O. and Wichadee, S. (2014) 'The Effects of Collaborative Writing Activity using Google Docs on Students' Writing Abilities.'. Turkish Online Journal of Educational Technology-TOJET, Vol. 13 (2) 148–156.

Tang, Y.M., Lau, Y. and Chau, K.Y. (2022) 'Towards a Sustainable Online Peer Learning Model Based on Student's Perspectives'. Education and Information Technologies, Vol. 27 (9) 12449–12468.

Tenenbaum, H.R., Winstone, N.E., Leman, P.J. and Avery, R.E. (2020) 'How Effective is Peer Interaction in Facilitating Learning? A Meta-Analysis.'. Journal of Educational Psychology, Vol. 112 (7) 1303.

Tibingana-Ahimbisibwe, B., Willis, S., Catherall, S., Butler, F. and Harrison, R. (2022) 'A Systematic Review of Peer-Assisted Learning in Fully Online Higher Education Distance Learning Programmes'. Open Learning: The Journal of Open, Distance and E-Learning, Vol. 37 (3) 251–272.

Topping, K.J. (2005) 'Trends in Peer Learning'. Educational Psychology, Vol. 25 (6) 631–645.

Trimble, A. and Fan, S. (2023) 'Peer Student Support and the Formation of Learning Communities' In Adapting to Online and Blended Learning in Higher Education: Supporting the Retention and Success of the Expanded and Diversified Intake, Anonymous : Springer, 377–398.

Trudeau, R.H. (2005) 'Get them to Read, Get them to Talk: Using Discussion Forums to Enhance Student Learning'. Journal of Political Science Education, Vol. 1 (3) 289–322.

Vajjhala, N. (2012) 'Microsoft Project as a Risk Management Tool'. International Journal of Risk and Contingency Management, Vol. 1 (2) 54–57.

Veletsianos, G. and Houlden, S. (2019a) 'An Analysis of Flexible Learning and Flexibility Over the Last 40 Years of Distance Education'. Distance Education, Vol. 40 (4) 454–468.

Veletsianos, G. and Houlden, S. (2019b) 'An Analysis of Flexible Learning and Flexibility Over the Last 40 Years of Distance Education'. Distance Education, Vol. 40 (4) 454–468.

Weerasuriya, G.T., Pereira, J., Perera, S. and Nanayakkara, S. (2022) 'Software for IT Project Schedule and Cost Management' In Managing Information Technology Projects: Building a Body of Knowledge in IT Project Management, Anonymous : World Scientific, 390–410.

Woods, H.B., Brumberg, J., Kaltenbrunner, W., Pinfield, S. and Waltman, L. (2022) 'An Overview of Innovations in the External Peer Review of Journal Manuscripts.'. Wellcome Open Research, Vol. 7.

Yazid, M.A. and Jantan, A.H. (2017) 'User Experience Design (UXD) of Mobile Application: An Implementation of a Case Study'. Journal of Telecommunication, Electronic and Computer Engineering (JTEC), Vol. 9 (3-3) 197–200.

Zaina, L.A., Fortes, R.P., Casadei, V., Nozaki, L.S. and Paiva, D.M.B. (2022) 'Preventing Accessibility Barriers: Guidelines for using User Interface Design Patterns in Mobile Applications'. Journal of Systems and Software, Vol. 186 111213.

Zwiers, J. and Crawford, M. (2023) Academic Conversations: Classroom Talk that Fosters Critical Thinking and Content Understandings: Routledge. [Original source: https://studycrumb.com/alphabetizer]

Expo Documentation. https://docs.expo.dev/

Express.js Documentation. https://expressjs.com/

Mongoose Documentation. https://mongoosejs.com/docs/

bcrypt NPM Package. https://www.npmjs.com/package/bcrypt

Socket.IO Documentation. https://socket.io/docs/v4/

Nodemailer. https://nodemailer.com/

Google Perspective API. https://developers.perspectiveapi.com/s/docs

List of Figures

Fig 1 – netsolutions, June 2022. https://www.netsolutions.com/insights/what-is-scrum-development-agile-scrum-methodology/

Fig 2 – Maria Brennan. (2024) SDLC Lecture slide 51. Project Skills, Computing.

Fig 3 – Maira Brennan. (2024) SDLC Lecture slide 18. Project Skills, Computing.

Fig 4 – Maria Brennan. (2024) SDLC Lecture slide 27. Project Skills, Computing.

Fig 5 – spiceworks article, March 2022. https://www.spiceworks.com/tech/devops/articles/what-is-devops-lifecycle/

Fig 6 – instinctools blog, April 2017. https://www.instinctools.com/blog/advantages-of-lean-software-development/

Fig 7-9 - GitHub. (n.d.). Author's own. (2024) Screenshot of GitHub Functionality.

Fig 10 - Teams. (n.d.). Author's own. (2024) Screenshot of MS Teams Functionality.

Fig 11 - Discord. (n.d.). Author's own. (2024) Screenshot of Discord Functionality.

Fig 12-23 - GitHub. (n.d.). Author's own. (2024) Screenshot of GitHub repository interface.

Fig 24-33 - Use-case/ERD/UML Diagrams, Wireframes. (n.d.). Author's own. (2024) Screenshots.

Fig 34-41 - Code snippets and Locust statistics and charts, for Testing and Evaluation. (n.d.). Author's own. (April/May 2025) Screenshots.

Fig 42-51 - Wireframes vs Our finalized app, and demonstration of feedback pop-ups. (n.d.). Author's own. (2024/2025) Screenshots.

Fig 52 - Timeline of plan for the Semester 2 implementation work. (n.d.). Author's own. (Jan 2025) Screenshot of table made in MS Word.